# HelmX: An AI-Powered Smart Helmet

*Edge Crash Detection, Driver-State Monitoring and Connected Rider Services*

**Final Year Project Report**

Session 2025–2026

A Final Year Project submitted in partial fulfilment of the requirements for the degree of

**Bachelor of Science in Computer Science**


| **Student** | **Registration No.** |
|---|---|
| Usman Azfar (Group Leader) | BCSF22M512 |
| Hamza Ahmad | BCSF22M536 |
| Abdul Hanan | BCSF22M505 |
| Awais Imtiaz | BCSF22M541 |

Supervised by: **Dr. Tayyaba Tariq**

**Department of Computer Science**

Faculty of Computing and Information Technology

**University of the Punjab, Lahore, Pakistan**

# Project Details

| | |
|---|---|
| **Project ID (office use)** | |
| **Type of project** | ☐ Traditional ☒ Industrial ☐ Continuing |
| **Nature of project** | ☐ Development ☒ Research & Development |
| **Sustainable Development Goals** | ☒ Good Health and Well-Being ☐ Quality Education<br>☒ Industry, Innovation and Infrastructure ☐ Gender Equality<br>☒ Decent Work and Economic Growth ☐ Climate Action |
| **Area of specialization** | ☒ Artificial Intelligence ☐ Blockchain ☐ Cybersecurity<br>☒ Data Science and Analytics ☐ Game Development<br>☒ Internet of Things ☒ Natural Language Processing<br>☒ Mobile App Development ☒ Web Development |

## Project group members

| Sr. | Role | Student name | Registration No. | Email | Signature |
|---|---|---|---|---|---|
| 1 | Group leader | Usman Azfar | BCSF22M512 | bcsf22m512@pucit.edu.pk | |
| 2 | Member | Abdul Hanan | BCSF22M505 | bcsf22m505@pucit.edu.pk | |
| 3 | Member | Hamza Ahmad | BCSF22M536 | bcsf22m536@pucit.edu.pk | |
| 4 | Member | Awais Imtiaz | BCSF22M541 | bcsf22m541@pucit.edu.pk | |

**Declaration:** The candidates confirm that the work submitted is their own and that appropriate credit has been given where reference has been made to the work of others.

# Plagiarism-Free Certificate

This is to certify that I, Usman Azfar, son of Azfar Shafiq, group leader of the Final Year Project under registration number BCSF22M512 at the Department of Computer Science, University of the Punjab, Lahore, declare that my FYP proposal has been checked by my supervisor and that its similarity index is _____ %, which is less than 20%, the acceptable limit set by the HEC. The originality report is attached as Appendix A.

| | | | |
|---|---|---|---|
| **Date:** | _____ | **Group leader:** | _____ |
| **Signature:** | _____ | | |
| **Supervisor:** | _____ | **Co-supervisor (if any):** | _____ |
| **Designation:** | _____ | **Designation:** | _____ |
| **Signature:** | _____ | **Signature:** | _____ |
| **Head of Department:** | _____ | **Signature:** | _____ |

# Abstract

Motorcycle riders are among the most vulnerable road users, and help after a crash often arrives late. HelmX is a research prototype of an affordable smart helmet built around a Raspberry Pi 4, developed as a Final Year Project at the University of the Punjab. This report documents five independently developed and evaluated components and is explicit about the evidence level of each.

The principal research contribution is a hybrid Conv1D crash-detection model that exposes a supervised crash probability and a normal-only reconstruction error. It was trained on 66 sessions from five third-party and simulated sources using session-level splits. On the frozen held-out test set the classifier reached ROC-AUC 0.946 and PR-AUC 0.539. A five-seed ablation showed that a supervised-only baseline was stronger in-distribution (mean PR-AUC 0.581 versus 0.527), whereas reconstruction error remained informative under an excluded simulated crash mechanism (ROC-AUC 0.868–0.873 versus 0.352 for the classifier). The 83,568-byte INT8 model was deployed on a Raspberry Pi 4 with a live-sensor p99 processing latency of 3.25 ms in a 90-minute soak and 3.03 ms in a 180-minute soak. Composed deployment-path decisions agreed with the reference on 99.65% of all windows but only 90.21% of crash-like windows, and the deployed policy produced about 14 false alerts per hour on independent e-scooter data.

A personalized, quality-gated MobileNetV3-Small/physiology/causal temporal-convolution drowsiness model achieved a subject-disjoint development macro-F1 of 0.642 on UTA-RLDD; full-video causal replay detected 2 of 12 drowsy-labelled clips at the committed warning threshold. The remaining components are a native Kotlin Android companion application, a Next.js product website with a 3D configurator and Stripe test-mode checkout, and Raspberry Pi services that include an offline Rhasspy voice assistant. External drowsiness validation, end-to-end SMS delivery and a Pi-to-phone safety-flag link have not been demonstrated; HelmX is therefore presented as a research prototype, not a certified safety device.

**Keywords:** smart helmet; crash detection; drowsiness detection; edge AI; Raspberry Pi; TensorFlow Lite; Bluetooth Low Energy; Android; deployment fidelity

# Acknowledgements

We thank our supervisor, Dr. Tayyaba Tariq, for her guidance and feedback throughout this project, and the Department of Computer Science, Faculty of Computing and Information Technology, University of the Punjab, for its support. We also acknowledge the creators of the public datasets and open-source tools on which this work depends, including UTA-RLDD, MRL Eye, MediaPipe, TensorFlow, PyTorch, OpenStreetMap, MapLibre, OSRM, Photon and Rhasspy.

# About This Report

This report documents the HelmX system as implemented and evaluated. Its results are based on the following artefact versions: the frozen crash-detection reproducibility release v1.1.0 (Zenodo, 2 November 2026), the drowsiness-research verification snapshot of 14 October 2026, the Android application at commit bce53f7, the web platform at commit 3b4be8e and the Raspberry Pi configuration snapshot of October 2026 (see Appendix B). Sections inherited from the original proposal are retained where they describe planned scope, and each requirement and test case states whether it was implemented, executed or remains proposed.

# Table of Contents

**1 Introduction ..... 1**
1.1 Background and Motivation ..... 1
1.2 Problem Statement ..... 1
1.3 Proposed Solution ..... 2
1.4 Objectives ..... 2
1.5 Contributions ..... 3
1.6 Assumptions and Constraints ..... 4
1.6.1 Assumptions ..... 4
1.6.2 Constraints ..... 4
1.7 Scope ..... 5
1.7.1 In Scope ..... 5
1.7.2 Optional Features (If Time Permits) ..... 6
1.7.3 Out of Scope ..... 6
1.8 Software Development Life Cycle ..... 6
1.8.1 Development Approach ..... 6
1.8.2 Justification ..... 6
1.8.3 Development Phases ..... 7
1.8.4 Ensuring Systematic Progress ..... 8
1.9 Report Organization ..... 8
**2 Requirement Analysis ..... 9**
2.1 Literature Review Overview ..... 9
2.2 Crash Detection Systems ..... 9
2.2.1 Threshold-Based Approaches ..... 9
2.2.2 Machine-Learning and Anomaly-Detection Approaches ..... 10
2.3 Drowsiness Detection Systems ..... 10
2.3.1 Vision-Based Detection ..... 10
2.3.2 Physiological-Sensor Detection ..... 11
2.3.3 Multi-Stage, Temporal and Quality-Aware Monitoring ..... 11
2.4 IMU and Camera Sensing ..... 12
2.4.1 IMU Sensors ..... 12
2.4.2 Helmet Cameras ..... 12
2.5 Navigation Systems ..... 12
2.6 Voice Control and Entertainment Systems ..... 13
2.7 Security and Theft Detection ..... 13
2.8 Web and Mobile Application Interfaces ..... 13
2.8.1 Product Website Versus Rider Telemetry ..... 14
2.8.2 Mobile Application Architecture ..... 14
2.9 Research Gaps and HelmX Contributions ..... 15
2.10 Consolidated Comparison ..... 16

2.11 Stakeholders ..... 17
2.12 Requirements Elicitation ..... 17
2.13 Functional Requirements ..... 18
2.14 Non-Functional Requirements ..... 22
2.15 Requirements Traceability Matrix ..... 24
2.16 Use Case Descriptions ..... 28
2.16.1 UC-01: Crash Detection and Emergency Alert ..... 28
2.16.2 UC-02: Drowsiness Detection and Alert ..... 29
2.16.3 UC-03: Smart Navigation Assistance ..... 30
2.16.4 UC-04: Ride Recording and Analytics Review ..... 32
2.16.5 UC-05: Offline Voice Command on the Helmet ..... 33
2.16.6 UC-06: Configure and Order a Helmet on the Website ..... 34
**3 System Design ..... 36**
3.1 Use Case Design ..... 36
3.1.1 Original Use Case Design ..... 36
3.1.2 Implemented Use Cases ..... 37
3.2 System Architecture ..... 38
3.2.1 Original Four-Layer Architecture ..... 38
3.2.2 Implemented Architecture ..... 39
3.3 Crash Detection Subsystem Design ..... 40
3.4 Drowsiness Detection Subsystem Design ..... 41
3.5 Android Application Design ..... 43
3.6 Helmet Services and Voice Design ..... 45
3.7 Web Platform Design ..... 46
**4 System Testing ..... 48**
4.1 Testing Strategy ..... 48
4.2 Test Case Design ..... 49
4.2.1 Crash Detection ..... 49
4.2.2 Drowsiness Detection ..... 50
4.2.3 Smart Navigation ..... 50
4.2.4 Dual Cameras (Original Scope) ..... 51
4.2.5 Voice Control and Entertainment ..... 51
4.2.6 Environment Monitoring ..... 52
4.2.7 Mobile and Web Applications ..... 52
4.2.8 Security and Authentication (Original Scope) ..... 53
4.3 Unit Testing ..... 53
4.3.1 Hardware Components ..... 53
4.3.2 Software Modules ..... 53
4.4 Integration Testing ..... 54
4.5 User Acceptance Testing (Planned) ..... 55
4.6 Success Criteria Summary ..... 56

4.7 Crash-Detection Experimental Results ..... 56
4.7.1 Model Architecture and Calibration ..... 57
4.7.2 Held-Out Model Comparison ..... 57
4.7.3 Distribution Shift and External False Alerts ..... 58
4.7.4 Quantization Fidelity and Raspberry Pi 4 Validation ..... 59
4.7.5 Deployment Fidelity ..... 60
4.7.6 Acquisition Integrity and Scheduling ..... 60
4.7.7 Session-Clustered Baselines, Jerk Isolation and Handling Risk ..... 60
4.7.8 Ethics, Scope and Validity ..... 61
4.8 Drowsiness-Detection Experimental Results ..... 61
4.8.1 Study Design, Separation and Labels ..... 61
4.8.2 Classification Performance ..... 61
4.8.3 Elapsed-Time Event Evaluation ..... 62
4.8.4 Ablations and Rejected Alternatives ..... 63
4.8.5 Software Verification, Export and Deployment Status ..... 63
4.8.6 Limitations ..... 63
4.9 Android, Web and Raspberry Pi Evidence ..... 64
4.9.1 Android Application ..... 64
4.9.2 Web Platform ..... 64
4.9.3 Raspberry Pi Configuration ..... 65
**5 Implementation ..... 66**
5.1 Work Breakdown Structure ..... 66
5.2 Team Roles and Responsibilities ..... 68
5.3 Development Timeline ..... 68
5.4 Tools and Technologies ..... 69
5.4.1 Machine-Learning Development ..... 69
5.4.2 Web Application Development ..... 69
5.4.3 Mobile Application Development ..... 70
5.4.4 Hardware Components ..... 70
5.4.5 Development and Collaboration Tools ..... 71
5.5 Implementation Details ..... 71
5.5.1 Crash Detection Model ..... 71
5.5.2 Drowsiness Detection Model ..... 72
5.5.3 Web Application ..... 73
5.5.4 Mobile Application ..... 74
5.5.5 Cameras and Media ..... 75
5.5.6 IMU and Environmental Sensors ..... 76
5.5.7 Offline Voice Assistant ..... 76
5.5.8 Theft Detection (Proposed, Not Implemented) ..... 77
5.5.9 Documentation ..... 77
5.6 Prototype and Application Screenshots ..... 78
5.6.1 Helmet Prototype ..... 78

5.6.2 Android Application ..... 78
5.6.3 Web Platform ..... 80
5.7 Challenges During Implementation ..... 80
**6 System Conversion ..... 82**
6.1 Introduction ..... 82
6.2 Conversion Method ..... 82
6.3 Deployment ..... 82
6.3.1 Deployment Steps ..... 82
6.3.2 Data Conversion ..... 83
6.3.3 User Training ..... 83
6.4 Post-Deployment Testing ..... 84
6.5 Deployment Challenges ..... 84
6.6 Summary ..... 84
**7 Cost Analysis ..... 85**
7.1 Introduction ..... 85
7.2 Hardware Cost Breakdown ..... 85
7.3 Development and Additional Costs ..... 86
7.4 Total Cost Estimation ..... 86
7.5 Cost Feasibility ..... 86
**8 Conclusion ..... 87**
8.1 Overview ..... 87
8.2 Achievements ..... 87
8.3 Limitations ..... 88
8.4 Future Work ..... 88
8.5 Final Remarks ..... 89
**References ..... 90**
**Appendix A: Originality Report ..... 94**
**Appendix B: Repository and Artefact Index ..... 95**

# List of Figures

**Figure 3.1:** Original FYP use-case design diagram (proposed scope). ..... 37
**Figure 3.2:** Implemented rider-facing use cases: helmet safety states, BLE telemetry, navigation with an offline map, and ride analytics with Firebase accounts. ..... 38
**Figure 3.3:** Original four-layer HelmX architecture from the FYP proposal (proposed design). . 39
**Figure 3.4:** Implemented HelmX architecture with the corrected local crash-alert route. ..... 40
**Figure 3.5:** Hybrid crash-detection model architecture used in the helmet edge-AI layer. ..... 41
**Figure 3.6:** End-to-end crash detection, confirmation and alert workflow on the Raspberry Pi. .. 41
**Figure 3.7:** Quality-aware causal drowsiness model and warning path. ..... 42
**Figure 3.8:** Personalized calibration and 12-feature causal physiology extraction. ..... 42
**Figure 3.9:** Elapsed-time alert-controller state logic (committed default configuration). ..... 43
**Figure 3.10:** Android companion architecture. ..... 44
**Figure 3.11:** Android phone-side safety-event handling and optional emergency response. ..... 44
**Figure 3.12:** Online routing and offline Lahore map boundaries. ..... 45
**Figure 3.13:** Raspberry Pi host, peripherals, on-helmet services and Android communication boundaries. ..... 45
**Figure 3.14:** Local Rhasspy–MQTT assistant path and voice-selected destination hand-off (end-to-end validation outstanding). ..... 46
**Figure 3.15:** Web-platform architecture: server-priced configuration, sandbox checkout and idempotent settlement. ..... 47
**Figure 4.1:** Data provenance, session-level split and external-validation boundaries of the crash-detection study. ..... 57
**Figure 4.2:** Five-seed in-distribution comparison of mean PR-AUC: supervised-only versus hybrid. ..... 58
**Figure 4.3:** Full-video causal replay on fold-2 development clips. ..... 62
**Figure 5.1:** Retrospective development timeline by functional workstream. ..... 69
**Figure 5.2:** HelmX working prototype: side view (left) and rear view with the Raspberry Pi 4, top-mounted IR camera and wiring harness (right) [8]. ..... 78
**Figure 5.3:** Drowsiness-detection prototype demonstration: (a) eyes open with an eye-state score of 0.94; (b) eyes closed for 3.6 s, raising an EYES CLOSED alert. ..... 78
**Figure 5.4:** Android application (1): (a) email/password sign-in; (b) home screen with BLE helmet status and live ride statistics (demo telemetry); (c) ride analytics with environmental readings; (d) MapLibre map with place search. ..... 79
**Figure 5.5:** Android application (2): (e) turn-by-turn guidance; (f) offline Lahore District map during active guidance; (g) crash alerts and emergency contacts with the opt-in automatic response; (h) test crash notification with the 30-second countdown and *I'm OK*, *Call* and *Send location* actions. ..... 79
**Figure 5.6:** Web platform: (a) 3D configurator with live preview and quote; (b) custom build with parts, storage and order summary; (c) test-mode payment confirmation; (d) contact form. ..... 80
**Figure A.1:** Originality report for the HelmX project proposal (similarity index 3%). ..... 94

# List of Tables

**Table 1.1:** Sprint plan and outcome ..... 7
**Table 2.1:** Comparison of related systems with HelmX ..... 16
**Table 2.2:** Primary and secondary stakeholders ..... 17
**Table 2.3:** FR1: Crash detection ..... 18
**Table 2.4:** FR2: Drowsiness detection ..... 19
**Table 2.5:** FR3: Smart navigation ..... 19
**Table 2.6:** FR4: Dual-camera system (original scope) ..... 20
**Table 2.7:** FR5: Voice control and entertainment ..... 20
**Table 2.8:** FR6: Safety and environment monitoring ..... 20
**Table 2.9:** FR7: Mobile application ..... 21
**Table 2.10:** FR8: Data management and web platform ..... 21
**Table 2.11:** NFR1: Performance ..... 22
**Table 2.12:** NFR2: Accuracy and reliability ..... 22
**Table 2.13:** NFR3: Usability ..... 22
**Table 2.14:** NFR4: Security and privacy ..... 23
**Table 2.15:** NFR5–NFR8: Scalability, compatibility, physical and cost constraints ..... 23
**Table 2.16:** Requirements traceability matrix ..... 24
**Table 2.17:** UC-01 summary ..... 28
**Table 2.18:** UC-02 summary ..... 29
**Table 2.19:** UC-03 summary ..... 30
**Table 2.20:** UC-04 summary ..... 32
**Table 2.21:** UC-05 summary ..... 33
**Table 2.22:** UC-06 summary ..... 34
**Table 4.1:** Test status definitions ..... 48
**Table 4.2:** Crash-detection test cases ..... 49
**Table 4.3:** Drowsiness-detection test cases ..... 50
**Table 4.4:** Navigation test cases ..... 50
**Table 4.5:** Dual-camera test cases ..... 51
**Table 4.6:** Voice and entertainment test cases ..... 51
**Table 4.7:** Environment-monitoring test cases ..... 52
**Table 4.8:** Mobile and web test cases ..... 52
**Table 4.9:** Security test cases ..... 53
**Table 4.10:** Android local unit-test suites ..... 54
**Table 4.11:** Success criteria and outcome ..... 56
**Table 4.12:** Crash-model comparison ..... 58
**Table 4.13:** Raspberry Pi 4 crash-model evidence ..... 59
**Table 4.14:** Drowsiness-model development metrics (pooled D2–D5) ..... 62

**Table 4.15:** Web-platform source evidence and validation boundaries ..... 65
**Table 5.1:** Team roles and responsibilities ..... 68
**Table 5.2:** Prototype hardware and interfaces ..... 70
**Table 5.3:** Implementation challenges ..... 80
**Table 6.1:** Post-deployment testing activities ..... 84
**Table 6.2:** Deployment challenges and responses ..... 84
**Table 7.1:** Budgeted hardware costs ..... 85
**Table 7.2:** Development and additional costs ..... 86
**Table 7.3:** Total estimated prototype cost ..... 86
**Table B.1:** Source repositories and archived artefacts ..... 95

# List of Abbreviations

| Abbreviation | Meaning |
|---|---|
| **AUC** | Area under the curve (ROC-AUC, PR-AUC) |
| **BLE** | Bluetooth Low Energy |
| **CSI** | Camera Serial Interface |
| **EAR / MAR** | Eye / mouth aspect ratio |
| **ECE** | Expected calibration error |
| **FR / NFR** | Functional / non-functional requirement |
| **GATT** | Generic Attribute Profile (BLE) |
| **GPIO** | General-purpose input/output |
| **GSM** | Global System for Mobile Communications |
| **I²C** | Inter-Integrated Circuit bus |
| **IMU** | Inertial measurement unit |
| **INT8** | 8-bit integer quantization |
| **MCC** | Matthews correlation coefficient |
| **MQTT** | Message Queuing Telemetry Transport |
| **ONNX** | Open Neural Network Exchange |
| **OSRM** | Open Source Routing Machine |
| **PERCLOS** | Percentage of eyelid closure over time |
| **RTM** | Requirements traceability matrix |
| **SUS** | System Usability Scale |
| **TCN** | Temporal convolutional network |
| **TFLite** | TensorFlow Lite |
| **UART** | Universal asynchronous receiver-transmitter |
| **UC** | Use case |
| **UTA-RLDD** | University of Texas at Arlington Real-Life Drowsiness Dataset |
| **WBS** | Work breakdown structure |

CHAPTER 1

# 1 Introduction

## 1.1 Background and Motivation

Road traffic crashes kill approximately 1.19 million people every year, and powered two- and three-wheeler users account for a large share of these deaths, particularly in low- and middle-income countries [1]. In Pakistan, road traffic deaths are estimated by the World Health Organization to exceed 10,000 per year, and two-wheeled vehicles are frequently involved [1]. Per vehicle-mile travelled, motorcyclists are about 28 times more likely than passenger-car occupants to die in a traffic crash [2]. Two factors make the outcome worse: after a crash a rider may be unconscious or immobilized and unable to call for help, and rider fatigue contributes to crashes while drowsiness-monitoring technology that is increasingly available in cars is largely absent for motorcyclists. In the United States alone, drowsy driving was involved in about 91,000 police-reported crashes in 2017 [3].

HelmX addresses these problems with distinct sensing and inference subsystems built into one helmet. Crash-like motion is analysed by an inertial-measurement-unit (IMU) model that combines classification and reconstruction signals. Drowsiness is monitored by a personalized camera pipeline that combines MobileNetV3-Small appearance embeddings, twelve ocular, mouth and head-pose features, modality-quality gating, causal temporal convolution and a separate elapsed-time alert policy. Both models are research prototypes with separate evidence and limitations, and this report keeps their claims separate.

## 1.2 Problem Statement

Motorcycle riders account for a disproportionate share of traffic fatalities, and their risk is amplified by limited protection, fatigue, navigation-related distraction and delayed emergency response when a rider is incapacitated. Existing products such as standard helmets and Bluetooth communicators provide fragmented features: commercial smart helmets typically focus on one capability, rarely include on-device artificial intelligence,

are expensive for developing markets, or provide little analytics. There is no affordable, integrated and honestly evaluated system that combines crash detection, drowsiness detection, navigation and emergency alerting for riders in low-resource settings.

## 1.3 Proposed Solution

HelmX is a smart-helmet ecosystem that combines edge intelligence, rider-facing applications and local emergency workflows. Crash detection runs on a Raspberry Pi 4 mounted on the helmet: a hybrid IMU model scores each motion window, a three-window confirmation policy suppresses transient spikes, and a rider-cancellable local alarm precedes an attempted SMS sent through a directly connected GSM modem. A camera-based drowsiness model estimates whether the rider is alert, in low vigilance or drowsy. A native Android application provides Bluetooth Low Energy (BLE) telemetry, navigation with an offline Lahore map, ride analytics and phone-side safety actions, and a separate web platform presents the product with a 3D configurator and test-mode checkout.

## 1.4 Objectives

The original project-wide target of greater than 85% accuracy is not an appropriate validation metric for either research model. The crash detector is therefore evaluated with PR-AUC, event-level recall, false-alert exposure and deployment fidelity, and the drowsiness model with subject-disjoint macro-F1, class-wise recall and causal event sensitivity. The objectives of the project are:

1. To design and assemble a helmet prototype with integrated sensors (IMU, cameras, microphone, GPS, GSM and environmental sensors).
2. To develop, calibrate and deploy on a Raspberry Pi 4 a hybrid IMU crash-detection model that combines supervised classification with reconstruction-based anomaly scoring.
3. To develop and evaluate a personalized, quality-aware temporal drowsiness research model under subject-disjoint validation and causal alert replay.
4. To implement phone-side navigation with online routing and an offline Lahore District map, and a local helmet voice assistant using Rhasspy, Porcupine, Kaldi, fsticuffs, Mosquitto MQTT and NanoTTS together with a hardware voice-recognition module on UART5.

5. To monitor ambient temperature and humidity (DHT22) and a configurable air-quality threshold (MQ135); the digital MQ135 signal does not measure calibrated PM2.5 or PM10 and does not support health recommendations.
6. To develop a native Kotlin Android companion application with BLE telemetry, opt-in safety alerts, navigation, ride history and authenticated accounts backed by Firebase Authentication and Cloud Firestore.
7. To develop a separate Next.js/React product website with a 3D helmet configurator, server-priced Stripe test-mode checkout, PostgreSQL order records and contact-form processing.

Success is assessed with subsystem-specific evidence: crash PR-AUC, class-stratified deployment fidelity, event-level sensitivity and false-alert exposure; drowsiness subject-disjoint macro-F1 and full-video causal event outcomes; and, for the applications, documented code and tests rather than independent field validation. The original targets of greater than 85% blanket accuracy, an emergency alert delivered in under 10 seconds, ±10 m GPS accuracy and a System Usability Scale (SUS) score above 70 remain historical targets, not measured outcomes.

## 1.5 Contributions

The main contributions of this project are:

- a crash-detection pipeline grounded in recent powered-two-wheeler literature, implemented as a hybrid Conv1D model that exposes both crash probability and reconstruction error;
- session-level evaluation, external-domain stress tests and transparent negative results, instead of a single simulated accuracy figure;
- measured Raspberry Pi 4 deployment of an INT8 model, including latency, long-duration soaks, timing validity and class-stratified deployment fidelity, released as a reproducibility package [4];
- a personalized, quality-gated drowsiness model with explicit handling of unavailable evidence and causal event replay [5];
- a broader helmet platform comprising an Android companion application [6], a web storefront [7] and on-helmet services including an offline voice assistant [8].

## 1.6 Assumptions and Constraints

### 1.6.1 Assumptions

- **Hardware availability:** the required cameras, IMU, GPS and GSM modules, microphones and edge computer are commercially available within the project budget and timeline.
- **Connectivity:** cloud-based application functions rely on internet connectivity, whereas the crash-alert path uses a SIM800L GSM module connected directly to the Raspberry Pi; GSM reception is not guaranteed.
- **User cooperation:** test riders wear the helmet correctly and follow safety protocols during testing.
- **Environment:** the system operates in normal riding conditions; extreme weather and temperatures outside −10 °C to 50 °C are excluded.
- **Data availability:** public third-party IMU datasets are available for model development; because authentic motorcycle-crash data are scarce, crash-detection results are interpreted in terms of benchmark datasets, controlled crash-like sessions and external validation rather than unrestricted real-world deployment.
- **Power:** the prototype is powered by two nominal 10,000 mAh power banks; whole-system battery runtime was not formally measured.
- **GPS accuracy:** consumer GPS is assumed to locate the rider to within 5–10 m in open-sky conditions; actual receiver error was not measured.
- **Cloud reliability:** Android rider and ride data use Firebase Authentication and Cloud Firestore, and the web storefront uses separate PostgreSQL/Prisma services. Helmet crash alarms and GSM attempts are local to the Pi; neither internet availability nor SMS delivery is guaranteed.

### 1.6.2 Constraints

- **Budget:** the project must stay within an estimated budget, which limits the use of commercial-grade sensors and the number of prototypes.
- **Time:** development had to fit the academic schedule.
- **Processing power:** the Raspberry Pi 4 runs the INT8 crash monitor and hardware services; drowsiness training uses PyTorch with ONNX export, and a complete Pi camera-to-alert benchmark has not been established.

- **Storage:** the Pi boots from a 128 GB USB-attached SanDisk X400 SSD; captured media is stored locally, and the Android app uploads ride summaries, not complete GPS traces.
- **Battery and latency:** power consumption, sensor sampling and network latency constrain continuous operation, especially in poorly connected areas.
- **Scope:** large-scale manufacturing, medical-grade validation, advanced hardware such as dynamic visor tinting and international safety certification are excluded.
- **Expertise, privacy and media quality:** development was constrained by the team's embedded-systems experience, by basic data-protection practice without formal legal certification, and by the trade-off between video quality, storage and processing power.

## 1.7 Scope

The project covers the physical helmet and its independently maintained software modules: the deployed crash-detection research software, the drowsiness-model source and software runtime, the Kotlin Android companion, the Raspberry Pi services and a separate Next.js storefront. Each has distinct validation evidence; a complete production-ready end-to-end system has not been demonstrated.

### 1.7.1 In Scope

- **Crash detection:** continuous IMU analysis, multi-window confirmation, a rider-cancellable local warning and a GPS-assisted SMS attempt when the alert is not cancelled.
- **Drowsiness monitoring:** a personalized, quality-aware three-class temporal model with explicit UNAVAILABLE handling and elapsed-time warnings; motorcycle-domain validation remains future work.
- **Navigation:** GPS-based route guidance on the phone with an offline Lahore District map.
- **Cameras:** a CSI infrared night-vision camera and a USB webcam with microphone; the voice assistant supports photo capture and approximately ten-second video clips. Continuous synchronized dual-camera recording, live streaming and automatic cloud upload are not established.

- **Voice control and entertainment:** offline voice commands for navigation requests, status queries, music, photos and short videos, with audio through the helmet speakers.
- **Accounts and security:** Firebase-authenticated Android accounts; web purchase and contact requests are validated separately. Theft detection and biometric rider authentication remain proposals.
- **Applications:** a Kotlin Android companion app for BLE telemetry, offline map viewing, online routing, ride history and phone-side safety actions, and a separate web platform.

### 1.7.2 Optional Features (If Time Permits)

Gesture control; posture detection and suggestions; traffic-adaptive sound control; distance monitoring and fuel estimation; rider feedback; public awareness of dangerous road areas; solar-assisted charging; a heads-up display with augmented reality; speed optimization; an AI bike-diagnosis assistant; and a personalized riding coach. None of these optional features was implemented.

### 1.7.3 Out of Scope

Hardware manufacturing at scale; advanced hardware such as dynamic tinting; medical-grade validation of health metrics; a mesh network between riders; gamified safe-riding scores; battery optimization; and research on the health effects of electromagnetic fields from the electronics.

## 1.8 Software Development Life Cycle

### 1.8.1 Development Approach

The project followed the Agile Scrum methodology with iterative sprints of two to four weeks over the eight-to-nine-month development period. Scrum suits a multidisciplinary project that combines hardware prototyping, AI model development, cloud services and application development across several platforms.

### 1.8.2 Justification

- **Iterative research:** the AI components evolved through staged validation. The historical >85% accuracy target was replaced by imbalanced-label PR-AUC, event-level false alerts and fidelity measurements for crash detection, and by three-class

macro-F1 and causal replay for drowsiness, all evaluated against frozen data protocols.

- **Integration complexity:** IoT sensors, embedded software, cloud services and mobile applications require frequent integration testing, which short sprints make possible.
- **Evolving requirements:** requirements changed as hardware capabilities and feedback became clear; Scrum accommodates change between sprints.
- **Parallel development:** four team members worked on different components and integrated them through sprint reviews.
- **Risk mitigation:** incremental delivery ensured that core functions were developed and tested first.

### 1.8.3 Development Phases

Table 1.1 summarizes the sprint plan. Items marked as planned were part of the original plan but were superseded or not completed; the implemented technology choices are described in Chapter 5.

**Table 1.1:** Sprint plan and outcome

| Phase | Planned activities | Outcome |
|---|---|---|
| Sprint 0 (weeks 1–2) | Literature review; requirements; four-layer architecture; procurement; product backlog | Completed |
| Sprints 1–3 (weeks 3–10) | Helmet assembly; hybrid crash pipeline with threshold calibration, INT8 export and Pi validation; drowsiness research model; local Pi services and application backends | Completed; AWS IoT Core and MongoDB were earlier plans replaced by Firebase and PostgreSQL |
| Sprints 4–6 (weeks 11–18) | MapLibre with OpenStreetMap-derived tiles, Photon search, OSRM routing and offline Lahore tiles; cameras; voice assistant; environmental sensors | Completed except continuous dual-camera recording (not implemented) |
| Sprints 7–9 (weeks 19–26) | Kotlin BLE telemetry, Firebase accounts, navigation and safety workflows; Next.js storefront with 3D configurator, Stripe test checkout and SMTP receipts; integration testing | Completed at source level; physical Pi-to-phone integration not verified |
| Sprints 10–12 (weeks 27–36) | Optimization; user acceptance testing; documentation; deployment of the APK and Vercel demonstration | Documentation and deployment completed; formal user acceptance testing not performed |

Continuous activities included daily stand-up meetings, version control with Git and branch-based development, unit and regression testing, and weekly progress updates to the supervisor.

### 1.8.4 Ensuring Systematic Progress

Each sprint began with planning in which user stories were selected, decomposed into tasks and assigned by expertise. The definition of done was evidence-based: classification and event-level measures for the crash detector, macro-F1 and false-alarm trade-offs for drowsiness, and target-device performance only where it was actually measured. Sprint retrospectives, incremental deliverables and reprioritization of optional features kept the minimum viable product on schedule.

## 1.9 Report Organization

Chapter 2 reviews related work and specifies stakeholders, requirements and use cases. Chapter 3 presents the system design. Chapter 4 describes the testing strategy and reports the crash-detection, drowsiness and application evidence. Chapter 5 covers the implementation, work breakdown, team roles, tools, screenshots and challenges. Chapter 6 discusses system conversion and deployment, Chapter 7 the cost analysis, and Chapter 8 the conclusions, limitations and future work.

CHAPTER 2

# 2 Requirement Analysis

## 2.1 Literature Review Overview

Smart-helmet research combines artificial intelligence, the Internet of Things, computer vision and wearable computing. This review covers the domains relevant to HelmX: crash detection, drowsiness detection, IMU and camera sensing, navigation, voice control and entertainment, security and theft detection, and mobile and web applications. It identifies gaps in existing approaches and positions the HelmX contributions.

## 2.2 Crash Detection Systems

Motorcycle crash detection is difficult because representative crash data are scarce, classes are severely imbalanced, and normal riding contains many hard negatives such as bumps, harsh braking and rapid steering. The literature therefore spans threshold-based detectors, supervised learning, one-class and anomaly-detection approaches, and lightweight edge deployment.

### 2.2.1 Threshold-Based Approaches

Early powered-two-wheeler work established the importance of false-alarm control and long-duration evaluation on normal riding. Gelmini *et al.* combined inertial and GNSS signals in an automatic crash-detection system for two-wheeled vehicles and evaluated it over approximately 5,300 km of non-crash riding [9]. Amin *et al.* detected accidents from the deceleration measured by a low-cost MEMS IMU and used an attitude-heading reference with Kalman filtering to estimate position during GPS outages [10]. Cismas *et al.* showed how an MPU6050 IMU can provide Euler angles and acceleration for crash detection in an embedded device [11]. Such systems demonstrate operational feasibility, but hand-crafted thresholds are sensitive to riding style and may degrade under conditions not represented during tuning. The e-scooter study of Lee *et al.* is particularly relevant because it compared threshold, supervised and unsupervised approaches in the same experimental setting, showing that the choice of method should be judged in context [12].

### 2.2.2 Machine-Learning and Anomaly-Detection Approaches

Rodegast *et al.* compared several supervised algorithms for motorcycle collision detection on simulation-generated data and reported strong AUC values, but did not validate deployment on constrained edge hardware [13]. One-class detection is also established: the automatic eCall work of Gelmini *et al.* learned normal motorcycle dynamics without crash-only training [14], and data-driven classifiers for powered two-wheelers have been proposed [15]. Related hybrid-learning and anomaly precedents include multi-task classification–reconstruction networks [16], weakly supervised autoencoder feature learning [17], semi-supervised fall detection with limited positive data [18] and autoencoder ensembles for unseen falls [19]. Further powered-two-wheeler work includes self-organizing-map hazard detection [20], critical-event recognition [21], edge-AI accident detection with MCUSUM features [22], a 2025 cross-dataset benchmark distributed as a preprint [23] and two Chalmers master's theses on motorcycle crash detection and notification [24], [25]. Multimodal self-supervised crash detection has also been shown to improve motion representations, although not in a powered-two-wheeler setting [26].

These studies show that the ingredients of a hybrid architecture are established. The open questions are how classification confidence and reconstruction error behave on a specific multi-source IMU benchmark, how they generalize under domain shift, and whether the deployed artefact remains faithful on the target edge device. HelmX does not find hybrid learning superior: the supervised-only baseline was stronger in-distribution, and the reconstruction branch was useful mainly as a complementary signal under a held-out simulated crash mechanism (Section 4.7).

## 2.3 Drowsiness Detection Systems

Drowsy driving is a recognized cause of crashes; in the United States it was involved in about 91,000 police-reported crashes in 2017 [3]. For motorcycle riders the consequences are especially severe because of reduced vehicle stability and minimal protection.

### 2.3.1 Vision-Based Detection

Ahmed *et al.* used a convolutional neural network and VGG16 to classify eye and mouth states (open, closed, yawning and not yawning) and tested the models on 2,900 images of eye conditions associated with driver sleepiness [27]. Maior *et al.* proposed real-time

drowsiness classification based on the eye aspect ratio (EAR), detecting long blinks and eye closures as fatigue indicators [28]. Magán *et al.* developed an advanced driver-assistance system that analyses 60-second image sequences with either a recurrent-convolutional network or deep features combined with fuzzy logic, with an emphasis on minimizing false positives [29]. Vision-based systems work well in controlled conditions but face computational cost, latency on edge hardware and sensitivity to lighting.

### 2.3.2 Physiological-Sensor Detection

Dhole *et al.* embedded an EEG acquisition device in a worker's safety helmet and used a random-forest classifier on time- and frequency-domain features to detect sleep deprivation, with an IMU for fall detection [30]. Von Rosenberg *et al.* recorded ECG, respiration and EEG from electrodes embedded in a standard helmet and validated the signals against reference measurements [31]. Physiological approaches can be accurate but require skin contact, calibration and specialized hardware, which limits comfort and practicality for riders.

### 2.3.3 Multi-Stage, Temporal and Quality-Aware Monitoring

Fixed EAR thresholds can recognize visible eye closures but do not by themselves establish vigilance state. The UTA Real-Life Drowsiness Dataset (UTA-RLDD) distinguishes alert, low-vigilance and drowsy video labels, exposing the difficulty of the intermediate class [32]. It contains indoor RGB recordings rather than motorcycle-camera footage, and its labels apply to whole videos rather than timed physiological onsets.

The HelmX approach extends eye geometry with personalized PERCLOS, blink dynamics, relative mouth opening, head-pose trajectories and measured input quality, while a MobileNetV3-Small encoder represents facial appearance. A learned gate combines the two streams before causal dilated temporal convolutions. The novelty claim is the implemented and audited combination, not the invention of EAR, PERCLOS, MobileNetV3 or temporal networks. Auxiliary pretraining uses the MRL Eye dataset (84,898 images from 37 subjects) as an eye-state resource, not as evidence of drowsiness classification [33]. Because high clip-level scores can coexist with limited alarm sensitivity, the study also requires full-video causal event evaluation. The implementation, training protocol and record of negative experiments are documented in the research repository [5]. Independent DROZY evaluation, helmet-mounted camera conditions and Raspberry Pi camera-to-alert performance have not yet been verified.

## 2.4 IMU and Camera Sensing

### 2.4.1 IMU Sensors

The MPU6050 is widely used in smart-helmet prototypes because it combines a three-axis accelerometer and a three-axis gyroscope in one package; it can provide Euler angles and acceleration for crash detection [11]. Wong *et al.* used an IMU-equipped full-face helmet to recognize head motions of motorcycle riders, found gyroscope data more useful than accelerometer data, and reported classification accuracy between 95.9% and 99.1% for different head motions [34]. Common limitations of IMU-based helmets are limited on-board processing, single-purpose design, calibration on constrained devices and sensitivity to vibration.

### 2.4.2 Helmet Cameras

Commercial smart helmets such as the JARVISH X-AR integrate cameras, a heads-up display and voice-assistant support [35]. Helmet cameras are mainly used to document incidents rather than as integrated safety sensors, and they face constraints of power consumption, storage management, weight and cost. A scoping review of smart helmets with multimodal sensing summarizes the range of sensors and applications in current research and notes that most systems remain prototypes [36].

## 2.5 Navigation Systems

Navigation assistance matters for riders because handlebar-mounted displays require visual attention to move away from the road. Wearable navigation research such as eLabrador combines GPS-based route guidance with on-body perception to guide visually impaired pedestrians over long distances, and it highlights that GPS alone suffers from location inaccuracy in urban areas [37]. Many helmet projects pair a microcontroller with a smartphone over Bluetooth so that the phone performs routing and the helmet presents simplified cues. Limitations include dependence on connectivity, positioning error in urban canyons, communication latency and reliance on visual displays. HelmX uses phone-side routing with OpenStreetMap-derived maps rendered by MapLibre [38], Photon geocoding and OSRM routing [39], plus an offline map package for Lahore.

## 2.6 Voice Control and Entertainment Systems

Commercial helmet communication systems increasingly support voice operation. The Sena Stryker smart helmet integrates a communication system with voice commands [40], and the Cardo PACKTALK Edge supports natural voice operation with the wake phrase "Hey Cardo" [41]. Dedicated audio hardware such as the Sena 10R communicator and Cardo's Sound by JBL helmet speakers address audibility at speed [42], [43]. Juniper Research forecast that 8.4 billion digital voice-assistant devices would be in use by 2024 [44], yet helmet integration remains largely limited to command interfaces that rely on a paired smartphone. Wind noise, limited language support and the lack of on-device processing are recurring limitations, which motivates the offline Rhasspy-based assistant used in HelmX.

## 2.7 Security and Theft Detection

Motorcycle theft is a significant problem: the National Insurance Crime Bureau reported 54,736 motorcycle thefts in the United States in 2022, the third consecutive annual increase [45]. Preetham *et al.* combined an alcohol sensor with face recognition (Viola–Jones detection and support-vector-machine classification) to authorize the rider [46]. Sudarsan and Kumaraguru proposed wireless bike authentication between helmet and motorcycle [47], and Kalkotwar *et al.* used an RF transmitter in the helmet so that the motorcycle starts only when the helmet is worn [48]. Multimodal biometric systems generally improve on single-modality recognition [49], but comprehensive multimodal authentication has not been demonstrated in a helmet. For tracking and emergency notification, Konnect sends accelerometer-detected accidents and GPS location through a cloud service [50]; Priya *et al.* combined a helmet unit with a vehicle unit for location tracking and engine control [51]; Vashisth *et al.* synchronized helmet and bike modules over RF with speed limiting, alcohol locking and accident detection [52]; and Alcantara *et al.* reported 90% accuracy in distinguishing real accidents and 88% accuracy in drunk-driver detection in a delivery-rider helmet with a Firebase-based mobile application [53]. HelmX does not implement theft detection; it remains future work (Section 5.5.8).

## 2.8 Web and Mobile Application Interfaces

Helmet systems usually expose their data through a companion application. Pardeshi *et al.* combined helmet-based authentication with a smart dashboard for two-wheelers [54],

and Islam *et al.* paired a helmet circuit with a mobile application through which the rider monitors riding status [55].

### 2.8.1 Product Website Versus Rider Telemetry

Earlier smart-helmet proposals, including the original HelmX proposal, combine rider dashboards and product websites into a single platform. The implemented HelmX system separates these concerns. Rider safety and ride history reside in the Kotlin Android application backed by Firebase, whereas the public website provides product information, 3D hardware configuration, ordering and contact functions [7]. The storefront uses a shared typed component catalog as the authoritative source for both user-interface pricing and server-side checkout validation: unknown part IDs are rejected, duplicate IDs are charged once, and the order is persisted through Prisma/PostgreSQL before a Stripe test-mode payment. The change from PENDING to PAID is atomic and idempotent, and an SMTP receipt is attempted on the successful transition without a durable retry guarantee. The contact form applies field validation, HTML escaping, in-memory per-instance throttling and database persistence. The deployed demonstration is not a production-validated e-commerce process and does not synchronize Android ride histories; its 3D configurator illustrates modular hardware choices rather than the exact tested prototype.

### 2.8.2 Mobile Application Architecture

A companion application must make explicit which computation runs on the helmet and what is received on the phone. The HelmX client uses Android Bluetooth Low Energy as a receive-only telemetry transport [56], while neural-model inference remains in the separate crash- and drowsiness-detection projects [6]. The app is written in native Kotlin with Android Views, ViewBinding, lifecycle-aware coroutines and StateFlow. Accounts and ride summaries use Firebase Authentication and Cloud Firestore, not Firebase Realtime Database [57]. Route search and routing use Photon and OSRM, with MapLibre GL JS inside an Android WebView displaying OpenStreetMap-derived vector tiles [38], [39]. Bundled Lahore District tiles allow offline map viewing and continued guidance on a previously calculated route; new routes require internet access. The safety layer adds app-wide notification, vibration and action handling for Boolean crash and drowsiness flags reported by the helmet, and an optional, permission-gated 30-second automatic SMS and call attempt that is separate from the Raspberry Pi GSM path. Neither path is evidence of an independently validated emergency-dispatch chain.

## 2.9 Research Gaps and HelmX Contributions

The crash-detection literature suggests that the opportunity for HelmX is not a wholly novel ingredient but a carefully engineered and honestly evaluated powered-two-wheeler crash detector whose contribution lies in its formulation, analysis and deployment evidence.

- **Complementary signals:** prior work usually studies threshold, supervised or anomaly-based detection separately; few studies explain how classification confidence and reconstruction error complement each other on motorcycle-style inertial data under one calibrated evaluation protocol.
- **Deployment evidence:** many papers report model accuracy but provide little evidence about target-device behaviour, external validation, long-duration non-crash operation or deployment fidelity, which determine whether offline results survive on the real device.
- **Transparent evaluation:** session-level splitting, domain-shift analysis and negative results remain rare.

Accordingly, the HelmX crash-detection contribution is: (i) a hybrid Conv1D model that exposes crash probability and reconstruction error; (ii) a harmonized multi-source IMU pipeline with session-level evaluation; (iii) a measured Raspberry Pi 4 deployment with INT8 export and operational decision logic; and (iv) a limitations analysis covering scarce real crash data, external-domain false alerts and incomplete end-to-end emergency validation [4]. HelmX also contributes a personalized, quality-gated MobileNetV3-Small/physiology/causal-TCN drowsiness research model with three-class subject-disjoint evaluation, time-aware warning logic and a reproducible causal replay harness [5], [32], [33]. Unlike the crash subsystem, the drowsiness model has no completed independent DROZY confirmation and no Raspberry Pi camera-to-alert benchmark; these remain open requirements.

## 2.10 Consolidated Comparison

Table 2.1 compares representative related work with the implemented HelmX system.

**Table 2.1:** Comparison of related systems with HelmX

| System | Crash detection | Drowsiness detection | Sensing | Navigation | Voice / audio | Security / emergency | App |
|---|---|---|---|---|---|---|---|
| Gelmini *et al.* [9] | Inertial + GNSS rule-based detection; validated over ≈5,300 km of normal riding | — | IMU, GNSS | — | — | — | — |
| Lee *et al.* [12] | Threshold, supervised and unsupervised models compared (e-scooter) | — | 6-axis IMU | — | — | — | — |
| Ahmed *et al.* [27] | — | CNN and VGG16 on 2,900 eye-state images | Camera | — | — | — | — |
| Dhole *et al.* [30] | IMU fall detection | EEG with random forest | EEG, IMU | — | — | Supervisor alerts | Android app |
| Chandran *et al.* [50] | Accelerometer threshold | — | Accelerometer, GPS | — | — | Cloud notification with location | — |
| Alcantara *et al.* [53] | 90% accident identification | Alcohol sensing (88%) | Accelerometer, GPS | — | — | Alerts to shop owners | Firebase app |
| Preetham *et al.* [46] | — | Alcohol sensing | Camera, IR | — | — | Face-recognition rider authorization | — |
| JARVISH X-AR [35] | — | — | Camera | Heads-up display | Voice assistant | — | Companion app |
| Sena / Cardo [40], [41] | — | — | — | — | Bluetooth intercom, voice commands | — | Companion app |
| **HelmX (this work)** | Hybrid Conv1D classifier + reconstruction; held-out ROC-AUC 0.946; INT8 (83,568 B) on Raspberry Pi 4 | Quality-gated MobileNetV3-Small + 12 features + causal TCN; UTA-RLDD macro-F1 0.642; external validation pending | 6-axis IMU (WHO_AM_I 0x70), IR and USB cameras, DHT22, MQ135 | Phone-side MapLibre / Photon / OSRM, offline Lahore map | Offline Rhasspy assistant, UART5 voice module | Rider cancellation, local GPS + GSM SMS attempt; opt-in phone SMS/call | Kotlin Android app; Next.js storefront |

## 2.11 Stakeholders

Table 2.2 lists the individuals, groups and organizations with an interest in HelmX, together with their role, interest and level of influence.

**Table 2.2:** Primary and secondary stakeholders

| Stakeholder | Role and responsibilities | Key interests | Involvement | Influence |
|---|---|---|---|---|
| Development team: Usman Azfar (group leader), Hamza Ahmad, Abdul Hanan, Awais Imtiaz | Requirements, architecture, implementation, integration of AI/IoT modules, testing and deployment | Successful completion; application of AI, IoT, cloud and full-stack skills | High, throughout the project | High |
| Project supervisor: Dr. Tayyaba Tariq | Mentorship, milestone approval, research quality and evaluation | Academic rigour, quality deliverables, research contribution | High (regular meetings and reviews) | High |
| End users: motorcycle riders in Pakistan, especially Lahore | Primary beneficiaries; testing and feedback | Safety, convenience, affordability, crash detection and navigation | Medium (testing and feedback) | Medium |
| University administration: Department of Computer Science, FCIT, University of the Punjab | Project approval, resources and final evaluation | Academic standards, innovation, institutional reputation | Low | Medium |
| Emergency response services: ambulance, hospitals, police | Recipients of emergency alerts | Accurate and timely alerts | Low (passive) | Low |
| Transportation safety authorities: NHMP, provincial traffic police | Potential regulators or adopters | Crash reduction and safety data | Low (future) | Low to medium |
| Hardware and component suppliers | Supply of sensors, cameras, batteries and modules | Sales and future demand | Low (transactional) | Low |
| Cloud and platform providers: Google Firebase, Vercel, Stripe (test mode) | Authentication, ride storage, web hosting, test payments | Platform usage | Low (service-based) | Low |
| Academic research community | Consumers and reviewers of research outputs | Methodology and technical contribution | Low (indirect) | Low |
| Technology enthusiasts and early adopters | Potential pilot users and testers | Innovation and early access | Low (optional) | Low |

## 2.12 Requirements Elicitation

Requirements were elicited through several complementary techniques:

- **Literature and document analysis** of academic papers, existing smart-helmet systems and motorcycle-safety research (Sections 2.1–2.9).
- **Stakeholder consultation:** discussions with the project supervisor to align objectives with academic requirements, and informal conversations with motorcycle riders in Lahore about safety concerns, navigation challenges and preferences.
- **Brainstorming sessions** within the team to identify features and prioritize them by safety impact and technical feasibility.
- **Market analysis** of commercial smart helmets and communication systems to identify gaps in affordability and integration.
- **Technical feasibility assessment** of hardware, AI algorithms, cloud services and development tools against the timeline and budget.

## 2.13 Functional Requirements

Functional requirements define the behaviours the system must provide. Each requirement lists its inputs and outputs, pre- and postconditions and priority. Requirements that describe features that were proposed but not implemented are retained for traceability; their status is given in the requirements traceability matrix (Section 2.15).

**Table 2.3:** FR1: Crash detection

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR1.1 | Continuously acquire 6-axis IMU data and maintain the canonical 60 × 6 window representation at an effective 50 Hz | MPU6050-class accelerometer and gyroscope stream → windowed motion tensor | Helmet powered and IMU initialized / stable window buffer for inference | High |
| FR1.2 | Run the hybrid model that outputs crash probability and reconstruction error, then apply calibrated OR-fusion decision logic | Window tensor, engineered features, deployed model → per-window decision and scores | Model, normalizer and thresholds deployed / decision produced for each window | Critical |
| FR1.3 | Require three consecutive positive windows before entering the alert state | Binary window decisions → confirmed crash candidate | Monitoring active / alert only after confirmation | Critical |
| FR1.4 | Issue a local rider warning with a 30-second cancellation period; if uncancelled, acquire GPS and attempt emergency notification | Confirmed event, GPS/GSM availability → local alarm, cancellation option, SMS attempt | Emergency contacts configured / alert cancelled or escalated | Critical |
| FR1.5 | Log event metadata, scores and alert outcome for audit and analysis | Scores, timestamps, alert outcome → persistent event record | Storage available / crash-analysis record created | Medium |

**Table 2.4:** FR2: Drowsiness detection

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR2.1 | Sample timestamped camera frames and track facial landmarks with image and landmark quality | RGB/IR camera frames → timestamped landmarks and face crop | FaceLandmarker assets available / quality-aware observation stream | High |
| FR2.2 | Calibrate neutral ocular, mouth and pose baselines and compute 12 causal physiology features | Quality-valid frames and landmark geometry → [T,12] physiology and [T,2] reliability | Neutral calibration completed / personalized features buffered | High |
| FR2.3 | Classify alert, low vigilance and drowsy using the quality-gated MobileNetV3-Small and causal temporal model | [T,3,H,W] images, features, quality → calibrated class probabilities | ONNX model exported, buffer filled / evidence passed to controller | Critical |
| FR2.4 | Apply an elapsed-time caution/warning/critical policy, suppress unreliable evidence and support UNAVAILABLE | Class probabilities, quality, monotonic time → state and rate-limited alarm | Controller configured / reliable warning or unavailable state | Critical |
| FR2.5 | Configure operating thresholds through a verified runtime configuration; app synchronization is future work | Versioned controller configuration → applied warning policy | Settings validated / specified persistence and cooldown | Medium |

**Table 2.5:** FR3: Smart navigation

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR3.1 | Provide a MapLibre map with OpenStreetMap-derived tiles, Photon place search and OSRM directions | GPS position, typed/voice/helmet destination → route geometry, steps, spoken guidance | Location available, network for new routes / navigation session | High |
| FR3.2 | Reroute when the rider leaves the selected OSRM route; live-traffic optimization is excluded | Current GPS and route, internet → updated route after off-route detection | Route previously calculated, online / route updated with cooldown | Medium |
| FR3.3 | Present turn-by-turn banners and optional spoken phone guidance | OSRM maneuver steps, GPS → maneuver banner, phone audio | Guidance enabled / guidance updated during ride | High |
| FR3.4 | Accept a destination from text, Android voice search or the helmet over BLE | Text, speech-to-text, BLE destination → resolved place and route request | Network for new place lookup / navigation opened or queued | Medium |

**Table 2.6:** FR4: Dual-camera system (original scope)

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR4.1 | Record video from front and rear cameras at 720p or better | Camera feeds → video recordings | Cameras active / footage stored | High |
| FR4.2 | Capture images by voice command | Voice input → image | Voice system active / image saved | Medium |
| FR4.3 | Compress video for storage optimization | Raw video → compressed video | Processing unit active / storage optimized | Medium |
| FR4.4 | Upload footage to cloud storage | Video files → cloud backup | Internet available / footage stored remotely | High |
| FR4.5 | Tag captured media with timestamp and GPS position | Media, time, GPS fix → tagged media | GPS available / tagged evidence | High |

**Table 2.7:** FR5: Voice control and entertainment

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR5.1 | Support offline wake-word detection, speech recognition, intent dispatch and spoken responses through Rhasspy (Porcupine, Kaldi, fsticuffs, NanoTTS) | USB microphone audio → local intent and spoken response | Rhasspy, MQTT and audio services configured / intent routed to assistant | High |
| FR5.2 | Execute sensor-status queries, navigation requests, music, photo and short-video commands through the local MQTT assistant | Intent from Rhasspy or the UART5 voice module → media, sensor or navigation action | Hardware and services available / action attempted and logged | Medium |
| FR5.3 | Publish a selected destination through `navigation_target.json` and the helmet BLE service | Navigation intent → BLE destination JSON for the phone | BLE peripheral and Android app connected / phone may start Photon search and OSRM guidance | High |

**Table 2.8:** FR6: Safety and environment monitoring

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR6.1 | Read the MQ135 digital air-quality threshold status (not calibrated PM2.5/PM10) | MQ135 digital GPIO input → qualitative air-quality signal | Sensor and BLE service connected / app can display status | High |
| FR6.2 | Read DHT22 ambient temperature and humidity approximately every 2.5 seconds | DHT22 data → temperature and humidity readings | DHT driver available / measurements updated | Medium |
| FR6.3 | Transmit environment readings to Android over BLE and write a local `sensor_data.json` snapshot about every 10 seconds | Sensor readings → BLE JSON and local snapshot | `helmx-ble` service running / readings available | Medium |

**Table 2.9:** FR7: Mobile application

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR7.1 | Receive BLE GATT notification payloads and display current helmet telemetry | JSON/CSV on characteristic FFE1 → battery, speed, environment, flags and link state | Rider pairs the HelmX BLE peripheral / StateFlow displayed on screen | High |
| FR7.2 | Show navigation, search and offline Lahore maps with online route calculation | GPS, Photon/OSRM responses, bundled tiles → guidance and map view | Location permission, network for new routes / online route or offline map | High |
| FR7.3 | Track filtered GPS rides and display authenticated ride history | GPS fixes and rider account → distance, duration, average speed, recent rides | Foreground location service and account / ride summary queued for Firestore | Medium |
| FR7.4 | Provide phone-side crash and drowsiness notifications and opt-in emergency actions | BLE safety flags, contacts, permissions → notification, alarm, cancel, optional SMS/call | Helmet flags and permissions / phone alert path active (delivery not implied) | Critical |
| FR7.5 | Support authenticated email/password accounts and user-scoped Firestore records | Email, password, profile → authenticated profile and per-user rides | Firebase Auth and Firestore configured / records restricted to the user ID | High |

**Table 2.10:** FR8: Data management and web platform

| ID | Requirement | Inputs → outputs | Pre- / postconditions | Priority |
|---|---|---|---|---|
| FR8.1 | Store Android account and ride records under `users/{uid}` and `users/{uid}/rides` | Firebase login and ride summaries → per-user Firestore documents | Firebase Auth and Firestore configured / user-specific history stored | High |
| FR8.2 | Persist web orders and contact submissions in Prisma/PostgreSQL, separately from Firestore | Customer email, cart parts, contact message → Order and ContactSubmission records | `DATABASE_URL` and schema configured / web transaction recorded | High |
| FR8.3 | Validate part IDs and recompute the PKR total on the server before creating a Stripe sandbox session | Part IDs, storage ID, email → validated quote and checkout redirect | Server catalog and Stripe test keys / PENDING order and session | Critical |
| FR8.4 | Verify Stripe signatures, settle orders idempotently and attempt itemized email receipts | Stripe events, checkout confirmation → PAID or FAILED state, receipt attempt | Webhook secret and SMTP configured / repeated events do not duplicate transitions | Critical |

The original FR8 described a proposed MongoDB/AWS telemetry backend. The implemented services are separate: Cloud Firestore for Android rider data and PostgreSQL for the web configurator, payments and contacts. Live IMU model scores and emergency dispatch are not streamed into either database.

## 2.14 Non-Functional Requirements

Non-functional requirements specify quality attributes, performance characteristics and constraints.

**Table 2.11:** NFR1: Performance

| ID | Requirement | Description | Metric / constraint | Priority |
|---|---|---|---|---|
| NFR1.1 | Crash-model inference latency | Target-device crash scoring must be comfortably real-time on the Raspberry Pi 4 | p99 inference latency ≤ 10 ms per scored window | Critical |
| NFR1.2 | Drowsiness detection latency | Video processing should remain responsive enough for rider alerts | Target ≤ 500 ms camera-to-alert (not validated on the Pi) | Critical |
| NFR1.3 | Emergency escalation timing | Notification only after confirmation and the rider-cancellation interval | 30 s cancellation precedes the SMS attempt; no < 10 s delivery claim | Critical |
| NFR1.4 | Application response time | The mobile app should load analytics and ride data quickly | ≤ 2 s on 4G/Wi-Fi | High |
| NFR1.5 | Battery performance | Practical continuous operation during rides | 2–3 h active use (target) | High |

**Table 2.12:** NFR2: Accuracy and reliability

| ID | Requirement | Description | Metric / constraint | Priority |
|---|---|---|---|---|
| NFR2.1 | Crash-detection evidence | Evaluate with classification and event metrics, not a single accuracy value | Report PR-AUC, F1, recall, event detection and false alerts per hour | Critical |
| NFR2.2 | False-alert control | Suppress brief transients and disclose external-domain false alerts | Measured across distinct benign and external domains | Critical |
| NFR2.3 | Drowsiness reliability | Evaluate the drowsiness model separately from crash detection | Macro-F1, balanced accuracy, low-vigilance recall, event sensitivity, false alarms per hour | High |
| NFR2.4 | System availability | The crash monitor should stay operational during rides and soak tests | Soak tests on the target device; no 99% uptime claim | High |
| NFR2.5 | Voice-recognition reliability | Commands interpreted accurately in normal noise | ≥ 85% recognition (target) | Medium |
| NFR2.6 | GPS position accuracy | Receiver accuracy evaluated against a reference before relying on emergency locations | 5–10 m (original target) | High |

**Table 2.13:** NFR3: Usability

| ID | Requirement | Description | Metric / constraint | Priority |
|---|---|---|---|---|
| NFR3.1 | System usability | Ease of use measured through user feedback | SUS score ≥ 70 | High |
| NFR3.2 | Voice interaction | Commands simple and natural for riders | No training required | High |
| NFR3.3 | User-interface design | Clean, intuitive and consistent app interface | Standard UI/UX guidelines | High |
| NFR3.4 | Alert clarity | Alerts clearly noticeable while riding | Audio, visual and haptic feedback | Critical |
| NFR3.5 | Setup efficiency | New users configure the system quickly | ≤ 10 min setup | Medium |

**Table 2.14:** NFR4: Security and privacy

| ID | Requirement | Description | Metric / constraint | Priority |
|---|---|---|---|---|
| NFR4.1 | Secure data transmission | Network communication protected against interception | TLS encryption | Critical |
| NFR4.2 | Authentication security | User accounts securely managed | Hashed passwords and token sessions | Critical |
| NFR4.3 | Data access control | Only authorized users access personal data | Authenticated, per-user access rules | High |
| NFR4.4 | Privacy compliance | Users informed and consenting to data use | Consent-based data collection | High |

**Table 2.15:** NFR5–NFR8: Scalability, compatibility, physical and cost constraints

| ID | Requirement | Description | Metric / constraint | Priority |
|---|---|---|---|---|
| NFR5.1 | Scalability | Backend handles a growing number of users | ≥ 50 concurrent users | Medium |
| NFR5.2 | Modular architecture | Components independently updatable | Loosely coupled modules | High |
| NFR5.3 | Maintainability | Code easy to understand and modify | Coding standards and documentation | High |
| NFR5.4 | Error handling and logging | Issues recorded for debugging | Logging on each component | High |
| NFR6.1 | Android compatibility | App runs on a wide range of devices | Android 7.0+ (API 24), target SDK 36 | High |
| NFR6.2 | Web compatibility | Website works across major browsers | Chrome, Firefox, Safari, Edge | High |
| NFR6.3 | Bluetooth integration | Standard connection between helmet and phone | Bluetooth Low Energy GATT | High |
| NFR6.4 | Hardware compatibility | Components fit a standard helmet | Standard mounting | Medium |
| NFR6.5 | API interoperability | Integration with external services | REST/HTTPS APIs | High |
| NFR7.1 | Operating temperature | System functions in varied climates | −10 °C to 50 °C | High |
| NFR7.2 | Weather resistance | Components tolerate rain and humidity | Water-resistant design | High |
| NFR7.3 | Weight limitation | Added components do not affect comfort | ≤ 300 g | High |
| NFR7.4 | Charging convenience | Standard charging options | USB-C / micro-USB | Medium |
| NFR8.1 | Hardware cost control | Prototype within budget | Within allocated budget | High |
| NFR8.2 | Cloud cost optimization | Cost-efficient services during development | Free and sandbox tiers | Medium |
| NFR8.3 | Software cost efficiency | Prefer free and open-source tools | Open-source frameworks | High |
| NFR8.4 | Project timeline | Development aligned with the academic schedule | 8–9 months | Critical |

## 2.15 Requirements Traceability Matrix

Table 2.16 maps every requirement to its source, system component, verification method and current status. Status values are deliberately precise: *Validated* means measured evidence exists; *Implemented* means the feature exists in source code with the stated level of testing; *Not implemented*, *Not measured* and *Not tested* mean that no evidence exists.

**Table 2.16:** Requirements traceability matrix

| ID | Requirement | Source | Component | Priority | Verification method | Status |
|---|---|---|---|---|---|---|
| FR1.1 | 6-axis IMU monitoring and window maintenance at 50 Hz | Stakeholders, literature | Pi IMU acquisition | High | Unit and integration test | Validated (live IMU acquisition) |
| FR1.2 | Hybrid model with classifier and reconstruction outputs | Literature | Pi crash monitor | Critical | Offline evaluation, integration test | Validated (frozen offline and INT8) |
| FR1.3 | Three-window confirmation policy | Safety engineering | Pi crash monitor | Critical | Integration test | Validated (software and live trigger) |
| FR1.4 | Rider cancellation and emergency escalation | Safety requirements | GPIO18 alarm, SIM800L | Critical | System test | Partial (SMS path mocked; delivery not validated) |
| FR1.5 | Crash-event logging and audit trail | System requirements | Local logs | Medium | System test | Partial (local logs only) |
| FR2.1 | Timestamped video and landmark/quality acquisition | Research | Drowsiness runtime | High | Unit test | Implemented in software; Pi performance not measured |
| FR2.2 | Personal baseline and 12-feature physiology | Literature | Drowsiness runtime | High | Integration test | Implemented; unit and contract tested |
| FR2.3 | Three-class causal multimodal inference (ONNX) | User needs | Drowsiness model | Critical | Subject-disjoint evaluation, ONNX parity, causal replay | Development-evaluated; no Pi end-to-end benchmark |
| FR2.4 | Elapsed-time warnings and UNAVAILABLE handling | User feedback | Alert controller | Critical | System test | Implemented and tested in software; target device not verified |
| FR2.5 | Verified runtime threshold configuration | Research | Alert controller | Medium | Configuration review | Implemented; app synchronization not implemented |
| FR3.1 | MapLibre map, Photon search and OSRM routing | User needs | Android navigation | High | NavigationLogicTest; emulator | Implemented; field routing not benchmarked |
| FR3.2 | Off-route detection and online OSRM rerouting (no live traffic) | User convenience | NavigationGuide | Medium | NavigationLogicTest | Implemented; traffic-based routing not supported |
| FR3.3 | Turn-by-turn banners and spoken phone guidance | User needs | NavigationActivity, TextToSpeech | High | Navigation tests; emulator | Implemented; audio clarity while riding not measured |

| ID | Requirement | Source | Component | Priority | Verification method | Status |
|---|---|---|---|---|---|---|
| FR3.4 | Text, voice-search and BLE destination intake | User convenience | NavigationActivity, HelmetDataParser | Medium | Parser tests; manual scenarios | Implemented; physical BLE hand-off not verified |
| FR4.1 | Front and rear video recording at ≥ 720p | Evidence collection | Cameras | High | Recording test | Not implemented (photo and 10 s clip only) |
| FR4.2 | Voice-command image capture | User convenience | Voice assistant, USB camera | Medium | Integration test | Implemented (voice photo); not formally tested |
| FR4.3 | Video compression for storage | Technical constraint | Pi media capture | Medium | Performance test | Not tested |
| FR4.4 | Cloud video upload | Evidence backup | Cloud | High | Integration test | Not implemented |
| FR4.5 | Timestamp and GPS tagging of media | Evidence integrity | Pi media capture | High | System test | Not implemented |
| FR5.1 | Offline Rhasspy recognition and intent flow | User convenience | Rhasspy, MQTT | High | Service inventory and code-path review | Configured on the Pi; recognition accuracy not measured |
| FR5.2 | Local assistant sensor, media and navigation actions | User needs | `helmx_assistant.py`, Mosquitto | Medium | Service inventory | Implemented; scenarios not independently re-run |
| FR5.3 | BLE destination hand-off from the assistant | Reliability | `navigation_target.json`, BLE server | High | Protocol and parser review | Interface documented; physical hand-off not verified |
| FR6.1 | MQ135 digital air-quality threshold | Health awareness | Pi BCM17, BLE | High | Unit test | Configured; quantitative accuracy not measured |
| FR6.2 | DHT22 temperature and humidity | Environmental data | Pi BCM22, BLE | Medium | Unit test | Configured; accuracy against a reference not measured |
| FR6.3 | BLE environment publication and local snapshot | Analytics | `helmx-ble` service | Medium | System test | Documented service path; link validation pending |
| FR7.1 | BLE GATT telemetry parsing and live UI | User monitoring | HelmetBleManager, StateFlow | High | Parser unit tests; emulator | Implemented; physical Pi BLE integration not verified |
| FR7.2 | Online maps, offline Lahore map, OSRM/Photon navigation | User insights | Android navigation, local tiles | High | NavigationLogicTest, OfflineTest; emulator | Implemented; real-world coverage not benchmarked |
| FR7.3 | Foreground GPS ride summary and Firestore history | User control | RideTrackingService, Firestore | Medium | RideTrackerTest; emulator GPS | Implemented; field precision not verified |
| FR7.4 | Phone crash/drowsiness alerts and opt-in SMS/call | User safety | SafetyMonitor | Critical | SafetySettingsTest; mock scenarios | Implemented; delivery not demonstrated |
| FR7.5 | Email/password login and per-user Firestore isolation | Security | Firebase Auth, Firestore rules | High | Authentication UI; rules review | Implemented; no penetration test |
| FR8.1 | Firebase Auth and per-user Firestore data | System integration | Android, Firebase | High | Integration test | Implemented; no independent security audit |
| FR8.2 | Separate PostgreSQL order and contact records | Data management | Web, Prisma | High | Integration test | Implemented; production database not tested |

| ID | Requirement | Source | Component | Priority | Verification method | Status |
|---|---|---|---|---|---|---|
| FR8.3 | Server-priced Stripe test checkout | System integration | Next.js checkout API | Critical | Vitest catalog tests | Implemented and unit-tested; live transaction not re-run |
| FR8.4 | Signed webhook, idempotent settlement, receipt attempt | System integration | Stripe webhook, SMTP | Critical | Integration test | Implemented; delivery guarantee not established |
| NFR1.1 | Edge crash scoring within the timing budget | Performance goal | Pi crash monitor | Critical | Performance test | Validated (p99 well below 10 ms); not a hard real-time guarantee |
| NFR1.2 | Drowsiness camera-to-alert latency on the Pi | Performance goal | Drowsiness runtime | Critical | Performance test | Not measured (workstation model-only p95 43.17 ms) |
| NFR1.3 | Cancellation, then local GSM/SMS attempt | Performance goal | GPIO18, SIM800L | Critical | Performance test | Not verified: live SMS delivery not measured |
| NFR1.4 | App load and render time | User experience | Android app | High | Cold-start timing benchmark | Not measured |
| NFR1.5 | Battery endurance | Hardware constraint | Power banks | High | System test | Not measured |
| NFR2.1 | Held-out and external crash metrics | Quality goal | Crash model | Critical | Validation test | Evaluated; domain-shift limitations remain |
| NFR2.2 | False-alert validation across domains | Quality goal | Crash model | Critical | Validation test | Evaluated; external false alerts remain |
| NFR2.3 | Drowsiness classification and event reliability | Quality goal | Drowsiness model | High | Subject-disjoint evaluation | Measured on development clips; no external domain |
| NFR2.4 | Operational reliability during soaks | Reliability goal | Pi crash monitor | High | Live soaks, timing-validity analysis | Validated in 90- and 180-minute soaks; no lifetime guarantee |
| NFR2.5 | Voice recognition ≥ 85% | Quality goal | Voice assistant | Medium | Validation test | Not measured |
| NFR2.6 | GPS location accuracy | Safety requirement | NEO-6M GPS | High | Ground-truth comparison | Not measured |
| NFR3.1 | SUS score ≥ 70 | Usability goal | Android app | High | Questionnaire study | Not measured (no SUS study) |
| NFR3.2 | Natural voice interaction | Usability goal | Voice assistant | High | User acceptance | Not measured |
| NFR3.3 | Consistent UI design | Usability goal | Android app, website | High | Design review | Implemented (Material components); not user-tested |
| NFR3.4 | Noticeable alerts | Safety goal | Pi alarm, SafetyMonitor | Critical | System test | Implemented (audio, vibration, notification); not rider-tested |
| NFR3.5 | Setup ≤ 10 minutes | Usability goal | Android app | Medium | User acceptance | Not measured |
| NFR4.1 | TLS for network communication | Security standard | Firebase, Vercel, Photon/OSRM | Critical | Security test | Platform-provided HTTPS; BLE link security not assessed |
| NFR4.2 | Secure authentication | Security standard | Firebase Authentication | Critical | Security test | Provided by Firebase; no independent audit |
| NFR4.3 | Authenticated per-user access | Security standard | Firestore rules | High | Rules review | Implemented (per-user rules) |

| ID | Requirement | Source | Component | Priority | Verification method | Status |
|---|---|---|---|---|---|---|
| NFR4.4 | Consent-based data collection | Privacy standard | Android permissions | High | Review | Partial (runtime permissions and privacy screen) |
| NFR5.1 | 50 concurrent users | Scalability goal | Firebase, PostgreSQL | Medium | Load test | Not measured |
| NFR5.2 | Modular architecture | Maintainability | All components | High | Design review | Met (independent branches and services) |
| NFR5.3 | Maintainable, documented code | Maintainability | All components | High | Code review | Met (documented repositories and tests) |
| NFR5.4 | Error handling and logging | Maintainability | Pi services, apps | High | Review | Partial (rotating crash-monitor log; no central logging) |
| NFR6.1 | Android 7.0+ (API 24) | Platform support | Android app | High | Gradle configuration review | Configured; cross-device testing not performed |
| NFR6.2 | Cross-browser website | Platform support | Website | High | Compatibility test | Not tested |
| NFR6.3 | BLE helmet–phone link | Platform support | BLE server, Android | High | Integration test | Implemented on both sides; physical link not verified |
| NFR6.4 | Standard helmet mounting | Hardware | Prototype | Medium | Inspection | Single prototype on one helmet |
| NFR6.5 | External API integration | Interoperability | Firebase, Photon, OSRM, Stripe | High | Integration test | Implemented |
| NFR7.1 | −10 °C to 50 °C operation | Environmental | Helmet hardware | High | Environmental chamber test | Not measured |
| NFR7.2 | Water resistance | Environmental | Helmet hardware | High | Water-ingress test | Not measured |
| NFR7.3 | Added weight ≤ 300 g | Physical | Helmet hardware | High | Weighing | Not measured |
| NFR7.4 | Standard charging | Physical | Power banks | Medium | Inspection | Met (USB power banks) |
| NFR8.1 | Within budget | Resource constraint | All | High | Budget review | Historical estimate PKR 103,000; not audited |
| NFR8.2 | Low cloud cost | Resource constraint | Cloud services | Medium | Review | Met (free and sandbox tiers) |
| NFR8.3 | Open-source tools | Resource constraint | All | High | Review | Met |
| NFR8.4 | 8–9 month timeline | Time constraint | All | Critical | Schedule review | Session work completed; research extended to Nov. 2026 |

## 2.16 Use Case Descriptions

This section describes the six principal use cases in a common format. Each use case gives its identifier, actors, priority and related requirements, followed by the description, preconditions, main flow, alternate flows and postconditions. UC-01 to UC-06 cover the helmet, the Android application and the website; theft detection is not included because it was not implemented.

### 2.16.1 UC-01: Crash Detection and Emergency Alert

**Table 2.17:** UC-01 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-01 |
| Name | Hybrid crash detection, rider confirmation and emergency notification |
| Primary actor | Motorcycle rider |
| Secondary actors | Configured emergency contact(s); Pi-connected SIM800L GSM modem and NEO-6M GPS |
| Priority | Critical |
| Related requirements | FR1.1–FR1.5, NFR1.1, NFR1.3, NFR2.1, NFR2.2 |

**Description:** The system continuously monitors helmet-mounted IMU data, scores each motion window with the hybrid model, confirms a crash candidate across three consecutive positive windows and presents a rider-cancellable local warning before any emergency notification is attempted.

**Preconditions:** the helmet is powered and the crash-monitor service is running; the IMU is streaming at the configured rate; the INT8 model, normalizer, thresholds and runtime configuration are available locally; emergency contacts are configured. A GPS fix and GSM service are not required for local detection.

**Main flow:**

1. The rider starts a journey and the IMU stream is buffered continuously.
2. The preprocessing pipeline harmonizes units, prepares a 60 × 6 window and extracts engineered features.
3. The hybrid model outputs a crash probability and a reconstruction error for the window.
4. The decision module applies calibrated OR-fusion and counts consecutive positive windows.

5. After three consecutive positive windows the helmet enters the alert state and activates the GPIO18 alarm.
6. The rider has 30 seconds to cancel the alert through the cancellation signal written by the voice workflow.
7. If the alert is cancelled, the event is logged as non-escalated and monitoring resumes after the 60-second cooldown.
8. If it is not cancelled, the Pi requests a GPS fix (15-second timeout) and attempts to send an SMS through the SIM800L modem; delivery is not asserted without confirmation.
9. Event metadata, scores and the alert outcome are logged.

**Alternate flows:**

- **A1, benign high-intensity motion:** a speed bump, drop or harsh braking produces a transient positive window; because the three-window condition is not met, no alert is raised.
- **A2, communication unavailable:** when GPS or GSM is unavailable, the software returns a structured `CrashAlertResult` reporting failure or missing location; no automatic SMS retry was demonstrated.

**Postconditions:** the alert is either cancelled or escalated through the emergency-notification path, and an incident record exists for audit and error analysis.

### 2.16.2 UC-02: Drowsiness Detection and Alert

**Table 2.18:** UC-02 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-02 |
| Name | Camera-based drowsiness detection and warning |
| Primary actor | Rider |
| Secondary actors | Camera; MediaPipe FaceLandmarker; ONNX Runtime; alert controller |
| Priority | Critical |
| Related requirements | FR2.1–FR2.5, NFR1.2, NFR2.3 |

**Description:** Camera-based driver-state monitoring combines per-rider neutral calibration, quality-scored MediaPipe landmarks, a visual/physiology causal temporal model and an independent alert controller. The prototype is a software and development-evaluation system, not a proven road-safety device.

**Preconditions:** the camera is accessible; the FaceLandmarker asset and exported ONNX model are available; the rider is stationary and willing to complete neutral calibration.

**Main flow:**

1. Capture frames, timestamp observations with a monotonic clock and extract ocular, mouth and head landmarks with image-quality scores.
2. Collect high-quality neutral measurements, fit the per-rider EAR, MAR and pose baseline and build a rolling physiology history.
3. Normalize a 24-observation visual sequence and 12 physiology features, with two measured reliability channels.
4. Produce calibrated alert, low-vigilance and drowsy probabilities with the quality-gated MobileNetV3-Small and causal TCN model.
5. Convert sustained drowsiness probability into CAUTION, WARNING or CRITICAL through elapsed-time thresholds, hysteresis and cooldown.
6. Raise an on-screen status and a rate-limited audio alarm for WARNING and CRITICAL, and log model and decision evidence where configured.

**Alternate flows:**

- **A1, poor visibility:** if face landmarks disappear or quality stays low, detection timers reset and the state becomes UNAVAILABLE after the configured interval; low-quality input is never interpreted as evidence of alertness.
- **A2, short blinks:** short-lived ocular changes do not raise a warning on their own; the classifier and elapsed-time persistence policy jointly determine warnings.

**Postconditions:** an interpretable alert state is available, with missing evidence reported separately from alertness. No field-validated prevention claim is made.

### 2.16.3 UC-03: Smart Navigation Assistance

**Table 2.19:** UC-03 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-03 |
| Name | Smart navigation assistance |
| Primary actor | Motorcycle rider |
| Secondary actors | Android voice search; Photon geocoding; OSRM routing; helmet BLE destination feed (optional) |
| Priority | High |
| Related requirements | FR3.1–FR3.4, FR5.3, FR7.2 |

**Description:** The rider requests navigation by typing, by Android voice search, from recent destinations or through a destination sent from the helmet. Spoken maneuver instructions are produced on the smartphone; safe hands-free use depends on permissions, circumstances and environment.

**Preconditions:** the HelmX Android app is installed on a location-enabled phone with location permission; GPS signal and phone audio output are available; BLE pairing with the helmet is optional.

**Main flow:**

1. The rider opens HelmX Navigation.
2. The rider types or speaks a place name, selects a recent place, or the helmet reports a destination over BLE.
3. Photon resolves candidate places when internet access is available.
4. The app sends a routing request to OSRM from the current GPS position.
5. When routing succeeds, NavigationActivity prepares the route preview, maneuver sequence and optional foreground ride recording.
6. The navigation session begins on the smartphone.
7. NavigationGuide tracks GPS progress along the OSRM route geometry.
8. The map shows the route, the next-turn banner, current speed and progress.
9. NavigationGuide computes the next maneuver and the app announces turns by phone audio.
10. Navigation continues until the destination is reached.
11. The app detects arrival near the route endpoint and ends guidance.
12. The session ends and ride statistics are updated.

**Alternate flows:**

- **A1, voice query not recognized:** background noise prevents recognition; voice search returns no usable query and the rider retries through the phone interface. No specific spoken error prompt is implemented.
- **A2, route deviation:** after a persistent off-route condition (two consecutive fixes beyond 45 m) the app requests a new OSRM route, subject to connectivity and a reroute cooldown, and refreshes the route and guidance. When offline, the app says "You are off the route. Rerouting needs an internet connection."; no generic "recalculating route" prompt is implemented.

- **A3, manual cancellation:** the rider stops guidance with the on-screen controls; voice cancellation and a spoken cancellation confirmation are not implemented.

**Postconditions:** guidance stops after detected arrival or cancellation (safe arrival is not guaranteed); the ride summary stores destination, distance, duration and average speed, and the complete GPS track is not uploaded; the system is ready for the next request. No assumption is made that navigation can always be started, adjusted or cancelled safely without touching the phone.

### 2.16.4 UC-04: Ride Recording and Analytics Review

**Table 2.20:** UC-04 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-04 |
| Name | GPS ride recording and Firestore summary review |
| Primary actor | Motorcycle rider |
| Secondary actors | Android foreground location service; Firebase Authentication; Cloud Firestore |
| Priority | High |
| Related requirements | FR7.3, FR7.5, FR8.1 |

**Description:** The signed-in rider starts and stops GPS ride recording on the Navigation screen and then reviews saved ride summaries in Analytics. The app does not upload a complete GPS trace and does not store per-ride drowsiness counts, harsh-braking counts, air-quality ranges or speed violations.

**Preconditions:** the rider is signed in with Firebase Authentication; location permission is granted; a recording has been started or previous rides exist in Firestore.

**Main flow:**

1. The rider selects Start; RideTrackingService records GPS fixes from the Fused Location Provider in a foreground service.
2. RideTracker ignores fixes less accurate than 50 m and movements under 5 m to suppress stationary jitter.
3. The rider stops recording; sessions shorter than 60 seconds or 0.1 km are discarded.
4. The app saves start and end time, destination, distance, duration and average speed to `users/{uid}/rides`; the coordinate trace is not uploaded.
5. Analytics reads the rider's summaries and shows ride history and daily totals.

6. Live temperature, humidity and air quality are displayed separately when the helmet is connected; they are not stored as per-ride series.

**Alternate flows:**

- **A1, offline:** Firestore queues the ride write locally for later synchronization; no custom offline banner is implemented.
- **A2, no GPS or permission:** recording cannot start or produce a valid summary, and the app reports the problem instead of inventing a route.

**Postconditions:** a qualifying ride summary is stored for the account or queued by Firestore. No maximum speed, harsh-braking count, drowsiness count or raw GPS trace is stored.

### 2.16.5 UC-05: Offline Voice Command on the Helmet

**Table 2.21:** UC-05 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-05 |
| Name | Offline voice command handled by the helmet assistant |
| Primary actor | Rider |
| Secondary actors | Rhasspy; Mosquitto MQTT; `helmx_assistant.py`; UART5 voice module; BLE server; Android app |
| Priority | High |
| Related requirements | FR4.2, FR5.1–FR5.3, FR6.1, FR6.2 |

**Description:** The rider speaks a wake word and a command. The helmet recognizes it offline and performs a local action or forwards a destination to the phone.

**Preconditions:** the `rhasspy`, `mosquitto`, `helmx_assistant` and `helmx-ble` services are running; the USB microphone and speakers are available.

**Main flow:**

1. The rider says the wake word, detected by Porcupine.
2. Kaldi converts the following speech to text and fsticuffs recognizes the intent.
3. Rhasspy publishes the intent on the local Mosquitto MQTT bus.
4. `helmx_assistant.py` handles the intent: it reads sensor values, plays music with mpg123, takes a photo with fswebcam or records an approximately 10-second video with ffmpeg; media is stored in the helmet vault.
5. For a navigation request, the assistant writes `navigation_target.json`; the BLE server sends the destination to the phone and clears the file.

6. The assistant confirms the action through NanoTTS on the helmet speakers.

**Alternate flows:**

- **A1, hardware module:** the UART5 voice-recognition module recognizes a command directly and passes it to the assistant.
- **A2, phone not connected:** the destination cannot be delivered; the rider can still enter it on the phone.

**Postconditions:** the requested action is attempted and logged locally. Recognition accuracy in wind noise and a physical end-to-end voice-to-route trace have not been measured.

### 2.16.6 UC-06: Configure and Order a Helmet on the Website

**Table 2.22:** UC-06 summary

| Attribute | Details |
|---|---|
| Use case ID | UC-06 |
| Name | Configure a helmet and complete a test-mode order |
| Primary actor | Website visitor |
| Secondary actors | Stripe Checkout (test mode); PostgreSQL; SMTP server |
| Priority | Medium |
| Related requirements | FR8.2–FR8.4, NFR6.2 |

**Description:** A visitor configures a helmet in the 3D configurator, pays in Stripe test mode and receives an itemized receipt.

**Preconditions:** the website is deployed with database, Stripe test keys and SMTP credentials.

**Main flow:**

1. The visitor opens the configurator and rotates the 3D helmet preview.
2. The visitor selects parts and one storage tier; the page shows a quote from the shared catalog.
3. The visitor enters an email address and selects Pay with Stripe.
4. The checkout API validates the part IDs, recomputes the PKR total on the server, stores a PENDING order and creates a Stripe Checkout session.
5. The visitor pays with a Stripe test card.
6. The signed webhook or the success page calls the idempotent settlement function, which changes the order to PAID exactly once.

7. The winning transition attempts one itemized receipt email.

**Alternate flows:**

- **A1, tampered request:** an unknown part ID is rejected and no order is created.
- **A2, cancelled or failed payment:** the session expires or fails and the order is marked FAILED; a PAID order is never downgraded.

**Postconditions:** the order is recorded in PostgreSQL. No real money is charged and no fulfilment process exists.

CHAPTER 3

# 3 System Design

This chapter presents the HelmX design at two levels. The proposal-stage design diagrams are retained because they define the proposed scope of the project; each is followed by the current diagram that reflects what was actually implemented. Subsystem designs for crash detection, drowsiness detection, the Android application, the helmet services and the web platform follow.

## 3.1 Use Case Design

### 3.1.1 Original Use Case Design

Figure 3.1 is the use-case design produced for the original FYP proposal. It shows five actors (web user, administrator, app user, rider and the helmet subsystems) and the full proposed scope, including the website, the Android application, crash and drowsiness detection, IMU sensing, media recording, the AI assistant and navigation.

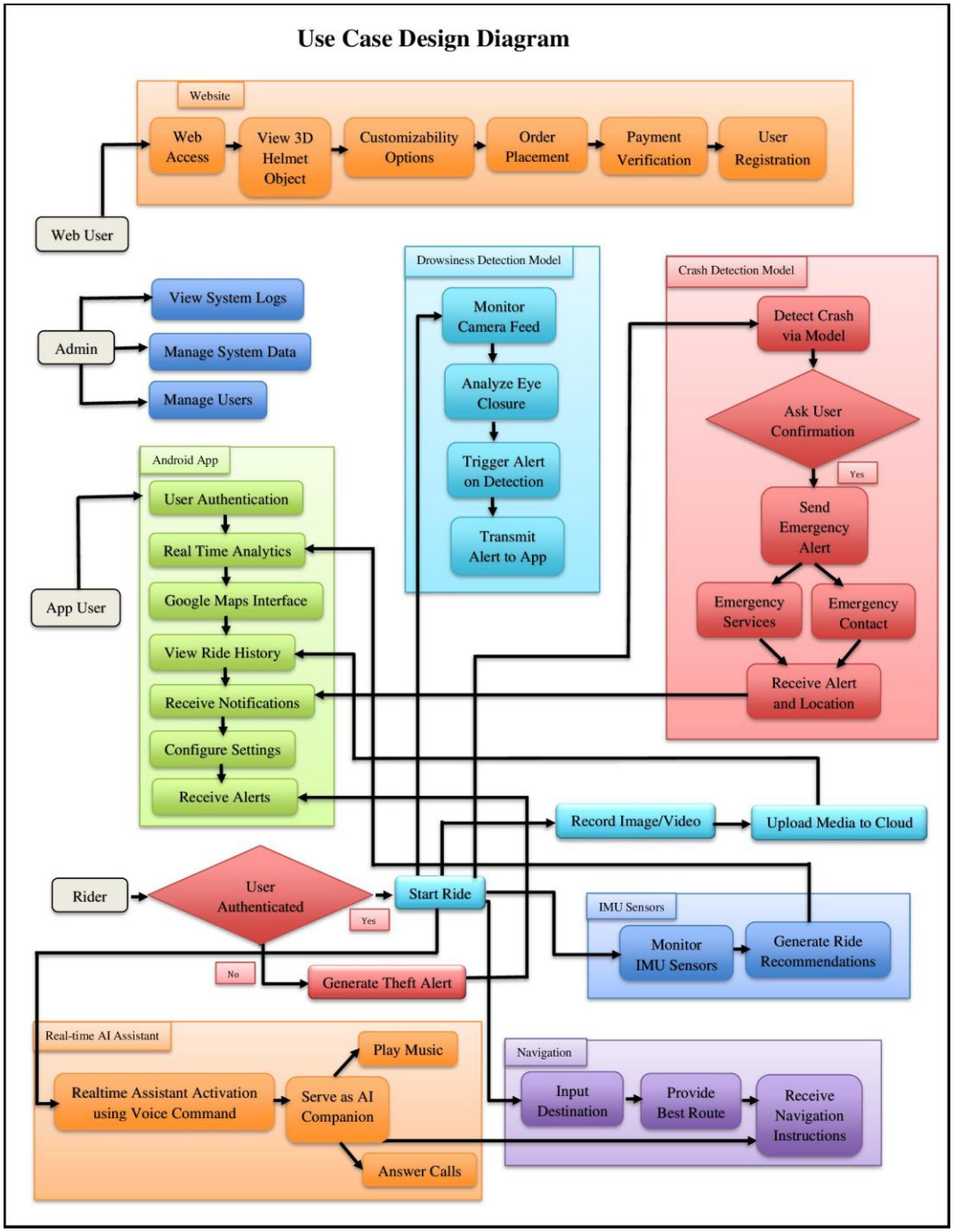



**Figure 3.1:** Original FYP use-case design diagram (proposed scope).

*Elements that were not implemented as drawn include the administrator functions, Google Maps, automatic cloud media upload, theft alerts, ride recommendations from IMU data and assistant call answering; see Figure 3.2 and Section 2.16.*

### 3.1.2 Implemented Use Cases

Figure 3.2 summarizes the rider-facing use cases that are implemented. The detailed use-case descriptions UC-01 to UC-06 are given in Section 2.16. Relative to the original design, navigation uses MapLibre, Photon and OSRM instead of Google Maps; drowsiness is detected by a quality-gated temporal model rather than a fixed eye-closure

rule; and the helmet's crash and drowsiness flags reach the phone only through a BLE protocol whose Pi-side publisher is not yet implemented.

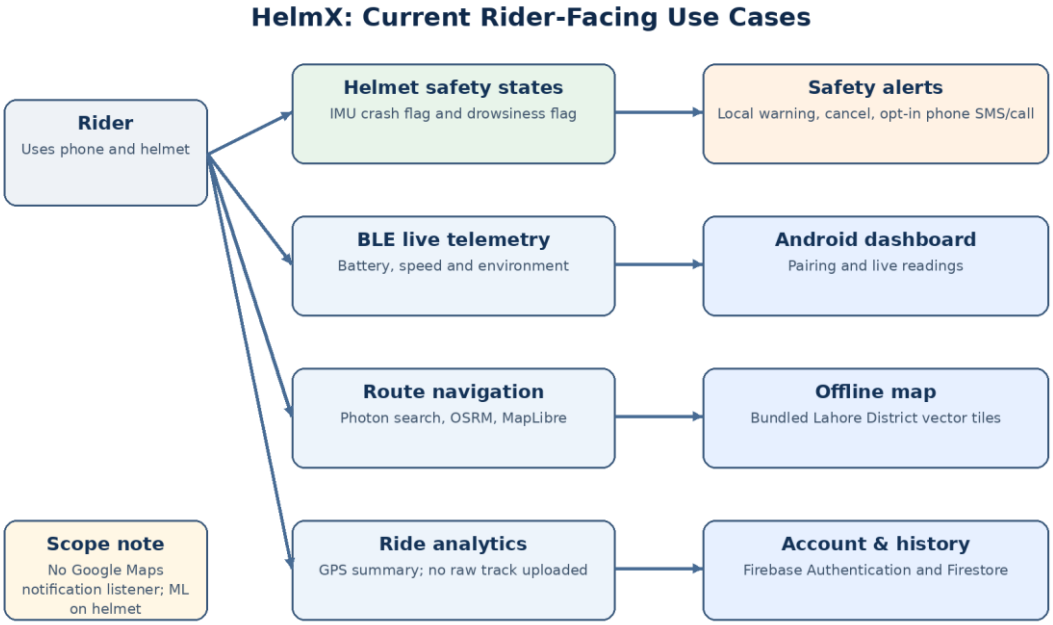



**Figure 3.2:** Implemented rider-facing use cases: helmet safety states, BLE telemetry, navigation with an offline map, and ride analytics with Firebase accounts.

## 3.2 System Architecture

### 3.2.1 Original Four-Layer Architecture

The original proposal organized HelmX into four layers (Figure 3.3): a hardware and sensor layer, an edge-AI processing layer on the Raspberry Pi, a cloud services layer and a user-application layer. The edge-AI and hardware layers were implemented largely as proposed. The cloud layer was not: AWS IoT Core, Node.js REST APIs, MongoDB Atlas and Twilio were replaced by Firebase Authentication and Cloud Firestore for the Android application, PostgreSQL and Stripe for the website, and a GSM modem connected directly to the Pi for emergency SMS.

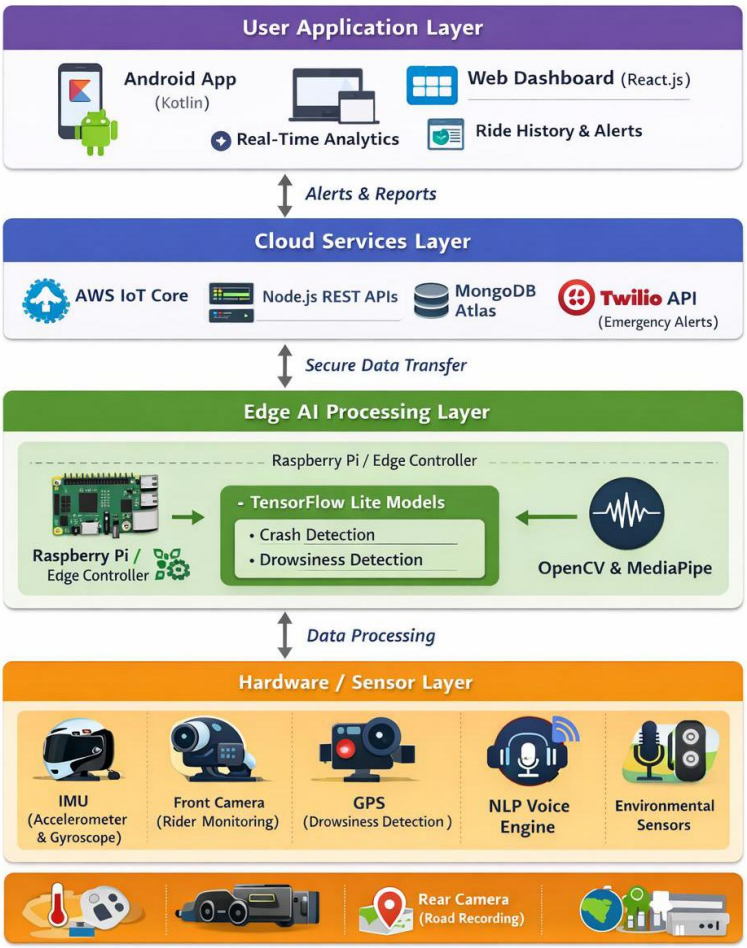



**Figure 3.3:** Original four-layer HelmX architecture from the FYP proposal (proposed design).
*The cloud services shown (AWS IoT Core, Node.js REST APIs, MongoDB Atlas, Twilio) were not used in the implemented system.*

### 3.2.2 Implemented Architecture

Figure 3.4 shows the implemented architecture with the corrected crash-alert route. The safety-critical path is entirely local to the helmet: the Raspberry Pi samples the IMU, runs the INT8 model and three-window confirmation, drives the GPIO18 alarm with a 30-second cancellation window and a 60-second cooldown, and attempts an SMS through the SIM800L GSM modem with a GPS fix. Cloud services support the applications but are not in the crash-SMS path. The phone and Pi emergency paths are independent; their coordination has not been verified.

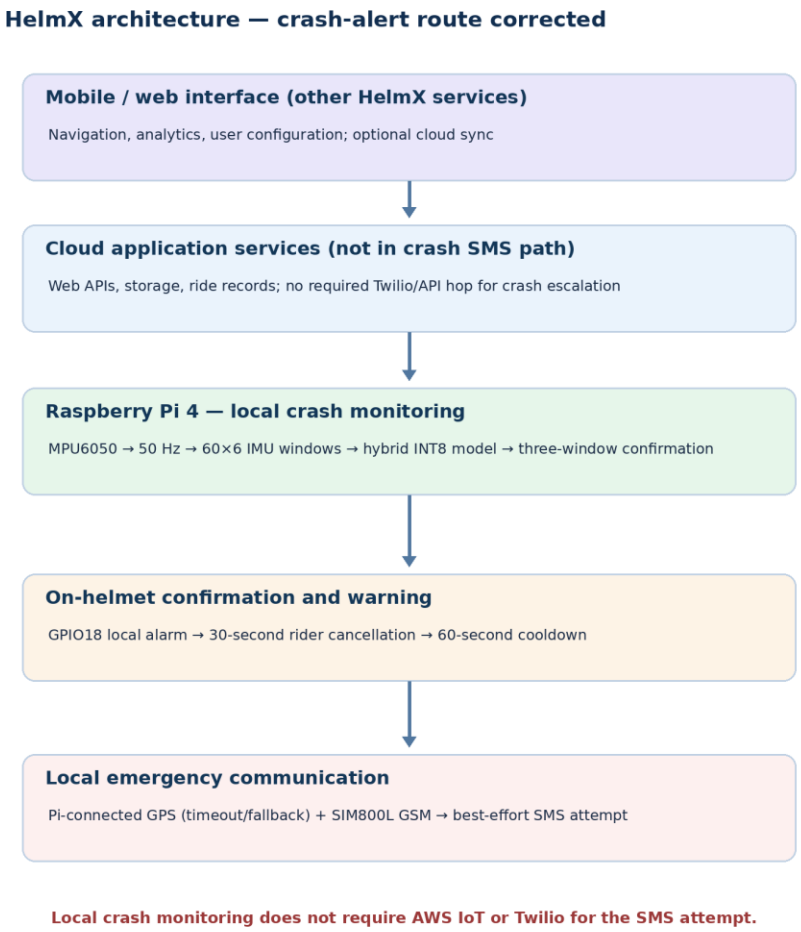



**Figure 3.4:** Implemented HelmX architecture with the corrected local crash-alert route.

## 3.3 Crash Detection Subsystem Design

The crash detector (Figure 3.5) takes a 60 × 6 IMU window (1.2 s at 50 Hz) together with seven engineered motion features. A shared Conv1D encoder feeds two branches: a decoder that reconstructs the input window, whose reconstruction error acts as an anomaly score, and a dense classifier that outputs a crash probability. A window is suspicious when either signal crosses its calibrated threshold (OR fusion), and an alert requires three consecutive suspicious windows. Figure 3.6 shows the end-to-end workflow from acquisition to logging and cooldown.

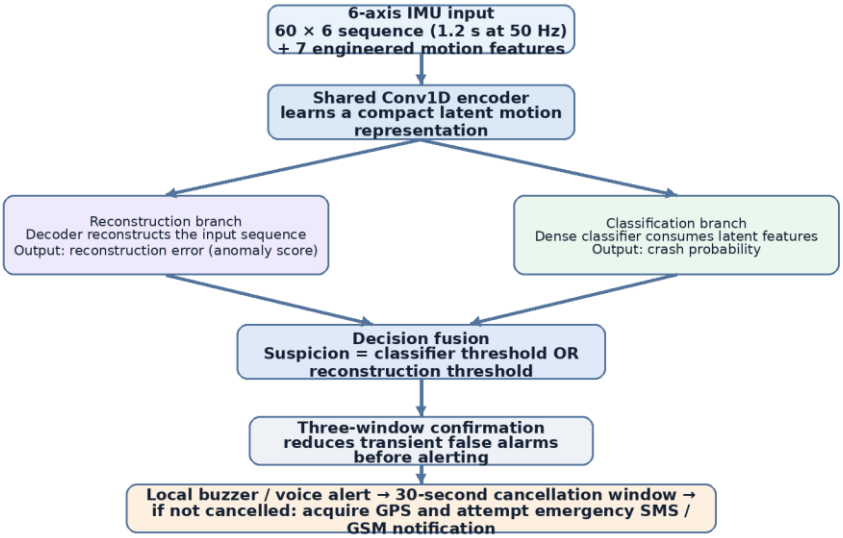



**Figure 3.5:** Hybrid crash-detection model architecture used in the helmet edge-AI layer.

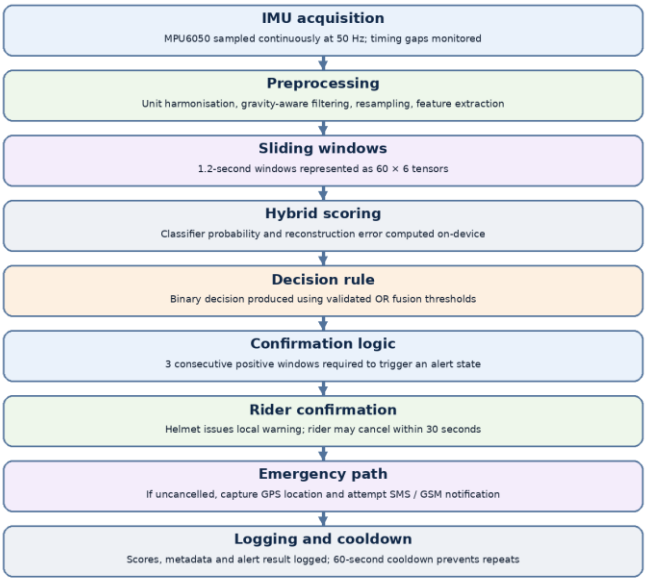



**Figure 3.6:** End-to-end crash detection, confirmation and alert workflow on the Raspberry Pi.

## 3.4 Drowsiness Detection Subsystem Design

The drowsiness model (Figure 3.7) fuses a MobileNetV3-Small embedding of a 160 × 160 face crop with a 192-dimensional projection of twelve causal physiological features. A learned reliability gate weights the two streams using two measured quality channels, four depthwise-separable causal TCN blocks with dilations 1, 2, 4 and 8 model time, and temperature-calibrated logits give alert, low-vigilance and drowsy probabilities. The twelve features are computed relative to a per-rider neutral calibration over a 60-second causal history (Figure 3.8). An elapsed-time controller (Figure 3.9) converts probabilities

into CAUTION, WARNING and CRITICAL states with hysteresis, quality suppression and cooldown, and reports UNAVAILABLE when input quality is poor.

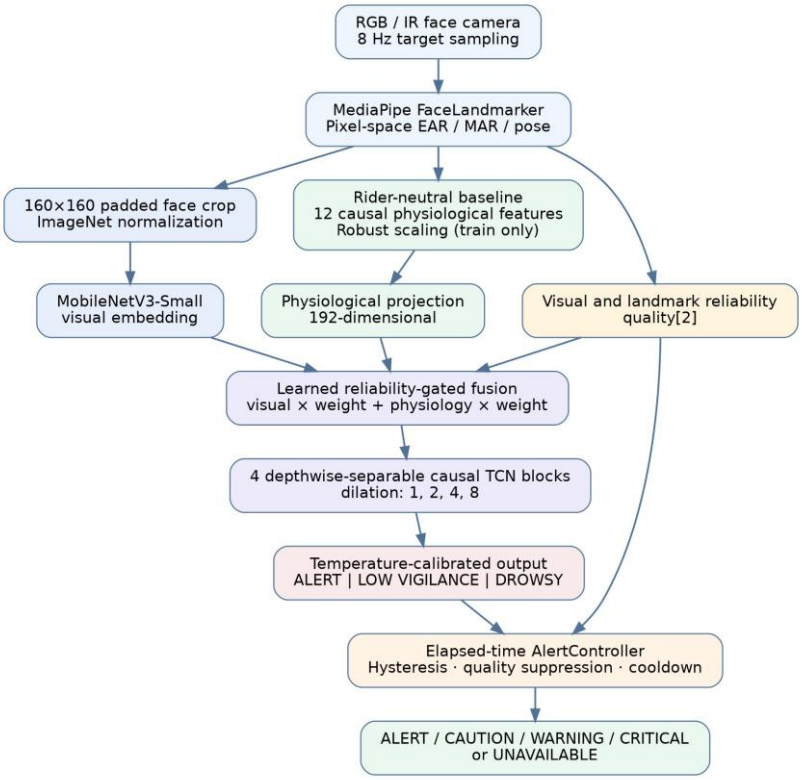



**Figure 3.7:** Quality-aware causal drowsiness model and warning path.

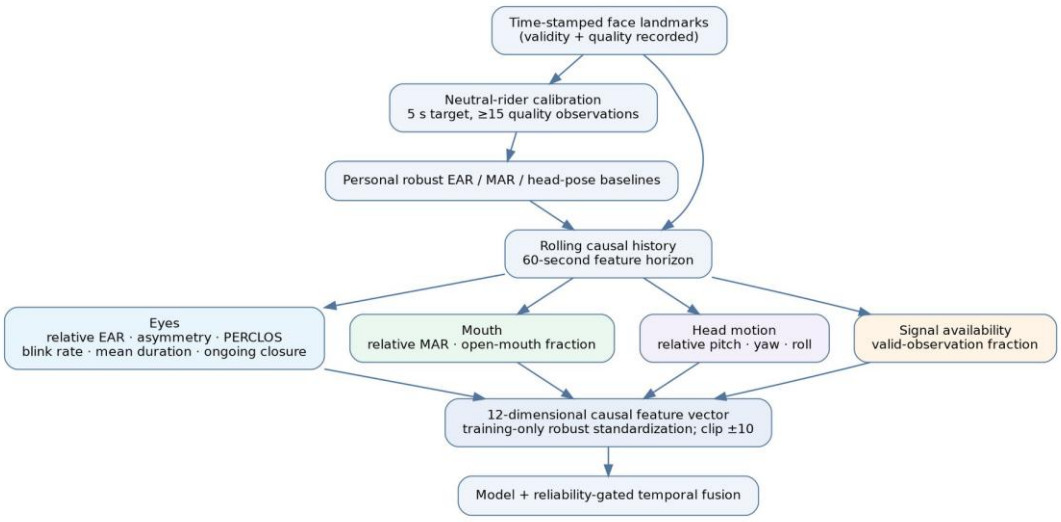



**Figure 3.8:** Personalized calibration and 12-feature causal physiology extraction.

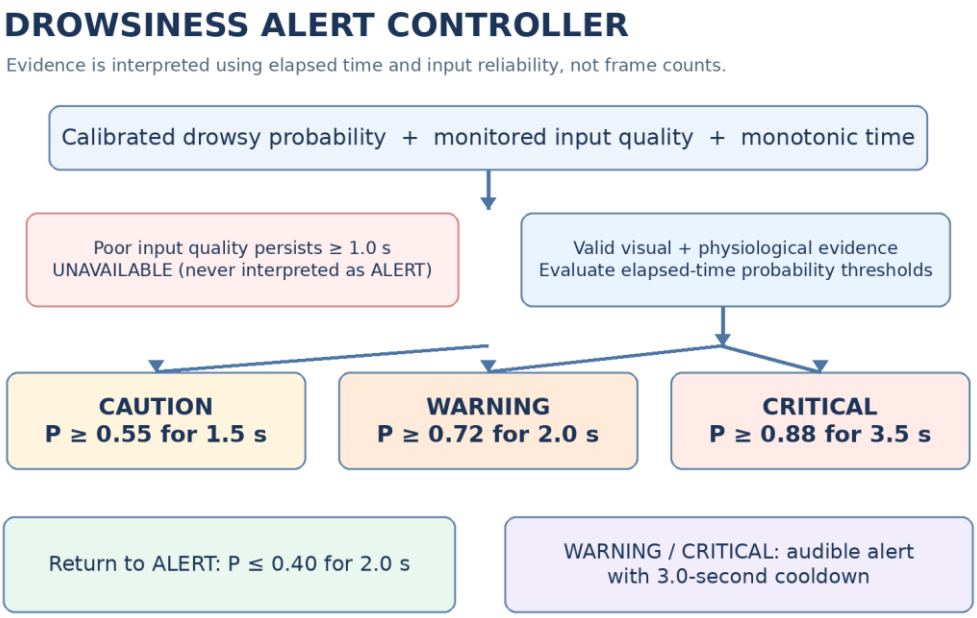



**Figure 3.9:** Elapsed-time alert-controller state logic (committed default configuration).

## 3.5 Android Application Design

The Android application is a BLE client and service-oriented phone application (Figure 3.10). HelmetBleManager connects to the helmet peripheral and passes parsed telemetry through HelmetDataParser into a StateFlow observed by the screens and by the app-wide SafetyMonitor. Firebase Authentication and Cloud Firestore hold accounts, emergency contacts and ride summaries, and RideTrackingService records filtered GPS fixes. The crash and drowsiness models do not run on the phone. The phone-side safety workflow is shown in Figure 3.11 and the navigation boundaries in Figure 3.12.

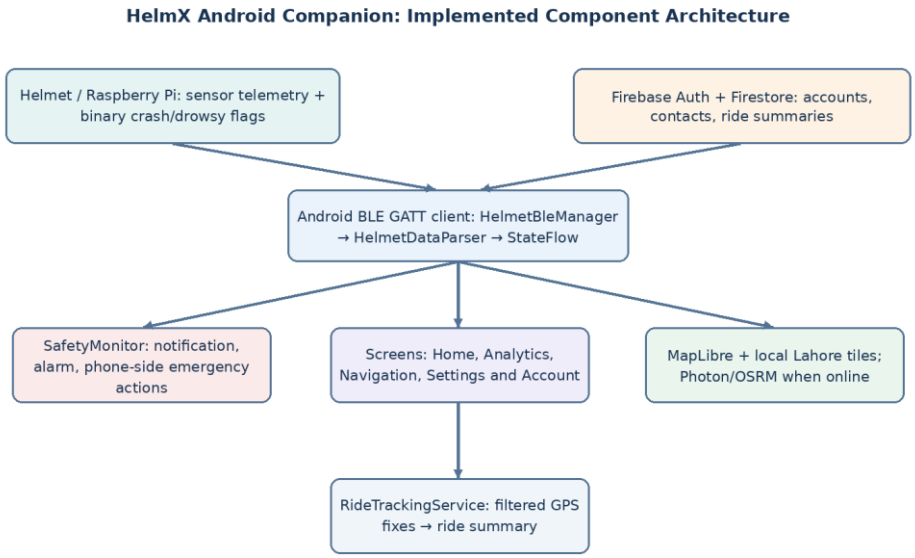



**Figure 3.10:** Android companion architecture.

*The Pi-side crash/drowsiness flag publisher (top left) is a defined protocol only;* `crash_monitor.py` *v1.1.0 does not publish BLE flags.*

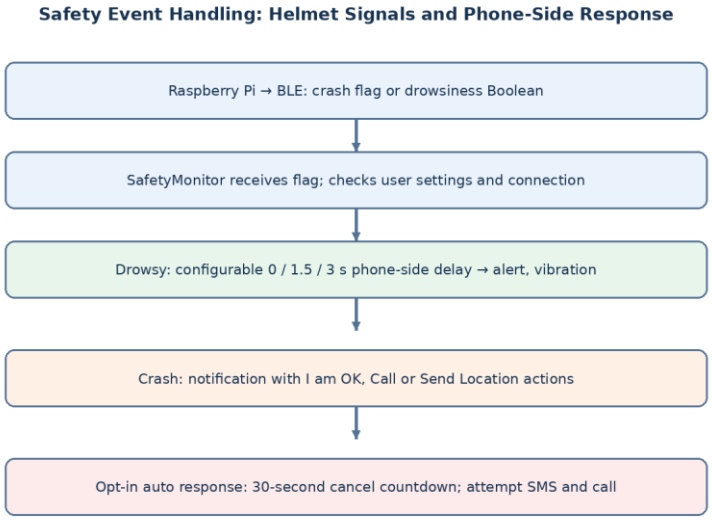



**Figure 3.11:** Android phone-side safety-event handling and optional emergency response.

*The first step assumes a helmet BLE flag publisher that is not yet implemented on the Pi.*

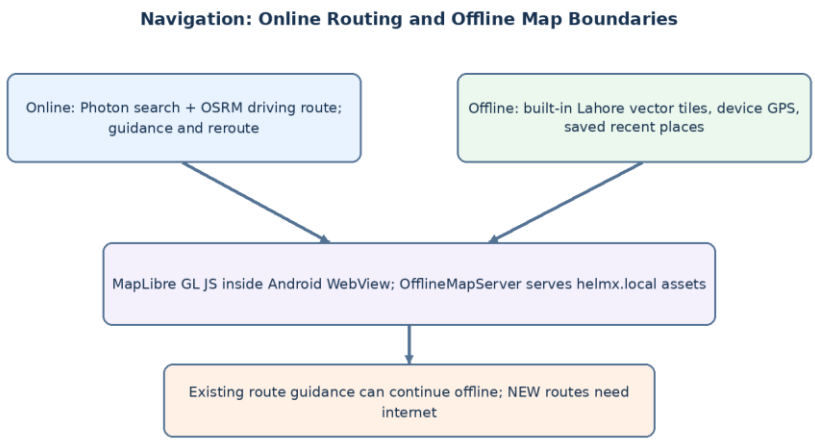



**Figure 3.12:** Online routing and offline Lahore map boundaries.

## 3.6 Helmet Services and Voice Design

Figure 3.13 summarizes the Raspberry Pi host, its peripherals and the on-helmet services documented in the main-branch hardware reference [8]. The offline voice path is shown in Figure 3.14: Rhasspy recognizes speech locally and publishes intents on a local MQTT bus, the assistant performs the action, and navigation requests are forwarded to the phone over BLE.

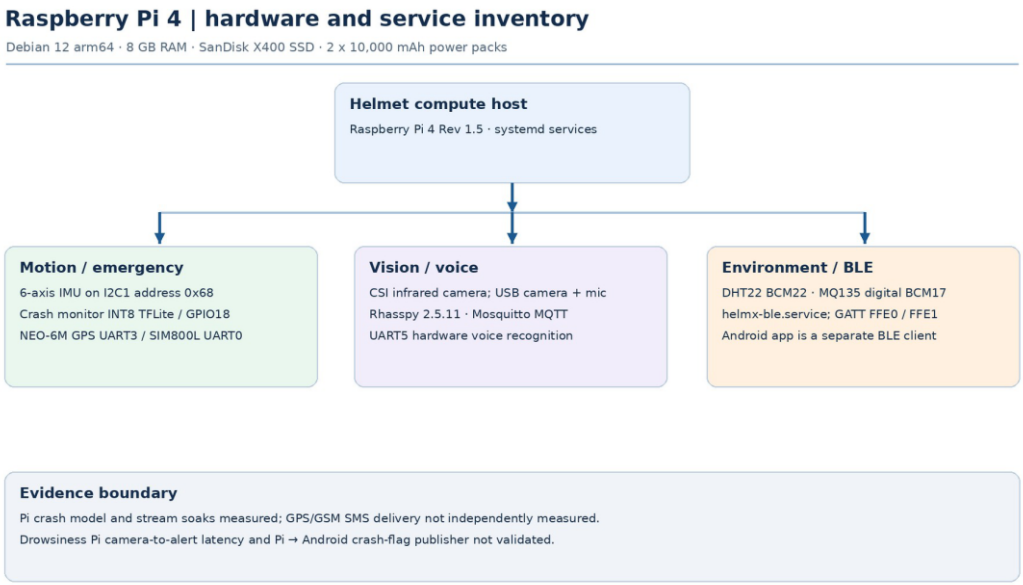



**Figure 3.13:** Raspberry Pi host, peripherals, on-helmet services and Android communication boundaries.

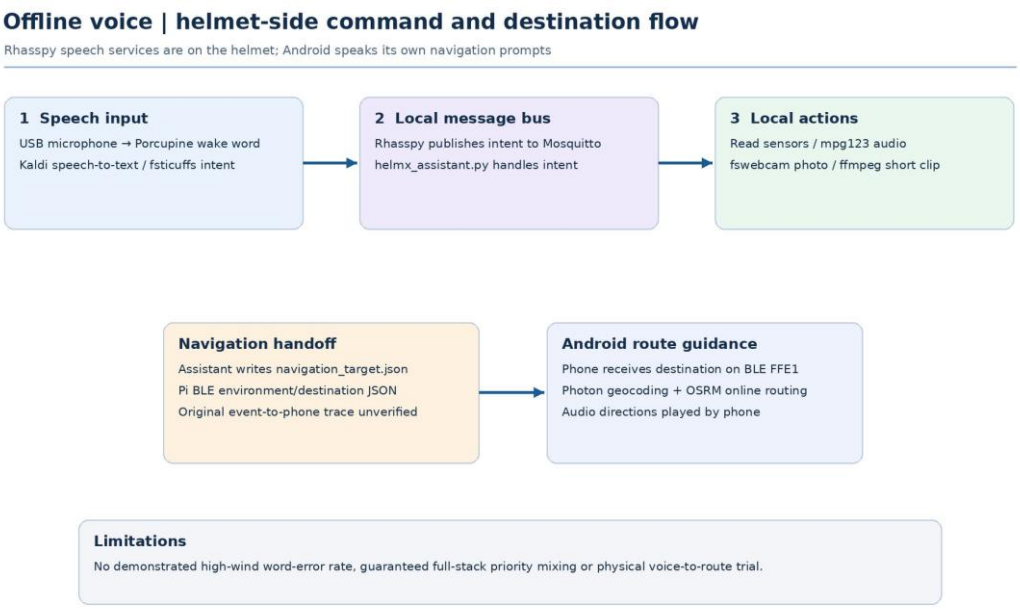



**Figure 3.14:** Local Rhasspy–MQTT assistant path and voice-selected destination hand-off (end-to-end validation outstanding).

## 3.7 Web Platform Design

The web platform (Figure 3.15) is a product website and storefront, separate from the rider-telemetry application. A shared catalog drives both the configurator interface and server-side pricing; the checkout API stores a PENDING order in PostgreSQL before creating a Stripe test-mode session; a signed webhook and the success page converge on one idempotent settlement function that changes the order to PAID once and attempts a single receipt email; and a separate contact path validates, throttles, stores and emails messages [7].

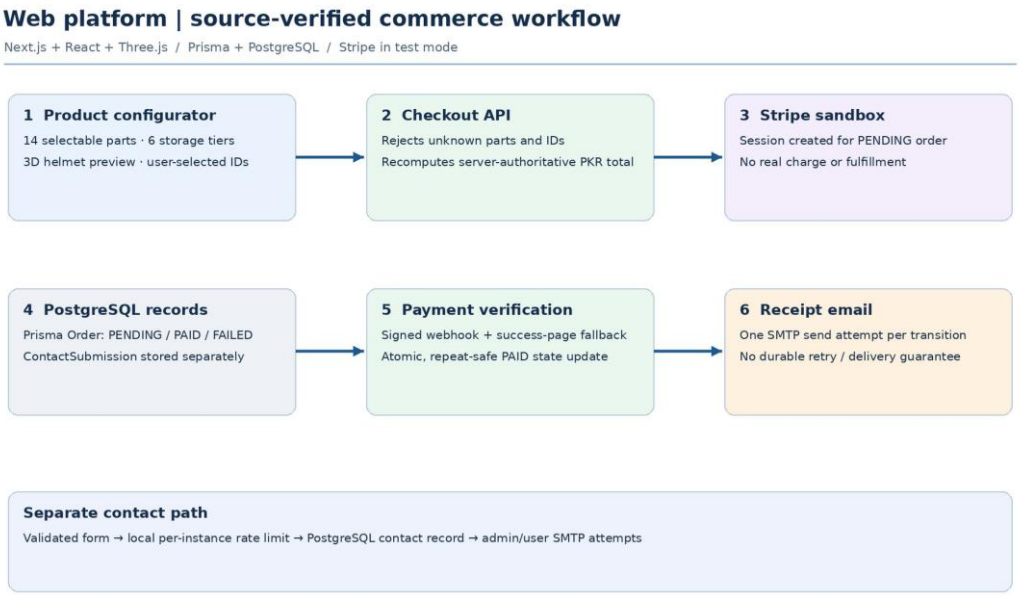



**Figure 3.15:** Web-platform architecture: server-priced configuration, sandbox checkout and idempotent settlement.

CHAPTER 4

# 4 System Testing

This chapter describes the testing strategy and reports the evidence obtained for each subsystem. Test cases follow the structure of IEEE 829 (identifier, objective, method, expected result and status) and are prioritized by safety criticality. Because HelmX combines executed research experiments with features that were planned but not built, every test case states its status explicitly.

## 4.1 Testing Strategy

Testing proceeds at four levels: unit tests of individual modules, integration tests of interfaces between modules, system-level experiments on the target hardware, and user acceptance testing. Table 4.1 defines the status values used throughout this chapter.

**Table 4.1:** Test status definitions

| Status | Meaning |
|---|---|
| Executed | Run with recorded results reported in this chapter |
| Partial | Run in software, with mocks or on an emulator; part of the intended scope not exercised |
| Implemented, not run | Test code or procedure exists in the repository but was not re-run for this report |
| Proposed | Test designed in the original plan; the feature or the test has not been carried out |

## 4.2 Test Case Design

### 4.2.1 Crash Detection

**Table 4.2:** Crash-detection test cases

| ID | Test case | Method | Status and result |
|---|---|---|---|
| CD-001 | Benign-motion false-alert characterization | Replay benign sessions and run live non-crash handling tests while monitoring scores, fused decisions and three-window confirmation | **Executed.** 0 confirmed false alerts in 0.4177 h of held-out normal data; 1 alert in 0.29 h of additional STRIDE sessions (≈3.44/h); ≈4–14 alerts/h on independent e-scooter data depending on model and policy; 6 classifier-confirmed alarms during guided donning, removal, rotation, placement and walking, unchanged (6 → 6) after the offline jerk-feature correction. No blanket false-alarm-suppression claim is made. |
| CD-002 | Held-out crash-like session evaluation | Confusion matrix, ROC-AUC, PR-AUC, F1 and event-level recall on the frozen session-level test split | **Executed.** Classifier ROC-AUC 0.946; window recall 29/143 (20.28%) at the balanced threshold; 5/5 held-out crash-like sessions detected by the replay policy (Section 4.7.2). |
| CD-003 | External validation and domain shift | Evaluate the reconstruction signal, classifier and deployed OR policy on an excluded DaRUS mechanism and on 2025 e-scooter data | **Executed.** Excluded mechanism: reconstruction ROC-AUC 0.868–0.873 vs. classifier 0.352; e-scooter: reconstruction ROC-AUC 0.826 (Section 4.7.3). |
| CD-004 | Cancellation workflow | Force a confirmed alert, cancel within 30 s, verify that no message is sent and the cooldown is observed | **Partial.** Software path and GPIO18 alarm exercised with mocked GPS/GSM; live SMS delivery not measured. |
| CD-005 | Raspberry Pi edge benchmark | Measure per-window latency, memory and stability during replay and live soaks | **Executed.** 90-minute live soak: 14,461 windows, no deadline misses or I²C failures; p99 3.25 ms (Section 4.7.4). Not a hard real-time or battery-runtime guarantee. |

### 4.2.2 Drowsiness Detection

**Table 4.3:** Drowsiness-detection test cases

| ID | Test case | Method | Status and result |
|---|---|---|---|
| DD-001 | Subject-disjoint three-class classification | UTA-RLDD development rotations D2–D5 without frame- or subject-level leakage; macro-F1, balanced accuracy, per-class recall, MCC, quadratic kappa, calibration and ROC-AUC | **Executed.** Pooled macro-F1 0.64212 across 146 videos from 48 subjects (Section 4.8.2). |
| DD-002 | Causal full-video event replay | Replay complete videos through the ONNX preprocessing and elapsed-time controller | **Executed.** Committed WARNING threshold 0.72: 2/12 drowsy-labelled fold-2 clips (16.67%) with zero observed false alerts. Validation-selected threshold 0.66: 4/12 (33.33%) with zero observed false alerts over 2.04 h of alert-labelled clips. Selection and reporting used the same clips, so the frontier is optimistic. |
| DD-003 | Reliability and missing-face handling | Blur, exposure, landmark loss, invalid geometry, 1-second UNAVAILABLE transition and timer resets | **Executed** as software contract tests; not field performance. |
| DD-004 | Export and optimization integrity | PyTorch-to-ONNX probability agreement and partial INT8 quantization gates | **Executed.** Maximum probability error $1.49 \times 10^{-7}$ with 100% argmax agreement; full convolution INT8 rejected (Section 4.8.5). |
| DD-005 | External generalization | Frozen DROZY or independently annotated helmet-camera evaluation without threshold selection on the target set | **Proposed.** Not yet completed. |

### 4.2.3 Smart Navigation

**Table 4.4:** Navigation test cases

| ID | Test case | Method | Status and result |
|---|---|---|---|
| SN-001 | Search and OSRM route parsing | NavigationLogicTest fixtures for query parsing, route decoding, instruction semantics and network-failure behaviour | **Implemented, not run** for this report (part of the 41 documented Android unit tests). External server uptime is not a unit-test result. |
| SN-002 | Turn-by-turn guidance and rerouting | NavigationGuide scenarios: projection onto the route, next maneuver, 45 m off-route threshold over two consecutive fixes, 25 m arrival proximity | **Implemented, not run** for this report. Voice intelligibility while riding not quantified. |
| SN-003 | Offline Lahore map | Bundled vector tiles, fonts, sprites and MapLibre assets load from the local WebView origin; existing guidance continues after connectivity loss | **Partial.** Unit tests and Pixel 8 emulator demonstration; new searches, routes and rerouting require internet. |
| SN-004 | Continuous GPS ride tracking | RideTrackerTest for haversine distance and jitter rules; emulator scenarios with simulated locations | **Partial.** No three-hour battery or physical-rider benchmark. |

### 4.2.4 Dual Cameras (Original Scope)

The dual-camera test cases were designed for the original continuous-recording design. The implemented helmet provides voice-triggered photos and approximately ten-second clips (Section 5.5.5), so these tests remain proposed.

**Table 4.5:** Dual-camera test cases

| ID | Test case | Method and pass criteria | Status |
|---|---|---|---|
| DC-001 | Continuous recording quality | Record a 2-hour ride from both cameras; ≥ 720p, no dropped frames, loop recording at 90% storage | **Proposed** (feature not implemented) |
| DC-002 | Voice-command image capture | Issue "capture image" 15 times at different speeds and noise levels; > 90% recognition, all images GPS-tagged | **Proposed** (photo capture implemented; test not executed) |
| DC-003 | Low-light recording | Record at dusk and night; licence plates readable at 10 m, hazards visible | **Proposed** |
| DC-004 | Video compression efficiency | Compare H.264 and H.265 on the same 1-hour session; > 40% size reduction with acceptable quality | **Proposed** |
| DC-005 | Cloud upload of crash footage | Upload the preceding 30 s after a simulated crash; start within 10 s, > 95% success | **Proposed** (feature not implemented) |

### 4.2.5 Voice Control and Entertainment

**Table 4.6:** Voice and entertainment test cases

| ID | Test case | Method | Status |
|---|---|---|---|
| VC-001 | Offline voice pipeline and intent routing | Inspect the Rhasspy 2.5.11 profile (Porcupine wake word, Kaldi ASR, fsticuffs intents, NanoTTS) and confirm that Mosquitto forwards intents to the assistant | **Partial** (implementation review); no word-error-rate benchmark under wind noise |
| VC-002 | Local assistant actions | Exercise intents for sensor readings, destination publication, music (mpg123), photos (fswebcam) and ≈10-second video (ffmpeg) | **Proposed** as a formal protocol; no success rate is claimed |
| VC-003 | Pi-to-phone destination hand-off | Assistant writes `navigation_target.json`; BLE server sends the destination and clears the file; Android starts online navigation | **Proposed**; no physical end-to-end trace recorded |
| VC-004 | Safety-path separation | Verify that helmet crash notification is local to GPIO18 and SIM800L and not routed through the Android flag | **Executed** by code review: the frozen `crash_monitor.py` has no BLE crash-flag publisher |

### 4.2.6 Environment Monitoring

**Table 4.7:** Environment-monitoring test cases

| ID | Test case | Method and pass criteria | Status |
|---|---|---|---|
| EM-001 | Temperature accuracy | Compare DHT22 readings with a calibrated thermometer at 0, 25 and 40 °C; error < 2 °C | **Proposed** |
| EM-002 | Environmental alert generation | Simulate AQI > 150 and verify an app notification within 30 s | **Proposed**; the app has no air-quality alert and MQ135 provides only a threshold signal |

### 4.2.7 Mobile and Web Applications

**Table 4.8:** Mobile and web test cases

| ID | Test case | Method | Status and result |
|---|---|---|---|
| APP-001 | BLE payload integrity | Unit-test the JSON and legacy CSV parsers, rejection of malformed payloads and clearing of state on disconnection | **Implemented, not run** (HelmetDataParserTest, 8 tests); no BLE latency claim |
| APP-002 | Navigation | Photon/OSRM parsing, guidance, rerouting and arrival code; map display and simulated GPS on a Pixel 8 emulator | **Partial**; offline map and existing-route guidance work without new network requests; new searches and rerouting do not |
| APP-003 | Ride history | Haversine distance with accuracy and jitter filters (50 m accuracy limit, 5 m movement floor, minimum 100 m / 60 s); summaries under `users/{uid}/rides` | **Partial** (RideTrackerTest and emulator); raw GPS traces are not uploaded |
| APP-004 | Crash and drowsiness phone alerts | Flag handling, notification and vibration, manual actions; the 30-second SMS/call countdown is opt-in, permission-gated and disabled for test alerts | **Partial** (SafetySettingsTest and emulator); delivery to emergency contacts not measured |
| APP-005 | Authentication and privacy | Email/password flows and per-user Firestore rules for profiles, rides and contacts | **Partial**; emulator and source review are not a third-party security audit |
| WEB-001 | Server-side pricing | Vitest catalog tests, including unknown and tampered part IDs | **Implemented, not run** (7 catalog tests) |
| WEB-002 | Receipts, validation and throttling | Vitest tests for receipt rendering, email validation and HTML escaping, and the rate limiter | **Implemented, not run** (3 receipt, 2 email and 6 rate-limit tests) |
| WEB-003 | Test-mode checkout end to end | Configure, pay with a Stripe test card, verify PAID state and receipt | **Proposed** for this report; not re-run |

### 4.2.8 Security and Authentication (Original Scope)

**Table 4.9:** Security test cases

| ID | Test case | Method and pass criteria | Status |
|---|---|---|---|
| SEC-001 | Voice PIN authentication | Speak the configured PIN with background noise; > 90% recognition, zero false accepts | **Proposed** (not implemented) |
| SEC-002 | Theft warning | Move the locked helmet without authentication; app alert within 10 s with location | **Proposed** (not implemented) |
| SEC-003 | Multi-user authorization | Add and revoke a secondary user and confirm access changes | **Proposed** (not implemented) |

## 4.3 Unit Testing

Unit tests verify individual modules in isolation before integration. The planned coverage target was above 80% for critical modules; coverage was not measured.

### 4.3.1 Hardware Components

- **IMU (original plan, superseded):** the plan specified initialization and range checks with Arduino tools. It was superseded by a register audit on the Raspberry Pi (WHO_AM_I = 0x70 in 300 of 300 reads, identifying MPU-6500-family silicon on a board labelled MPU6050) and by the live-sensor soaks in Section 4.7.
- **Camera (proposed):** capture success above 95% and encoding below 100 ms per frame under varied lighting; not executed.
- **GPS (proposed):** position error within ±5 m against surveyed reference points in open sky; not executed (NFR2.6 not measured).

### 4.3.2 Software Modules

- **Crash detection:** recorded benign and crash-like IMU sequences are passed through preprocessing, scoring, fusion and three-window confirmation, and the metrics must reproduce the frozen experiment set within tolerance with a target-device p99 inference latency of at most 10 ms per window. Tools: Python, TensorFlow/TFLite, NumPy and the Raspberry Pi benchmarking scripts released in the reproducibility package [4].
- **Drowsiness detection:** FaceLandmarker geometry, the 12 features and train-derived scaling, input shapes, timestamp causality, quality suppression, hysteresis and event replay. The verification snapshot of 14 October 2026 records 150 passing automated tests together with Ruff and strict MyPy checks [5]; this is software verification, not device safety certification.

- **Voice recognition (proposed):** 50 commands × 10 speakers (500 samples) with > 85% recognition; not executed.
- **Android application:** Table 4.10 lists the 41 local JVM unit tests documented in the repository [6]. They run with `./gradlew testDebugUnitTest`; this report did not re-run Gradle. Because the app does not use Google Maps or a NotificationListenerService, no notification-parsing tests apply.
- **Web platform:** 18 Vitest test cases in four files cover catalog pricing, email validation and HTML escaping, receipt rendering and rate limiting, alongside ESLint, TypeScript type-checking and production-build scripts [7]. No executed fifty-endpoint API report exists; the three API routes (checkout session, Stripe webhook and contact) are exercised indirectly through these library tests.

**Table 4.10:** Android local unit-test suites

| Test suite | Tests | Scope |
|---|---|---|
| NavigationLogicTest | 11 | Route geometry, search and route parsing, guidance and arrival |
| HelmetDataParserTest | 8 | BLE JSON and CSV telemetry parsing and malformed input |
| OfflineTest | 7 | Local offline map asset resolution |
| RideTrackerTest | 6 | Haversine distance, accuracy and jitter filters, minimum ride rules |
| SafetySettingsTest | 6 | Drowsiness delay and automatic-response settings |
| ValidatorsTest | 3 | Input validation |
| **Total** | **41** | |

## 4.4 Integration Testing

Integration testing validates data flow between components, from sensors through edge processing to applications.

- **Level 1, IMU to crash model:** physical helmet or bench setup with the IMU connected to the Raspberry Pi running the deployment service; controlled motions and replayed sessions are monitored from sensor read to decision. **Executed** through the live-sensor experiments and trigger campaign in Section 4.7: deployed decisions were compared with the offline replay, and timing jitter, window-buffer errors and dropped samples were measured.
- **Level 1, camera to drowsiness model:** live OpenCV capture or causal replay through MediaPipe FaceLandmarker, the calibrated ONNX model and the alert controller, monitoring quality, features, the 24-observation buffer, probabilities and

state transitions. **Partial:** implemented and assessed through contract tests and replay; on-helmet timing, power and alarm output on the Pi are not demonstrated.

- **Level 2, data storage:** verify that Android ride summaries are stored in per-user Firestore documents and that web orders and contact messages are stored in separate PostgreSQL tables (test setup: Android test account with simulated GPS and the web app with a test database; each record must be stored once under the correct user or order). **Proposed.** Live helmet sensor streaming to a unified database was not implemented.
- **Level 3, Firestore to mobile application:** the Analytics screen reads the signed-in rider's Firestore collection directly through the authenticated Firebase SDK; no custom ride-history REST API exists. The test checks that history is displayed correctly for the signed-in user and that the live BLE telemetry display works with emulator demo telemetry. **Partial** (emulator); load time not measured.
- **Level 4, end-to-end ride workflow:** full system from helmet power-on through ride completion and data review. **Proposed;** the Pi-to-phone BLE safety-flag link is not implemented.
- **Multi-user scenario (optional):** five helmets with separate users, mixed crash and drowsiness events, checking isolation and responsiveness. **Proposed.**
- **Crash detection to emergency alert:** the Raspberry Pi alert stack with the GPIO18 alarm and simulated or mocked GPS and SIM800L outcomes; trigger a confirmed event, verify that the local alert starts and can be cancelled within 30 s, allow one run to continue and verify GPS timeout and fallback and that the SIM800L send is invoked. **Partial:** `CrashAlertResult` and alarm transitions were logged and mocked success and failure paths checked; live SMS delivery is left to future field testing.

## 4.5 User Acceptance Testing (Planned)

A seven-week user acceptance study was planned but **not performed**; no SUS or other usability score is reported. The plan comprised: setup and training in week 1 (target setup time under 15 minutes); daily use in weeks 2–5 with at least ten rides per week, collecting usage logs, error reports and weekly surveys; structured scenarios in week 6 (long rides, night riding, emergency simulation and high-speed voice control); and a final evaluation in week 7 using the System Usability Scale, a feature-satisfaction survey, Net Promoter Score and semi-structured interviews. Feedback was to be triaged as critical (fixed within 48 hours), high priority (next sprint) or lower priority (future releases).

## 4.6 Success Criteria Summary

**Table 4.11:** Success criteria and outcome

| Area | Criterion | Outcome |
|---|---|---|
| Crash detection | PR-AUC, event-level and false-alert reporting; millisecond-scale Pi inference | Met: held-out ROC-AUC 0.946, PR-AUC 0.539; live p99 3.25 ms; external false alerts reported |
| Drowsiness detection | Development macro-F1, low-vigilance recall and causal sensitivity/false-alarm trade-off | Reported: macro-F1 0.642, low-vigilance recall 0.542; external validation and Pi latency open |
| Voice recognition | > 80% accuracy | Not measured |
| Navigation | Parser and routing logic tested; 95% real-road accuracy | Unit-tested in source; real-road accuracy not established |
| Battery | Three-hour target | Not measured |
| Application stability | Crash-rate target | Not measured beyond unit tests and emulator demonstrations |
| Usability | SUS ≥ 70 | Not measured (no study performed) |

## 4.7 Crash-Detection Experimental Results

The results in this section are taken from the frozen HelmX crash-detection research package, version 1.1.0 (2 November 2026) [4], and are not a field safety certification. The multi-source campaign used 66 sessions from five source groups: CARLA (7), road/STRIDE car-mounted smartphone sessions (9), near-fall (7), fall (4) and simulated DaRUS (39). Sessions were partitioned into 46 training, 10 validation and 10 test sessions, giving 18,720, 4,713 and 5,437 windows respectively; the test set comprised 5,294 normal and 143 crash-like windows (Figure 4.1). Session-level separation limits overlap leakage but does not remove all cross-domain bias.

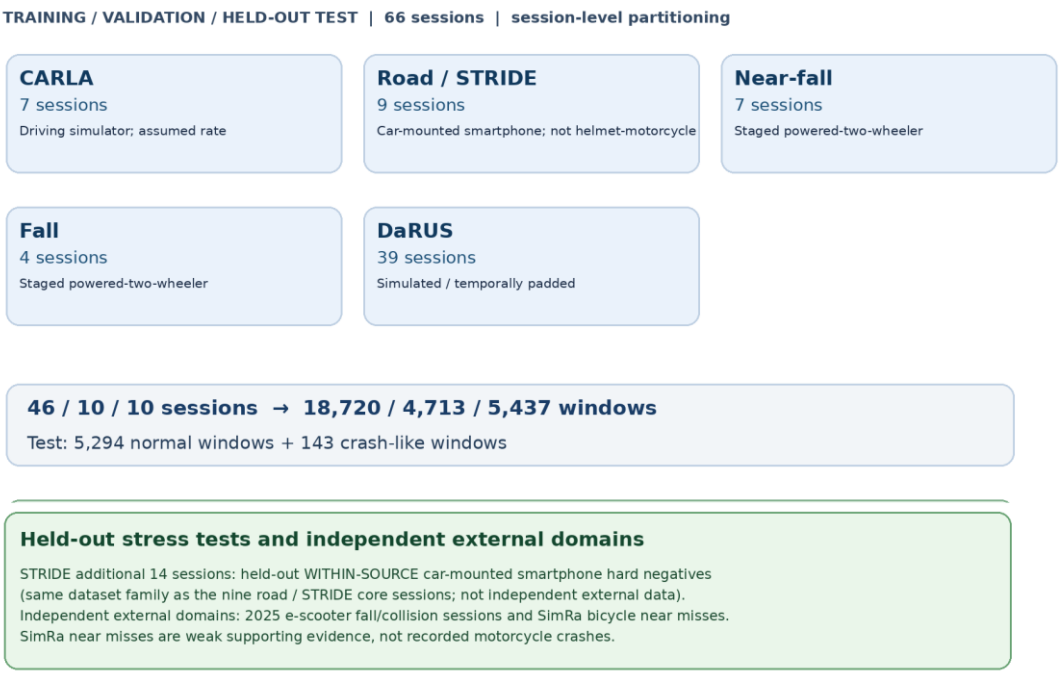



**Figure 4.1:** Data provenance, session-level split and external-validation boundaries of the crash-detection study.

### 4.7.1 Model Architecture and Calibration

The Conv1D encoder has 32, 64 and 64 filters. A normal-window reconstruction decoder and a supervised classification head share it; the head consumes pooled encoder features, one reconstruction-error scalar and seven engineered motion features. The deployed classifier and reconstruction thresholds are 0.9892255 and 10.8787359. The training objective weights classification and normal-only reconstruction in a 1:2 ratio. A 60-sample (1.2 s) window at 50 Hz advances by 15 samples (0.3 s), and the event policy requires three consecutive positive windows. Decision-making is distinct from SMS delivery: a local warning and a 30-second cancellation interval precede any GSM notification attempt.

### 4.7.2 Held-Out Model Comparison

The primary result is a negative ablation: across five seeds, the supervised-only Model B exceeded the hybrid formulation on in-distribution ROC-AUC and PR-AUC (Table 4.12, Figure 4.2). The study therefore does not support a claim that hybrid training improves in-distribution discrimination or label efficiency. The hybrid five-seed F1 was 0.317 ± 0.058; precision and recall depend strongly on the operating threshold.

**Table 4.12:** Crash-model comparison

| Model / measurement | ROC-AUC | PR-AUC | Scope |
|---|---|---|---|
| Supervised-only Model B | 0.958 ± 0.014 | 0.581 ± 0.080 | Five-seed, in-distribution |
| Hybrid classifier | 0.951 ± 0.011 | 0.527 ± 0.050 | Five-seed, in-distribution |
| Hybrid, balanced threshold | 0.9459 | 0.5394 | Single frozen test split |
| Reconstruction, held-out DaRUS | 0.868–0.873 | Not comparable | Excluded-mechanism domain shift |
| Classifier, held-out DaRUS | 0.352 | Not comparable | Excluded-mechanism domain shift |

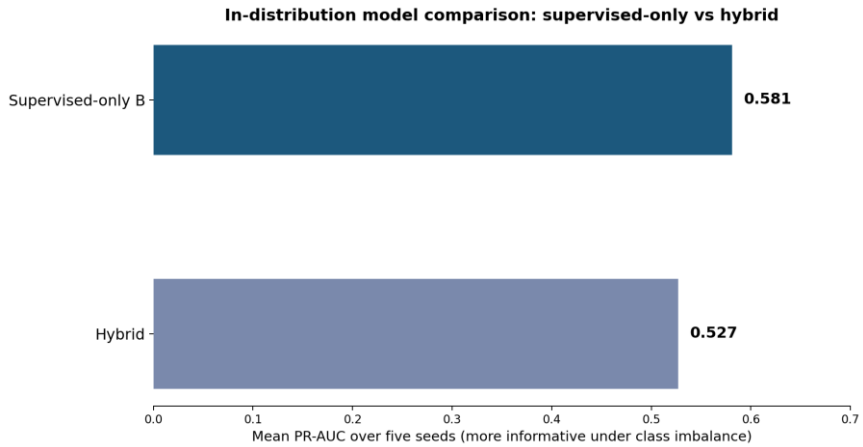



**Figure 4.2:** Five-seed in-distribution comparison of mean PR-AUC: supervised-only versus hybrid.

At the frozen balanced operating point on the 5,437-window held-out test set the classifier gave TP = 29, FN = 114, FP = 2 and TN = 5,292: ROC-AUC 0.9459, PR-AUC 0.5394, precision 93.55%, recall 20.28% and F1 0.3333. Accuracy alone (97.87%) is misleading because only 143 of 5,437 windows are positive. The three-window replay policy detected 5 of 5 held-out crash-like sessions without an event during 0.4177 h of held-out normal data; this is session-level behaviour on a small sample, not real-world crash sensitivity.

### 4.7.3 Distribution Shift and External False Alerts

In the excluded-DaRUS-mechanism experiment, classifier ROC-AUC fell to about 0.352 while reconstruction ROC-AUC was about 0.868–0.873. This contrast motivates keeping the reconstruction signal as a complement, but it is not a universal gain. On independent 2025 e-scooter data, reconstruction ROC-AUC was 0.826 and detected 8 of 9 positive sessions, whereas the two supervised configurations detected 6 of 9 and 2 of 9. The deployed OR policy also produced about 14.19 false-alert events per hour of normal riding on that shifted domain. Additional held-out STRIDE hard-negative data from the

same source family produced 1 false event in about 0.29 h (≈3.44/h). The held-out zero-alert result must therefore not be generalized across domains.

Other negative findings include the absence of consistent hybrid gains in five-seed label-scarcity experiments, no robust improvement from further pretraining or enhanced reconstruction features, and overfitting risk with stacking or hierarchical fusion. A tuned OR policy improved F1 by only about 0.0095, which is not statistically convincing. The SimRa near-miss material is weak corroborating evidence, not an independent crash benchmark.

### 4.7.4 Quantization Fidelity and Raspberry Pi 4 Validation

The frozen INT8 TFLite model is 83,568 bytes (about 81.6 KiB). At the deployed threshold, INT8 decisions agreed with the Keras reference on 99.89% of windows (FP16 agreed on 100%), and causal on-device preprocessing agreed with offline preprocessing on 99.61% of windows and 100% of events in the specified experiment. These are agreement measurements on tested inputs, not evidence that every riding condition is reproduced. Table 4.13 summarizes the target-device measurements.

**Table 4.13:** Raspberry Pi 4 crash-model evidence

| Target-device metric | Recorded value | Interpretation |
|---|---|---|
| INT8 model footprint | 83,568 bytes (≈81.6 KiB) | Exported model size |
| Isolated INT8 inference | median 0.553 ms; p99 1.682 ms | Initial benchmark |
| Long-tail inference | p99 3.94 ms | 50,000-run stress study |
| Live sensor pipeline | p99 3.19 ms / 3.25 ms | 12-minute trace / 90-minute soak |
| 90-minute live soak | 14,461 windows | No deadline misses or I²C failures observed |
| 180-minute live soak | 33,734 windows; p99 3.03 ms; max 55 °C | Separate extended soak |
| System footprint (90-min soak) | CPU ≈2.35%; RSS ≈50 MB; max 53 °C | Measured in the specified soak |
| Battery runtime / SMS delivery | Not formally measured | No runtime or delivery guarantee |

A separate 30-minute thermal run reported no throttling. These are workload-specific measurements, not a hard real-time bound, a system-wide uptime guarantee or a battery-endurance measurement. The frozen trigger campaign detected 35 of 36 replay sessions, and live trigger injection exercised the physical GPIO18 alarm. Under CPU and background load, Linux timing gaps invalidated about 6.9% of windows in one stress case. End-to-end GPS/GSM SMS delivery was not measured; software-level success and failure handling and the missing-GPS fallback were tested instead.

### 4.7.5 Deployment Fidelity

Deployment fidelity must be distinguished from classifier accuracy. In the composed-path test, deployed decisions agreed with the training-time path on 99.65% of all windows but only 90.21% of crash-like windows; this class-stratified result exposes minority-class degradation hidden by the high aggregate agreement. Evaluated separately at the deployed threshold, the causal-preprocessing-only comparison (L2) agreed on 99.87% of all windows and 96.50% of crash-like windows, and the INT8-only conversion (L3) agreed on 99.89% overall and 95.80% of crash-like windows. These stage-specific figures are not interchangeable with the composed 99.65%/90.21% result, and none of them is a crash-detection accuracy.

### 4.7.6 Acquisition Integrity and Scheduling

The 90-minute live-sensor experiment completed 267,891 error-free IMU reads, but sample-stream validity was 93.07% and usable decision coverage 80.99% under the reported validity definitions: zero I²C read failures does not imply an unbroken valid decision stream. The 180-minute soak extended coverage to 539,981 samples and 33,734 scored windows with p99 processing latency 3.03 ms and a maximum device temperature of 55 °C; this value belongs to the separate 180-minute soak and must not be substituted for the 90-minute or isolated-inference figures. Register-verified polling-versus-FIFO tests established acquisition correspondence in the measured configurations, and the corrected polling–FIFO–polling A–B–A test produced zero alerts in all three arms. SCHED_FIFO real-time scheduling did not improve all timings and conditions, and Linux scheduling remained a source of occasional invalidated windows. No hard real-time determinism, all-ride availability or battery endurance is claimed.

### 4.7.7 Session-Clustered Baselines, Jerk Isolation and Handling Risk

Paired F1 differences (baseline minus HelmX supervised classifier) with 95% confidence intervals from 10,000 session-clustered bootstrap resamples were: logistic regression −0.181 [−0.597, −0.035]; acceleration-peak threshold −0.227 [−0.770, −0.052]; gyroscope-peak threshold −0.283 [−0.944, +0.342]. The gyroscope interval crosses zero, and these comparisons do not establish hybrid superiority.

A documented train–deploy mismatch in the engineered jerk feature was isolated without altering the frozen artefacts: the alternative jerk implementation changed 1 of 5,437 classifier decisions in the held-out replay and did not remove the six classifier-confirmed

alarms in the guided handling battery (6 → 6). These handling alarms are operationally important because a rider may not always be able to cancel a false alert promptly.

### 4.7.8 Ethics, Scope and Validity

The guided helmet-motion and Raspberry Pi experiments were conducted by the author using controlled non-crash activities; no formal institutional ethics approval is claimed. The third-party training data were not collected by the project team, and their source and label limitations are retained. Controlled fall and near-fall sessions use weak top-30%-peak-acceleration window labels; DaRUS is simulated and temporally padded to the 60-sample window; and no natural, unstaged motorcycle crashes are included. The study contributes a controlled comparison of supervised classification and normal-only reconstruction, a concrete distribution-shift stress case and reproducible Raspberry Pi execution evidence. It does not demonstrate fewer injuries, universal crash recall, zero external false alarms or certified emergency delivery. The code and evidence are archived in the reproducibility release [4]; the device is a research prototype and has not been safety-certified.

## 4.8 Drowsiness-Detection Experimental Results

### 4.8.1 Study Design, Separation and Labels

The retained quality-gated fusion model was selected using UTA-RLDD development rotations D2–D5. The corpus used locally comprised 60 subjects and 182 videos; pooled out-of-fold results cover 146 videos from 48 subjects. UTA-RLDD provides one label per complete video (alert, low vigilance or drowsy), not a physiological onset time. Subject and session leakage checks, training-only physiology statistics, validation-based selection and temperature calibration are part of the evidence contract [5], [32].

### 4.8.2 Classification Performance

The selected categorical cross-entropy model achieved the pooled development results in Table 4.14. These are development results, not a claim of external or motorcycle-camera generalization.

**Table 4.14:** Drowsiness-model development metrics (pooled D2–D5)

| Metric | Value |
|---|---|
| Macro-F1 | 0.64212 |
| Balanced accuracy | 0.64389 |
| Matthews correlation coefficient | 0.46782 |
| Quadratic weighted kappa | 0.58655 |
| Macro one-vs-rest ROC-AUC | 0.77917 |
| Expected calibration error | 0.10533 |
| Recall: alert / low vigilance / drowsy | 0.75000 / 0.54167 / 0.64000 |

### 4.8.3 Elapsed-Time Event Evaluation

Full-video causal replay through the shared ONNX preprocessing and elapsed-time controller detected 2 of 12 (16.67%) drowsy-labelled clips at the committed WARNING threshold of 0.72, with zero observed false alerts. A separately tuned WARNING threshold of 0.66 detected 4 of 12 (33.33%) with zero observed false alerts over 2.04 h of alert-labelled video. The broader frontier (Figure 4.3) gives 7 of 12 at 0.490 false alerts per hour, 8 of 12 at 2.451 per hour and 12 of 12 at 3.922 per hour. The D2 checkpoint and event thresholds were evaluated and selected on the same fold-2 development clips (36 videos, 12 drowsy), so these operating points are selection-influenced and optimistic, not independent estimates. Because UTA labels cover entire clips, the appropriate timing measure is time-to-first-alert within a drowsy-labelled clip, not drowsiness-onset latency.




**Figure 4.3:** Full-video causal replay on fold-2 development clips.
*The committed WARNING threshold 0.72 detected 2/12 clips; the tuned threshold 0.66 detected 4/12 with zero observed false alerts over 2.04 h. Tuned settings were selected and measured on the same clips, so they are not independent test estimates.*

### 4.8.4 Ablations and Rejected Alternatives

Fourteen alternatives were tested, including visual-only models, alternative fusion, multiple-instance learning (MIL), ordinal and hybrid objectives, backbone freezing, balanced sampling and temporal label confidence; none met all frozen advancement gates. One earlier MIL variant scored 0.51736 video macro-F1 on its selected validation fold but only 0.33232 on a preregistered separate confirmation fold, compared with the retained model's 0.56481 under identical aggregation. Screening used seed 13; the intended five-seed confirmation was not carried out. UTA fold 1 was consumed by the earlier MIL check and excluded from later screening.

### 4.8.5 Software Verification, Export and Deployment Status

The 14 October 2026 verification snapshot reports 150 automated tests, Ruff checks and strict MyPy type checking. In a selected pilot bundle, the maximum PyTorch-versus-ONNX probability error was $1.49 \times 10^{-7}$ across 12 held-out windows, with 100% argmax agreement. A partial operator-level INT8 experiment preserved argmax decisions on a 100-sample parity set; full convolution INT8 was rejected. Historical workstation model-only latency was 37.905 ms median and 43.168 ms p95 with ONNX Runtime on CPU, and 16.185 ms median and 21.320 ms p95 with PyTorch on CUDA. These figures exclude video capture, MediaPipe, preprocessing, display and alarms and are not Raspberry Pi measurements; the documented Pi service inventory does not include a drowsiness service.

### 4.8.6 Limitations

The model has no independent DROZY confirmation, no prospective helmet-camera rider dataset, no complete Raspberry Pi camera-to-audio benchmark, no target-device power or long-duration soak record and no certification. Low vigilance is its weakest class (recall 0.54167). The committed threshold detected 2 of 12 drowsy-labelled clips; the tuned 0.66 threshold detected 4 of 12 on the same clips and is therefore optimistic. Video-level labels cannot establish physiological fatigue onset. Future data collection must use informed consent, appropriate ethics review and stationary or simulated conditions.

## 4.9 Android, Web and Raspberry Pi Evidence

### 4.9.1 Android Application

The evidence refers to the `android-app` branch at commit bce53f7, built with Kotlin 2.1.10 and Android Gradle Plugin 8.13.1 for minimum SDK 24 and target SDK 36 [6].

- **Data boundary:** Firebase Authentication and Cloud Firestore (`users/{uid}`, `users/{uid}/rides`) hold accounts and ride summaries. The branch has no dependency on Firebase Realtime Database or Firebase Cloud Messaging and no custom ride-history REST API.
- **Navigation boundary:** MapLibre GL JS in a WebView with OpenFreeMap/OpenStreetMap-derived tiles, Photon place search and OSRM driving routes. The Lahore District tiles allow offline map display, recent destinations and continuity of an already loaded route, but not offline route creation, offline search or live-traffic routing.
- **Safety boundary:** HelmetBleManager receives one-way JSON/CSV notifications that can include Boolean crash and drowsiness flags. SafetyMonitor adds notifications, alarm and vibration, an opt-in 30-second automatic SMS/call attempt and non-sending test alerts. The phone's drowsiness sensitivity (0, 1.5 or 3 s alarm delay) affects only the phone alarm, not the helmet model thresholds, where the committed 0.72 and the tuned 0.66 have different event sensitivities.
- **Evaluation boundary:** 41 local JVM unit tests and manual Pixel 8 (Android 14) emulator scenarios with debug-only simulated helmet readings and simulated Lahore GPS. The demo telemetry never reports crash or drowsiness. This does not establish a physical Pi-to-phone BLE link, real SMS or call delivery, broad device compatibility, UI response times, SUS scores or multi-user load results.

The crash response has two independent possible paths: the Pi's local SIM800L GSM attempt and the optional Android 30-second automatic response. Their coordination has not been verified, and the frozen `crash_monitor.py` v1.1.0 does not publish BLE crash flags.

### 4.9.2 Web Platform

Table 4.15 summarizes the source evidence for the web platform at commit 3b4be8e [7]. The presence of test files and scripts is source-level evidence; this report did not re-run them or complete a payment. Stripe is configured in test mode, and receipt delivery and multi-instance rate limiting have not been independently demonstrated.

**Table 4.15:** Web-platform source evidence and validation boundaries

| Component | Source evidence | Validation boundary |
|---|---|---|
| Catalog and 3D configurator | 14 parts in three groups and six storage tiers; Three.js / React Three Fiber | The catalog is not the prototype's bill of materials |
| Checkout API | Server-side pricing; unknown IDs rejected; duplicate IDs counted once | Stripe test mode only; purchase not re-run |
| Orders and webhook | Prisma schema; signed Stripe webhook; atomic PENDING/FAILED → PAID transition | One SMTP receipt attempt; no delivery guarantee |
| Contact endpoint | Field validation, HTML escaping, per-IP fixed-window throttling, database record | Rate limiter is per instance, not distributed |
| Quality controls | 18 Vitest test cases; lint, type-check and build scripts; security headers | Test code inspected; suite not re-run |

### 4.9.3 Raspberry Pi Configuration

The main-branch hardware reference documents an October 2026 snapshot of a Raspberry Pi 4 Model B (8 GB) running Debian 12 arm64 from a 128 GB SSD, with its sensor buses, peripherals and systemd services [8]. Device nodes and service states are documented snapshots, not a fresh hardware audit for this report. The drowsiness camera-to-alert benchmark and the crash-model BLE event publisher remain unverified and unimplemented respectively.

CHAPTER 5

# 5 Implementation

This chapter documents how HelmX was implemented: the work breakdown, team responsibilities and schedule, the tools and hardware used, the implementation of each subsystem, screenshots of the prototype and applications, and the challenges encountered.

## 5.1 Work Breakdown Structure

The work breakdown structure decomposes the project into six work packages. Items marked *(proposed)* were planned but not implemented.

**WBS 1 — Documentation**

- 1.1 Project proposal, literature review and requirements specification
- 1.2 System architecture documentation and implementation progress reports
- 1.3 Final report, FYP poster and research reproducibility package

**WBS 2 — Machine-learning models**

- 2.1 Crash detection: dataset collection and harmonization; hybrid supervised–reconstruction architecture and training strategy; training and validation; INT8 TFLite export and on-device benchmarking; session-split evaluation, external robustness analysis and Raspberry Pi 4 deployment validation
- 2.2 Drowsiness detection: UTA-RLDD (primary subject-disjoint driver-state data) and MRL Eye (auxiliary eye pretraining) integration; quality-gated MobileNetV3-Small/physiology causal TCN in PyTorch; OpenCV and MediaPipe video pipeline with EAR/MAR features; training and validation; ONNX export and optimization
- 2.3 Offline helmet assistant: Rhasspy with Porcupine wake word and Kaldi recognition; Mosquitto MQTT intent handling and NanoTTS playback; UART5 hardware command recognition; commands for navigation, status, music, photo and short video; future validation of wind noise, audio concurrency and BLE hand-off

**WBS 3 — Web application**

- 3.1 Frontend: React component architecture, Next.js routing, TypeScript types, custom UI components, responsive layout, landing-page sections, 3D GLB model preview with product customization, checkout, payment-result and contact pages
- 3.2 Backend: Next.js API routes for server-priced checkout, signed Stripe webhook and contact form; Prisma/PostgreSQL Order and ContactSubmission schema; order idempotency, validated form fields, per-instance rate limiter and SMTP

**WBS 4 — Mobile application**

- 4.1 Core features: Kotlin project setup; registration, login and authentication; Firebase Authentication and Cloud Firestore integration; Firestore security rules and per-user schema; profile management
- 4.2 Navigation: MapLibre WebView with OpenStreetMap-derived tiles; Photon search; OSRM routing and turn-by-turn geometry; Android voice search and spoken guidance; off-route detection and rerouting without live traffic; bundled offline Lahore District map; navigation UI
- 4.3 Safety and tracking: phone-side SafetyMonitor for crash and drowsiness flags; opt-in automatic SMS/call; foreground GPS ride tracking and analytics

**WBS 5 — Hardware**

- 5.1 Procurement and setup: helmet; camera modules; sensors with I²C, UART and GPIO wiring; Raspberry Pi 4 with Debian 12 on SSD
- 5.2 Cameras: IR CSI camera and USB camera integration; voice-triggered photo and short-video capture; continuous dual-camera recording *(proposed)*
- 5.3 IMU and sensors: IMU configuration and register audit; sensor calibration; data acquisition; real-time processing for the crash monitor; DHT22 and MQ135 environmental sensing
- 5.4 Theft detection *(proposed)*: voice-PIN authentication concept; unauthorized-motion alert; mobile theft notification

**WBS 6 — Integration and evaluation**

- 6.1 Pi services (BLE server, assistant, crash monitor) and Android BLE client
- 6.2 Crash-model target-device experiments (soaks, fidelity, timing validity)
- 6.3 Drowsiness causal replay and software verification

## 5.2 Team Roles and Responsibilities

Table 5.1 lists the responsibilities of each team member. Contribution attribution follows the team-verified FYP responsibilities and the individual crash and drowsiness research releases. Some broad team-credit summaries in earlier repository README files describe historical collaboration and should not be interpreted as authorship of the individual research publications.

**Table 5.1:** Team roles and responsibilities

| Team member | Primary activities | Secondary activities |
|---|---|---|
| Usman Azfar (group leader) | Mobile application development (Kotlin); Firebase integration and database setup; user authentication; drowsiness-detection model development; lead author and developer of the hybrid Conv1D crash-detection research model, experiments and reproducibility artefacts | Hardware integration planning; system testing coordination; crash-data acquisition, deployment-fidelity analysis and target-device benchmark coordination |
| Hamza Ahmad | Web application development and backend integration; React/Next.js implementation; custom UI components; IMU data processing; drowsiness-detection development | Web services and database implementation; backend integration and general testing collaboration |
| Abdul Hanan | Crash-detection development; navigation and documentation contributions; hardware integration and project support | System testing; alert integration support; prototype assistance |
| Awais Imtiaz | Offline voice-assistant development (voice-control agent); documentation (proposal and reports); mobile application development; UI/UX design implementation; theft-detection and dual-camera design; literature review compilation | API integration and testing; prototype deployment planning; hardware requirements documentation |

## 5.3 Development Timeline

Figure 5.1 shows the functional workstreams across the nine project months. The chart summarizes planned and completed phases; it does not assign individual authorship or establish exact completion dates.

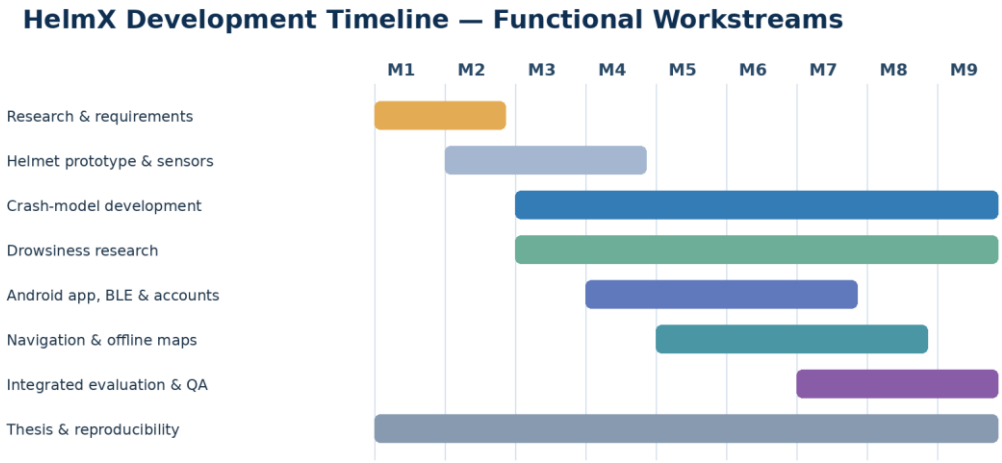



**Figure 5.1:** Retrospective development timeline by functional workstream.

## 5.4 Tools and Technologies

### 5.4.1 Machine-Learning Development

**Crash detection:** Python with TensorFlow/Keras for offline training and TensorFlow Lite INT8 for deployment; scikit-learn for preprocessing, calibration utilities and metrics; pandas for data handling; custom modules for resampling, normalization, feature extraction and deployment synchronization; Jupyter notebooks for controlled experiments, VS Code for production code and Raspberry Pi scripts for target-device execution.

**Drowsiness detection:** PyTorch MobileNetV3-Small visual encoder, twelve-feature physiological projection, learned reliability-gated fusion and four-block causal TCN with a three-class categorical cross-entropy objective. Inputs are 24 RGB crops at 160 × 160, twelve time-dependent physiology features and two quality signals at an 8 Hz target rate on subject-disjoint UTA-RLDD folds. Optimization uses AdamW with learning rates of $3 \times 10^{-4}$ for new layers and $3 \times 10^{-5}$ for the pretrained backbone, a 40-epoch cap, early stopping and validation-based temperature calibration. The runtime uses MediaPipe FaceLandmarker, OpenCV capture, ONNX Runtime on CPU with validation-gated partial quantization and the elapsed-time alert controller.

### 5.4.2 Web Application Development

The website uses the Next.js 16 App Router with React 19, TypeScript 5 and Tailwind CSS 4, with Radix/shadcn UI components. The 3D configurator uses React Three Fiber, Three.js, Drei and a GLB helmet model; a single typed catalog (`lib/catalog.ts`) defines

14 selectable parts grouped as core, sensors and interaction, and six storage options, priced in PKR. The backend uses Next.js route handlers, Prisma 7 with PostgreSQL (Order and ContactSubmission models), Stripe Checkout in test mode and Nodemailer over SMTP. Quality controls are Vitest, ESLint and TypeScript checking. The site is deployed on Vercel (https://helmx-silk.vercel.app) as a sandbox demonstration without a fulfilment process [7].

### 5.4.3 Mobile Application Development

- **Language and UI:** Kotlin 2.1, Android Views, XML layouts, ViewBinding, Material components, AndroidX lifecycle, coroutines and StateFlow.
- **Build:** Android Studio, Gradle Kotlin DSL, Android Gradle Plugin 8.13, compile and target SDK 36, minimum SDK 24 (Android 7.0).
- **Helmet communication:** BLE GATT client for the peripheral named `HelmX`, service FFE0 and notification characteristic FFE1, MTU 512, JSON and legacy CSV telemetry [56].
- **Identity and persistence:** Firebase Authentication and Cloud Firestore (`users/{uid}`, `users/{uid}/rides`, emergency contacts); SharedPreferences for local settings and recent places [57].
- **Maps:** MapLibre GL JS in a WebView, OpenFreeMap/OpenStreetMap vector tiles, Photon geocoding, OSRM routing, Fused Location Provider and offline Lahore tiles [38], [39].
- **Background services and safety:** foreground RideTrackingService, SafetyMonitor, high-priority notifications, vibration and alarm, opt-in SMS and phone permissions.

### 5.4.4 Hardware Components

Table 5.2 lists the hardware fitted to the prototype as documented in the October 2026 inventory [8].

**Table 5.2:** Prototype hardware and interfaces

| Component | Qty | Interface | Role |
|---|---|---|---|
| Raspberry Pi 4 Model B (Rev 1.5, 8 GB) | 1 | — | Edge computer; Debian 12 arm64, fixed 1.8 GHz |
| SanDisk X400 SSD, 128 GB | 1 | USB 3 SATA bridge | Root filesystem |
| 6-axis IMU board labelled MPU6050 (MPU-6500-family silicon, WHO_AM_I = 0x70) | 1 | I²C1, address 0x68 | Accelerometer and gyroscope for crash detection |
| Raspberry Pi IR night-vision camera with two IR illuminators | 1 | CSI | Low-light camera |

| Component | Qty | Interface | Role |
|---|---|---|---|
| USB camera with built-in microphone | 1 | USB (`/dev/video0`) | Voice input and media capture |
| DHT22 temperature and humidity sensor | 1 | GPIO BCM22 | Environmental sensing |
| MQ135 air-quality sensor (digital threshold) | 1 | GPIO BCM17 | Qualitative air quality |
| SIM800L GSM modem | 1 | UART0 (`/dev/serial0`), 9600 baud | Emergency SMS |
| NEO-6M GPS receiver | 1 | UART3 (`/dev/ttyAMA3`), 9600 baud | Location for the emergency SMS |
| Voice-recognition module | 1 | UART5 (`/dev/ttyAMA5`), 9600 baud | Hardware voice commands |
| Speakers | 2 | Audio | Voice responses, alerts and music |
| Lithium power bank, 10,000 mAh | 2 | USB | Portable power |
| Crash alarm (buzzer/indicator) | 1 | GPIO BCM18 | Local alert during the cancellation window |
| Super Air SP 912 helmet | 1 | — | Base helmet (DOT approval reported for the unmodified helmet; the modified prototype is not certified) |

### 5.4.5 Development and Collaboration Tools

Git and GitHub for version control and repository hosting; Jira for task tracking and sprint management; Microsoft Word for documentation; Figma for UI/UX design; draw.io for diagrams; WhatsApp for team coordination.

## 5.5 Implementation Details

### 5.5.1 Crash Detection Model

**Design approach:** the crash detector uses a hybrid learning strategy rather than a single unsupervised autoencoder. It learns a discriminative crash-classification signal while exposing reconstruction error as a complementary anomaly signal, acknowledging that crash labels are limited without assuming that anomaly detection alone is sufficient.

**Data and windowing:** six-axis IMU sequences are harmonized and converted into 1.2-second windows represented as 60 × 6 tensors at an effective 50 Hz, with a small set of engineered motion features that complement the learned representation. Publicly available third-party multi-source datasets were split at session level to reduce leakage.

**Model and decision policy:** a shared Conv1D encoder feeds a supervised classifier and a reconstruction decoder (Figure 3.5). A calibrated OR-fusion policy flags a candidate

when either signal crosses its threshold, and three consecutive positive windows are required before the rider is warned and given 30 seconds to cancel. Only uncancelled alerts proceed to GPS capture and the SMS attempt. This operational logic is essential to false-alert control and is part of the detection system.

**Deployment:** the model runs on the Raspberry Pi 4 as an 83,568-byte INT8 TFLite model inside `helmx-crash-monitor.service`, configured by `deploy/config.yaml`: 50 Hz sampling, 60-sample windows with a 15-sample stride, gravity removal with a 0.3 Hz high-pass filter, a timing guard that accepts sample gaps of 10–60 ms and resets the stream after gaps longer than 1.0 s, a 60-second cooldown, the BCM18 alarm with the `/tmp/helmx_cancel_alert` cancellation signal, a 15-second GPS timeout and rotating logs [8]. The measured performance is reported in Section 4.7.

**Status:** the hybrid model supersedes the earlier autoencoder-only draft. Its limitations remain: real motorcycle crashes are scarce, external-domain false alerts can rise substantially, and the GPS/GSM path must not be described beyond what was directly validated.

### 5.5.2 Drowsiness Detection Model

**Input contract:** the model replaces the earlier eye-state CNN/threshold prototype with a personalized, quality-aware, causal multimodal model taking [B,T,3,H,W] face imagery, [B,T,12] physiology and [B,T,2] reliability, normally with T = 24 at 8 Hz and H = 160. ImageNet normalization is applied to padded RGB crops. Both branches depend on MediaPipe FaceLandmarker geometry but are otherwise distinct.

**Personalization and features:** a brief high-quality neutral calibration (five-second target, at least 15 quality observations) fits robust median EAR, MAR and head-pose baselines for each rider. A 60-second causal history produces relative EAR, inter-eye asymmetry, PERCLOS, blink rate, mean blink duration, current closure duration, relative MAR, open-mouth fraction, relative pitch, yaw and roll, and valid-observation fraction. Training-set robust centre and scale are exported, and standardized features are clipped to ±10.

**Fusion and temporal inference:** a MobileNetV3-Small encoder and a physiological projection generate 192-dimensional representations; a learned reliability gate uses the two quality channels to weight them; four depthwise-separable causal residual TCN blocks (dilations 1, 2, 4, 8) produce the temporal representation; and temperature-calibrated logits give the three class probabilities.

**Training:** the reported model is selected with categorical cross-entropy, not a multitask objective; eye, yawn, head-pose and quality heads exist but their loss weights are zero in the retained configuration. MRL Eye is used only for auxiliary eye-state pretraining. Development uses UTA-RLDD subject-disjoint folds with validation-only model selection.

**Alert policy:** the network outputs probabilities but does not trigger audio itself. The committed controller settings are CAUTION ($P \geq 0.55$ for 1.5 s), WARNING ($P \geq 0.72$ for 2.0 s), CRITICAL ($P \geq 0.88$ for 3.5 s), release ($P \leq 0.40$ for 2.0 s), UNAVAILABLE after 1 s of poor-quality evidence and a 3-second audible cooldown. The validation-selected WARNING threshold of 0.66 produced the 4/12 tuned event result but is not the committed default. The phone companion's drowsiness delay does not change these helmet-side thresholds. Workstation ONNX inference is implemented and measured; a Raspberry Pi camera-to-audio benchmark, independent DROZY confirmation and motorcycle-domain testing remain pending.

### 5.5.3 Web Application

**Architecture:** the public HelmX website is a responsive Next.js 16 / React 19 / TypeScript application that presents the helmet, its features and technology, and an independently developed 3D configurator. It is not a live ride-telemetry dashboard and does not replace the Android app [7].

**3D configuration:** a React Three Fiber / Three.js scene supports interactive rotation; visitors choose among 14 catalogued parts in three groups and one of six storage tiers, and the page shows a quote from the shared catalog. The catalog is a commercial mock-up, not an audited bill of materials of the tested prototype.

**Trusted pricing and orders:** the checkout route accepts the customer email and selected part and storage IDs, validates them, removes duplicates and recomputes the PKR amount on the server. It stores a PENDING order through Prisma/PostgreSQL and then creates a Stripe Checkout test-mode session.

**Idempotent settlement and receipt:** signed Stripe webhook events and the post-checkout confirmation route both call `finalizePaidSession`. A conditional database update prevents a repeated transition to PAID, and only the winning transition attempts one itemized SMTP receipt. Because sending is not queued, an SMTP failure after the PAID transition can leave a receipt undelivered. Expired sessions and failed

asynchronous payments mark a pending order FAILED; a PAID order is never downgraded.

**Contact and security:** the contact API validates required fields, email format and length, applies per-IP fixed-window throttling, escapes input for HTML email, stores a ContactSubmission and attempts administrator and sender emails. The limiter is in-memory and per instance. Security headers (including `nosniff` and frame and referrer policies) are sent on every response, the developer test-email route is disabled in production, and the site generates `robots.txt`, `sitemap.xml` and link-preview metadata.

### 5.5.4 Mobile Application

**Architecture:** the HelmX companion is native Kotlin with Android Views and ViewBinding. Its activities are Login, Sign-up, Dashboard, Analytics, Navigation, Crash Alerts, Crash Action, Drowsiness, Voice & Entertainment, Settings, Account, Privacy and About, supported by service and helper classes. HelmetBleManager stores link state and parsed telemetry in a StateFlow that SafetyMonitor observes app-wide [6].

**Authentication and persistence:** Firebase Authentication handles email/password accounts, sign-in, profile updates and account deletion. Cloud Firestore stores a user document, cached emergency contacts and ride summaries under `users/{uid}/rides`; security rules restrict reads and writes to the authenticated user. The app does not use Firebase Realtime Database or Firebase Cloud Messaging.

**Bluetooth protocol:** the phone scans for the `HelmX` peripheral, connects as a GATT client, requests MTU 512, verifies service FFE0 and characteristic FFE1 and subscribes to notifications. It parses JSON for environmental readings and destination requests, and CSV for battery, speed, drowsy, crash and distance (or legacy temperature and humidity). Connection states include reconnecting, not found and service mismatch, and stale telemetry is cleared after disconnection. The link is receive-only.

**Crash and drowsiness handling:** the phone receives Boolean safety flags, not the crash scores or the drowsiness model's probabilities and controller states. For drowsiness it raises sound, vibration and a notification after a configurable delay (high 0 s, medium 1.5 s, low 3 s). For crashes it offers *I am OK*, *Call* and *Text my location*. The optional *Always allow* mode requires opt-in, emergency contacts and SMS/CALL_PHONE permissions and, after a 30-second cancellation window, attempts an SMS and a call. This path is

separate from the Pi SIM800L path, and neither physical delivery nor coordination of the two paths has been verified.

**Navigation:** NavigationActivity embeds MapLibre GL JS in a WebView. MapsApi requests Photon suggestions and OSRM driving routes while online; NavigationGuide computes route projection, remaining distance, rerouting and arrival. OfflineMapServer serves the packaged Lahore District vector map, fonts, sprites and styles from the virtual `helmx.local` origin; online maps switch to the offline package without internet, and guidance on a route already started can continue.

**Ride recording and analytics:** RideTrackingService runs as a foreground location service. RideTracker computes haversine distance after discarding fixes less accurate than 50 m and movements under 5 m; rides shorter than 100 m or 60 seconds are not saved. Firestore queues ride summaries (distance, duration, average speed and destination); the raw GPS track is not uploaded. Recent rides and daily totals appear in AnalyticsActivity.

**Verification status:** 41 local unit tests and manual Pixel 8 emulator scenarios with simulated Lahore GPS. A debug-only simulated helmet supplies non-safety demo values for screenshots and UI checks and never generates crash or drowsiness flags. This validates the app logic but not a physical Raspberry Pi BLE link, real SMS or call delivery, or the helmet's road-safety effectiveness.

### 5.5.5 Cameras and Media

**Hardware:** a Raspberry Pi infrared night-vision camera with two IR illuminators is connected through CSI, and a USB webcam with a microphone enumerates as `/dev/video0` [8].

**Implemented functions:** the local assistant supports voice-triggered photo capture with fswebcam and approximately ten-second video clips with ffmpeg, stored in a local vault on the Pi. The infrared camera provides a device for low-light acquisition; its low-light performance has not been quantified.

**Not established:** continuous synchronized front and rear recording, automatic crash-triggered cloud upload, live streaming, GPS timestamp overlays and validated retention policies were part of the original design (FR4.1, FR4.3–FR4.5) but are not implemented.

### 5.5.6 IMU and Environmental Sensors

**IMU identity:** the physical board is labelled MPU6050, but a register audit returned WHO_AM_I = 0x70 in 300 of 300 reads, identifying MPU-6500-family silicon. References to the MPU6050 in this report therefore denote the board label, not a verified chip identity. This matters for acquisition-register settings.

**Acquisition pipeline:** the six-axis stream is sampled for the 50 Hz, 60 × 6 model input; raw samples pass through unit harmonization, causal filtering, sliding-window segmentation and engineered-feature extraction to reproduce exactly the representation expected by the frozen model. The deployment monitors timing gaps, maintains the rolling buffer and feeds the three-window confirmation policy; these safeguards matter as much as the model because benign sensor artefacts can otherwise cause spurious detections.

**Environmental sensors:** `bluetooth.py` reads the DHT22 (BCM22) and the MQ135 digital threshold output (BCM17) about every 2.5 seconds. The BLE payload contains temperature, humidity, a qualitative air-quality value and, optionally, a voice-selected destination, and `sensor_data.json` is updated about every 10 seconds. The MQ135 output must not be reported as calibrated particulate concentration.

**Remaining work:** automate calibration, quantify sensor drift over long rides and complete field validation of the full emergency workflow.

### 5.5.7 Offline Voice Assistant

**Topology:** the deployed configuration uses Rhasspy 2.5.11 on the Raspberry Pi rather than a cloud language model. The USB camera microphone provides audio; Porcupine detects the wake word, Kaldi performs speech recognition, fsticuffs classifies intents and NanoTTS produces speech; Mosquitto MQTT passes intents to `helmx_assistant.py` [8].

**Hardware command path:** a dedicated voice-recognition module on UART5 (BCM12/13, `/dev/ttyAMA5`) supplements the software stack. Equivalent accuracy or latency of the two paths at riding speeds has not been established.

**Command handlers:** the assistant reports environmental readings, accepts navigation requests, plays local music with mpg123, takes photos with fswebcam and records approximately 10-second videos with ffmpeg. Media remains in the vault under

`/home/helmx/helmx_vault`; cloud upload and conversational dialogue are not implemented.

**Navigation hand-off:** a navigation request is written to `navigation_target.json`; the BLE service reads the file, sends the destination as JSON on characteristic FFE1 and clears the file; the Android app can then search the destination with Photon and request an OSRM route. This is documented source behaviour; a physical voice-to-route test has not been completed.

**Audio and safety boundaries:** helmet speech and media use the helmet speakers, while NavigationActivity uses phone-side spoken guidance. The crash monitor operates the GPIO18 alarm and SIM800L and does not publish an Android crash flag in v1.1.0. The `rhasspy`, `mosquitto`, `helmx_assistant` and `helmx-ble` services are configured to restart automatically, but stable integrated uptime has not been benchmarked.

### 5.5.8 Theft Detection (Proposed, Not Implemented)

Theft detection was part of the original proposal but was not implemented. No voice-PIN, theft-alert or tracking code exists in any HelmX branch or Pi service, and the Android app has no theft or unauthorized-motion alert. The proposed design would capture a spoken PIN through the voice-recognition module, verify it against a securely stored PIN, and on failed authentication raise an alert, capture the GPS location, notify the owner's mobile application and log the event. Implementing secure PIN storage, recognition accuracy optimization, a multi-attempt lockout and tracking integration is future work.

### 5.5.9 Documentation

Documentation produced for the project comprises the proposal, literature review, requirements specification, system architecture, SDLC and sprint documentation, progress reports, this report, and the README files and research records of the five repository branches. Microsoft Word was used for reports, Jira for project management, and draw.io and Figma for diagrams.

## 5.6 Prototype and Application Screenshots

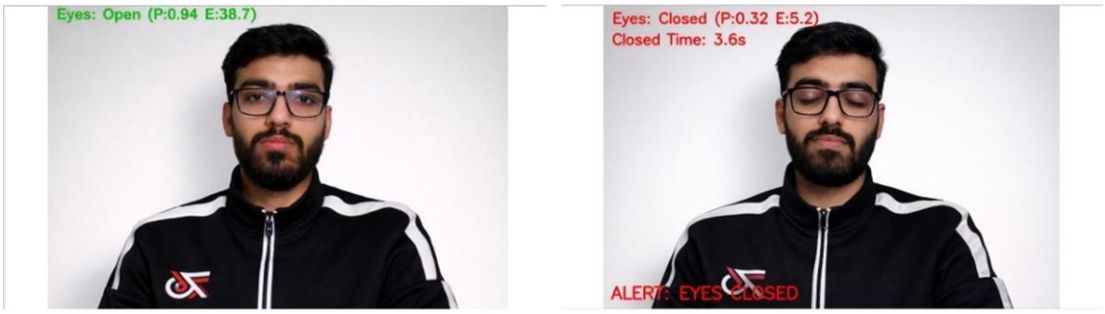
### 5.6.1 Helmet Prototype

Figure 5.2 shows the working prototype: the Raspberry Pi 4 is mounted at the rear of the shell, the IR camera on a raised mount at the top, the USB camera with microphone on the chin bar, and the DHT22 and battery enclosure on the side.



**Figure 5.2:** HelmX working prototype: side view (left) and rear view with the Raspberry Pi 4, top-mounted IR camera and wiring harness (right) [8].

Figure 5.3 shows a live camera demonstration of the eye-state drowsiness prototype. The overlay reports the detected eye state with its score, and an alert is raised once the eyes remain closed for a sustained period (3.6 s in panel b). The demonstration illustrates the alerting behaviour; the quantitative drowsiness results in Section 4.8 refer to the quality-gated three-class model.



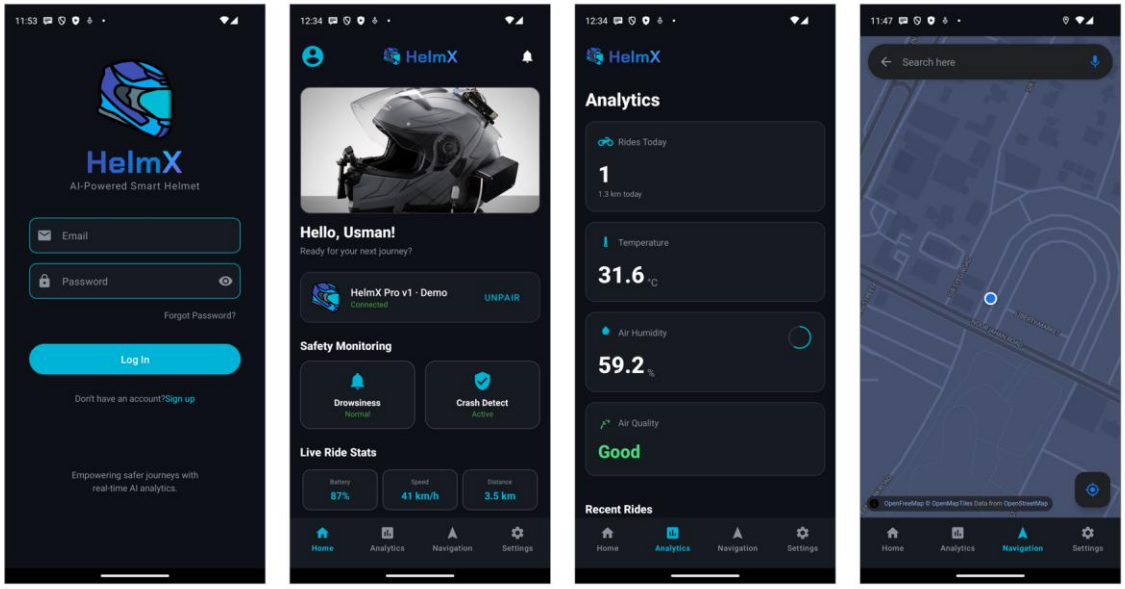

(a) Eyes open (b) Eyes closed for 3.6 s: alert raised

**Figure 5.3:** Drowsiness-detection prototype demonstration: (a) eyes open with an eye-state score of 0.94; (b) eyes closed for 3.6 s, raising an EYES CLOSED alert.

### 5.6.2 Android Application

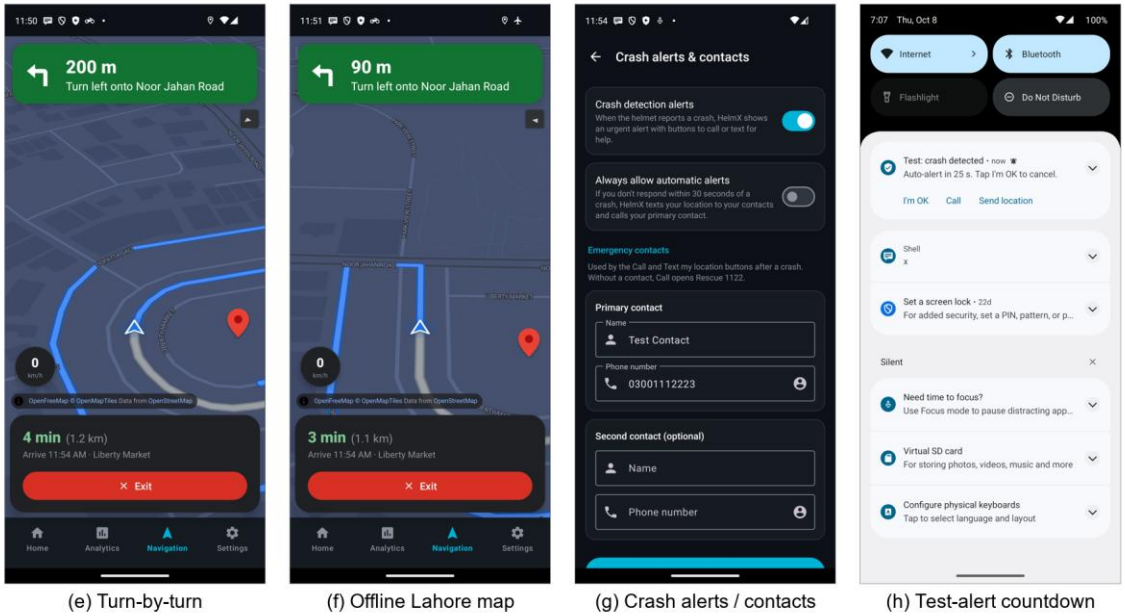
Figure 5.4 and Figure 5.5 show screens of the Android application at commit bce53f7, captured on a Pixel 8 Android 14 emulator with simulated Lahore GPS [6]. The home and

analytics screens display debug demo telemetry, not readings from the physical helmet, and the crash countdown in panel (h) is a non-sending test alert.




(a) Sign-in (b) Home / BLE status (c) Ride analytics (d) Map and search

**Figure 5.4:** Android application (1): (a) email/password sign-in; (b) home screen with BLE helmet status and live ride statistics (demo telemetry); (c) ride analytics with environmental readings; (d) MapLibre map with place search.

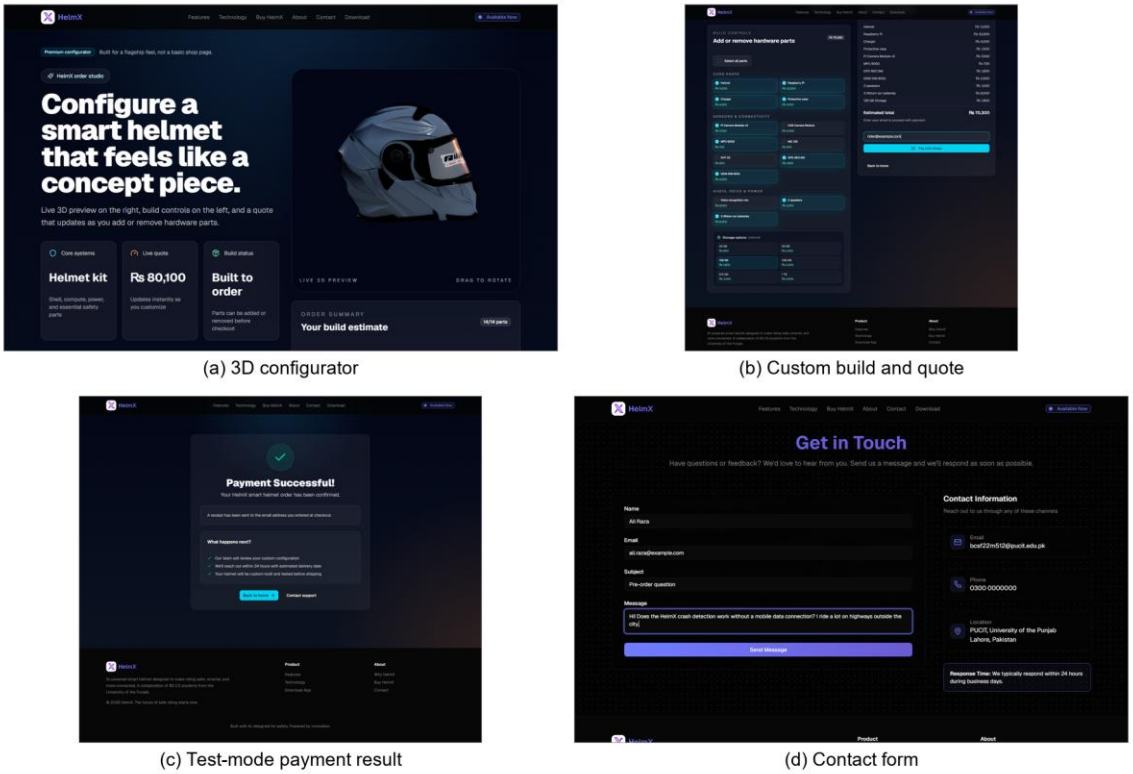



(e) Turn-by-turn (f) Offline Lahore map (g) Crash alerts / contacts (h) Test-alert countdown

**Figure 5.5:** Android application (2): (e) turn-by-turn guidance; (f) offline Lahore District map during active guidance; (g) crash alerts and emergency contacts with the opt-in automatic response; (h) test crash notification with the 30-second countdown and *I'm OK*, *Call* and *Send location* actions.

### 5.6.3 Web Platform

Figure 5.6 shows the deployed web platform at commit 3b4be8e [7]. Payments run in Stripe test mode; the confirmation screens use sample data.




(a) 3D configurator (b) Custom build and quote

(c) Test-mode payment result (d) Contact form

**Figure 5.6:** Web platform: (a) 3D configurator with live preview and quote; (b) custom build with parts, storage and order summary; (c) test-mode payment confirmation; (d) contact form.

## 5.7 Challenges During Implementation

Table 5.3 summarizes the main challenges, their impact, the mitigation applied and the current status.

**Table 5.3:** Implementation challenges

| No. | Challenge | Impact | Mitigation | Status |
|---|---|---|---|---|
| C1 | Scarce and mismatched authentic crash data | Strong offline results can overstate real-world readiness | Hybrid formulation, session-level evaluation, external validation and reporting of negative results | Experimentally stronger; broader real-world validation still required |
| C2 | Deployment fidelity on the target device | Quantization, live streaming and scheduling can silently degrade decisions | Frozen artefacts, measured Pi latency, train–deploy consistency checks and class-stratified fidelity | INT8 deployment functional and measured; verification continues |
| C3 | Weak temporal labels and subtle low vigilance | Clip labels lack onset times; low-vigilance recall 0.54167 | Time-to-first-alert reporting; no onset-latency claim | Open |

| No. | Challenge | Impact | Mitigation | Status |
|---|---|---|---|---|
| C4 | Drowsiness sensitivity versus false warnings | Zero false events only at 4/12 (tuned 0.66) or 2/12 (committed 0.72); higher sensitivity costs 0.49–3.92 false alerts/h | Explicit operating-point frontier; committed default kept separate | Open; external confirmation needed |
| C5 | Missing face data and transferability | Occlusion, visors, illumination and camera placement degrade features | Quality gating and UNAVAILABLE state | Motorcycle-mounted RGB/IR robustness unvalidated |
| C6 | Drowsiness edge readiness | Pi latency, power, thermal behaviour and long-run stability unknown | ONNX export, parity tests and partial INT8 | Pi benchmark pending; no Pi service installed |
| C7 | 3D helmet configurator complexity | Required Three.js/WebGL expertise | Learning React Three Fiber; lazy-loaded 3D scene | Resolved: configurator with 14 parts, six storage tiers and server-side pricing implemented |
| C8 | Backend integration delays | Frontend progressed ahead of the APIs; the telemetry-dashboard assumption no longer fitted the storefront | Mock data and early API contracts; separate PostgreSQL backend | Resolved: checkout, signed webhook and contact APIs implemented |
| C9 | Navigation and offline data | Google Maps SDK plan replaced; public services need connectivity for new searches and routes | MapLibre, Photon and OSRM with bundled Lahore tiles | Implemented; provider fair-use limits remain |
| C10 | Reliable Bluetooth telemetry | Interoperability with the Pi peripheral uncertain | GATT scanning, subscription, parsing, cleanup and reconnection in the app | Pi-side safety-flag publisher not implemented; field link not demonstrated |
| C11 | Background safety and permissions | Automatic SMS and calls need restricted permissions and may be blocked in the background | Opt-in, permission-gated design; non-sending test alerts | Real emergency delivery not established; Play Store policy review needed |
| C12 | Data and user privacy | Personal data in the cloud; third-party routing services receive queries | Cloud Firestore with per-user rules, cached contacts, account deletion | No third-party security assessment |
| C13 | Hardware component selection | Performance, power and budget trade-offs delayed procurement | Component research and modular design | Resolved; all components procured and wired |
| C14 | Power management | Risk of not meeting the three-hour target | Power budgeting; two 10,000 mAh power banks | Runtime not formally measured |
| C15 | Parallel development | Interdependent components caused integration bottlenecks | Stand-ups, early interface contracts, mocks and stubs | Improved through sprint planning |
| C16 | Documentation overhead | Time pressure caused documentation gaps | Dedicated documentation sprints and inline comments | Core documentation complete |
| C17 | Supply chain and procurement delays | Availability and price changes of components | Early ordering, alternative suppliers, contingency budget | Resolved; minor delays absorbed |

CHAPTER 6

# 6 System Conversion

## 6.1 Introduction

System conversion is the transition from development prototypes and simulated environments to an operational setup. For HelmX this means deploying the machine-learning models to the helmet, integrating the hardware, configuring the cloud services used by the applications and establishing communication between modules, so that the system can be evaluated reliably under real riding conditions while maintaining performance, security and usability.

## 6.2 Conversion Method

A **phased conversion** approach was selected because HelmX consists of independent modules (models, mobile app, website, hardware and services); gradual integration reduces risk, allows debugging at each stage and establishes stability before full deployment. The phases are:

1. Deployment of the mobile and web applications.
2. Integration of the machine-learning models (crash and drowsiness detection).
3. Hardware integration (IMU, cameras, GPS, GSM and environmental sensors).
4. Connection of the helmet, phone and cloud services.
5. Testing and optimization under real-world conditions.

## 6.3 Deployment

### 6.3.1 Deployment Steps

1. **Helmet host:** prepare the Raspberry Pi 4 with Debian 12 arm64, enable I²C1 and UART0, UART3 and UART5, connect the sensors and cameras, verify the USB-attached SSD, and install the crash, BLE and voice systemd services following the hardware reference [8].
2. **Crash detection:** deploy the frozen v1.1.0 INT8 TFLite model with its normalizer and thresholds. The local decision path comprises three-window confirmation, the

GPIO18 alarm, the 30-second cancellation window, a GPS fix and a SIM800L SMS attempt; delivered SMS and a BLE crash notification to the phone are not established [4].

3. **Drowsiness detection:** maintain the PyTorch development code and ONNX export; the working software runtime is not equivalent to a validated Pi camera-to-alarm deployment [5].
4. **Android:** build the Kotlin application with BLE GATT, Firebase Authentication and Firestore, and MapLibre/Photon/OSRM navigation, and distribute it as an APK. Offline Lahore map assets are packaged; new routes need internet access. Emulator and demo-telemetry tests do not prove a physical helmet link [6].
5. **Website:** deploy the Next.js application with PostgreSQL/Prisma, SMTP and Stripe sandbox credentials on Vercel. The public demonstration accepts only test payments and makes no fulfilment or financial-certification claim [7].
6. **Evidence:** retain per-branch source hashes, service snapshots and test artefacts, and keep recorded measurements, software implementations and unverified integration paths clearly separated.

### 6.3.2 Data Conversion

IMU preprocessing follows the frozen research pipeline, including physical-unit harmonization and normalization. The drowsiness model uses a personalized five-second neutral calibration and quality-aware causal features; it does not run on the phone. Android data are stored in Cloud Firestore under `users/{uid}` and `users/{uid}/rides` as GPS-derived ride summaries without raw location traces. The website uses separate PostgreSQL Order and ContactSubmission models. No unified telemetry database spanning Firebase, the website and the helmet was implemented.

### 6.3.3 User Training

User-facing setup consists of installing the Android app, signing up with email and password, granting BLE, location and notification permissions, pairing the helmet, adding emergency contacts and, optionally, enabling the automatic phone response. The app provides in-app settings and an About page; it has no separate onboarding tutorial. Riders should understand that the phone's drowsiness sensitivity changes only the phone alarm delay (0, 1.5 or 3 s) and does not alter the helmet model thresholds; that helmet voice navigation is distinct from the phone's spoken instructions; and that the app has no theft

alert. A structured SUS study, real purchase fulfilment or a delivered emergency alert would require evidence not currently available.

## 6.4 Post-Deployment Testing

Table 6.1 lists the post-deployment tests and their status.

**Table 6.1:** Post-deployment testing activities

| Activity | Status |
|---|---|
| Functional testing of all modules | Planned |
| Real-time crash detection validation | Completed on the Raspberry Pi (Section 4.7) |
| Drowsiness software and causal replay evaluation | Completed in software; Pi camera-to-alert validation pending |
| Mobile app performance testing | Not yet performed |
| Firestore and PostgreSQL record verification | Not yet performed |
| Hardware reliability testing | Crash-monitor soaks only |

Only the crash-monitor Raspberry Pi experiments and the documented unit tests have been executed; the remaining activities are planned.

## 6.5 Deployment Challenges

**Table 6.2:** Deployment challenges and responses

| Challenge | Description | Response |
|---|---|---|
| Hardware integration | Synchronizing sensors with software | Modular integration and per-peripheral checks |
| Real-time processing | Latency of model predictions on the Pi | INT8 quantization and a lightweight architecture (measured p99 3.25 ms) |
| Network dependency | Cloud services and routing need connectivity | Local crash/GSM path; offline Lahore map; Firestore offline queue |
| Power consumption | Limited battery life | Resource management; runtime still to be measured |
| Map services | Dependence on third-party routing and geocoding | MapLibre with OpenStreetMap-derived tiles, Photon and OSRM; offline Lahore tiles |

## 6.6 Summary

The integrated prototype demonstrates the specified crash-model capabilities under laboratory, offline and controlled live-sensor conditions. Deployment for unattended real-world safety use requires further false-alert, power, GPS/GSM delivery and real-crash validation, a Pi-side BLE safety-flag publisher, and an on-device drowsiness benchmark.

CHAPTER 7

# 7 Cost Analysis

## 7.1 Introduction

This chapter presents the estimated cost of developing the HelmX prototype, including hardware, supporting equipment, labour and miscellaneous expenses. The aim is to assess financial feasibility and identify opportunities for cost reduction in future production. The figures are the budget prepared for the project, not an audited expenditure statement.

## 7.2 Hardware Cost Breakdown

Table 7.1 lists the budgeted hardware. The entries sum to PKR 81,900. The budget includes three 10,000 mAh power banks, whereas the tested prototype has two, and a Pi Camera Module v2, whereas the fitted camera is a Raspberry Pi IR night-vision CSI camera. The IMU board is labelled MPU-6050, but register readback identified an MPU-6500-family device (WHO_AM_I = 0x70). The table is therefore an estimate, not a verified as-built bill of materials.

**Table 7.1:** Budgeted hardware costs

| Component | Quantity | Unit cost (PKR) | Total (PKR) |
|---|---|---|---|
| Raspberry Pi 4 Model B | 1 | 32,000 | 32,000 |
| Raspberry Pi case | 1 | 1,000 | 1,000 |
| Raspberry Pi charger | 1 | 4,000 | 4,000 |
| Cooling fan and heat sinks | 1 | 1,500 | 1,500 |
| Pi Camera Module v2 (budgeted) | 1 | 7,000 | 7,000 |
| USB camera | 1 | 4,000 | 4,000 |
| MPU-6050 IMU board | 1 | 700 | 700 |
| MQ-135 gas sensor | 1 | 500 | 500 |
| DHT22 temperature and humidity sensor | 1 | 600 | 600 |
| NEO-6M GPS module | 1 | 1,800 | 1,800 |
| SIM800L GSM module | 1 | 2,500 | 2,500 |
| Voice-recognition microphone module | 1 | 6,500 | 6,500 |
| Speakers | 1 set | 1,000 | 1,000 |
| 10,000 mAh power banks | 3 (budgeted) | 2,000 | 6,000 |

| Component | Quantity | Unit cost (PKR) | Total (PKR) |
|---|---|---|---|
| 128 GB storage (SSD in prototype) | 1 | 1,800 | 1,800 |
| Helmet | 1 | 11,000 | 11,000 |
| **Total hardware** | | | **81,900** |

## 7.3 Development and Additional Costs

**Table 7.2:** Development and additional costs

| Category | Cost (PKR) |
|---|---|
| Labour | 12,000 |
| Miscellaneous expenses | 9,100 |
| **Total additional costs** | **21,100** |

## 7.4 Total Cost Estimation

**Table 7.3:** Total estimated prototype cost

| Category | Amount (PKR) |
|---|---|
| Total hardware cost | 81,900 |
| Additional costs (labour and miscellaneous) | 21,100 |
| **Grand total** | **103,000** |

## 7.5 Cost Feasibility

The historical estimate of PKR 103,000 is a prototype-scale figure. In commercial production the cost could be reduced substantially through bulk purchasing, custom printed circuit boards in place of development boards, integration of several sensors into compact modules and further optimization of component selection. The prices in the website's 3D configurator form an independent demonstration catalog and must not be substituted for the prototype's bill of materials. Future cost claims require verified procurement records and a production-volume analysis; the estimate does not demonstrate a commercially manufacturable, safety-certified product.

CHAPTER 8

# 8 Conclusion

## 8.1 Overview

HelmX is a multi-component research prototype that aims to improve motorcycle-rider safety through artificial intelligence, the Internet of Things and mobile and web technologies. It combines crash detection, drowsiness detection, phone-based navigation, an offline voice assistant with photo and short-video capture, and environmental sensing. Continuous dual-camera recording and theft protection remain proposed features.

## 8.2 Achievements

- **Crash detection:** development and Raspberry Pi 4 deployment of a hybrid supervised–reconstruction crash detector. Systematic ablations showed that the supervised-only model is stronger in-distribution, whereas reconstruction provides complementary evidence under a held-out simulated crash mechanism. The INT8 model achieved millisecond-scale live inference (p99 3.25 ms in a 90-minute soak and 3.03 ms in a 180-minute soak), and its deployment fidelity, timing validity and external false-alert exposure are documented in a public reproducibility release [4].
- **Drowsiness detection:** a personalized, three-class, quality-gated MobileNetV3-Small/physiology/causal-TCN research model with subject-disjoint development metrics (macro-F1 0.642), ONNX export and elapsed-time alert control [5].
- **Android application:** a native Kotlin companion app with BLE telemetry parsing, Firebase Authentication and Firestore, an offline Lahore map, online navigation, foreground GPS ride summaries and permission-gated phone alerts; its tests are local and emulator-based, not a physical helmet-to-phone safety trial [6].
- **Web platform:** a responsive Next.js storefront with a 14-part 3D configurator, six storage tiers, server-priced Stripe test checkout, idempotent order transitions, a PostgreSQL schema and contact and receipt workflows; payments run in sandbox mode only [7].
- **Helmet systems:** a documented Raspberry Pi 4 configuration with sensor buses, BLE environmental telemetry, an offline Rhasspy/MQTT/NanoTTS voice assistant,

a UART5 voice module, media-capture commands and the deployed INT8 crash monitor [8].

## 8.3 Limitations

- Authentic motorcycle-crash data are scarce, so crash results rely on benchmark, controlled and external validation datasets rather than natural crashes.
- External-domain false alerts remain material (about 14 per hour on e-scooter data), and guided helmet handling triggered six alarms, so unrestricted field readiness cannot be claimed.
- End-to-end GPS/GSM SMS delivery, battery runtime and GPS accuracy were not measured.
- The drowsiness model has weak video-level labels, a low-vigilance recall of 0.54167, an event sensitivity of 2/12 at the committed threshold (4/12 at the tuned threshold) and no external or target-device confirmation.
- The Pi-side BLE safety-flag publisher is not implemented, so the phone's crash and drowsiness alerts have not been driven by the real helmet.
- The website runs Stripe in test mode, sends receipts without a durable retry and uses an in-memory, per-instance rate limiter.
- High-wind voice recognition, rider usability (SUS) and multi-user scalability were not evaluated.

The two safety subsystems have different evidence levels. The IMU crash detector has measured Raspberry Pi stream and inference validation, with explicitly limited natural-crash and SMS evidence; the drowsiness detector has multi-fold development evaluation and ONNX software tests but lacks independent DROZY confirmation and Raspberry Pi camera-to-alert validation. They must not be presented as sharing a single accuracy, latency or field-readiness claim.

## 8.4 Future Work

- Implement and test a versioned Pi-to-Android BLE safety-event publisher with freshness, reconnection and deduplication semantics, and test the independent phone and GSM alert paths without using live emergency numbers.
- Extend real-world motorcycle validation with larger and more diverse inertial datasets and better-characterized false-alert exposure, and continue improving deployment fidelity and communication robustness.

- Validate the drowsiness model independently on DROZY and on consented helmet-camera data, then measure Raspberry Pi camera-to-alarm latency, robustness to occlusion and visors, thermal behaviour and power.
- Evaluate voice commands under helmet wind noise and concurrent audio, measure battery endurance and GPS accuracy, and run the planned user acceptance study with the System Usability Scale.
- Test web checkout and receipt failure-and-recovery scenarios before any production use, and add support for further languages in the voice module.
- Explore integration with traffic authorities and mature the research prototype into a robust product-grade platform.

## 8.5 Final Remarks

The implementation and experimental evidence establish a multi-component research prototype comprising an experimentally evaluated Raspberry Pi crash monitor, a software-verified drowsiness model, a native Android companion, locally configured voice, environment and camera services, and a separate Stripe test-mode storefront. They do not establish certified accident prevention, delivered emergency notification, validated cross-device crash and drowsiness telemetry, independent motorcycle drowsiness validation or commercial product readiness. Within those boundaries, HelmX shows that an affordable, edge-first smart helmet can be engineered and evaluated transparently, and it provides a reproducible basis for the field validation that must come next.

# References


[1] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: WHO, 2023, ISBN 978-92-4-008651-7.

[2] National Highway Traffic Safety Administration, "Motorcycles: 2023 data," *Traffic Safety Facts*, U.S. Department of Transportation, Washington, DC, USA, 2025.

[3] National Highway Traffic Safety Administration, "Drowsy driving," U.S. Department of Transportation. [Online]. Available: https://www.nhtsa.gov/risky-driving/drowsy-driving (accessed Nov. 2026).

[4] U. Azfar, "HelmX: Reproducibility package for deployment-fidelity validation of a helmet-mounted IMU event-detection policy on Raspberry Pi 4," version 1.1.0, Zenodo, 2026, doi: 10.5281/zenodo.23770989.

[5] U. Azfar, "HelmX driver drowsiness detection: Quality-aware temporal monitoring research prototype," source repository, branch `drowsiness-detection-model`, 2026. [Online]. Available: https://github.com/Usman-Azfar/HelmX-AI-Powered-Smart-Helmet/tree/drowsiness-detection-model

[6] U. Azfar and the HelmX FYP team, "HelmX: AI-powered smart helmet — Android companion application," source code and release documentation, branch `android-app`, commit bce53f7, 2026. [Online]. Available: https://github.com/Usman-Azfar/HelmX-AI-Powered-Smart-Helmet/tree/android-app

[7] HelmX team, "HelmX web platform: Next.js 16 / React 19 storefront, 3D configurator and Stripe test checkout," source code, branch `web-app`, commit 3b4be8e, 2026. [Online]. Available: https://github.com/Usman-Azfar/HelmX-AI-Powered-Smart-Helmet/tree/web-app

[8] HelmX team, "HelmX: AI-powered smart helmet — Raspberry Pi hardware and integration reference," `docs/hardware/PI_SETUP.md` and `BRING_UP.md`, branch `main`, commit 6c14ce1, Oct. 2026. [Online]. Available: https://github.com/Usman-Azfar/HelmX-AI-Powered-Smart-Helmet/tree/main

[9] S. Gelmini, S. Strada, M. Tanelli, S. M. Savaresi, and C. De Tommasi, "Automatic crash detection system for two-wheeled vehicles: Design and experimental validation," *IFAC-PapersOnLine*, vol. 52, no. 5, pp. 498–503, 2019, doi: 10.1016/j.ifacol.2019.09.079.

[10] M. S. Amin, M. B. I. Reaz, S. S. Nasir, and M. A. S. Bhuiyan, "Low cost GPS/IMU integrated accident detection and location system," *Indian Journal of Science and Technology*, vol. 9, no. 10, 2016, doi: 10.17485/ijst/2016/v9i10/80221.

[11] A. Cismas, I. Matei, V. Ciobanu, and G. Casu, "Crash detection using IMU sensors," in *Proc. 21st Int. Conf. Control Systems and Computer Science (CSCS)*, 2017, pp. 672–676, doi: 10.1109/CSCS.2017.103.

[12] S. Lee, B. Koo, and Y. Kim, "Pre-impact fall detection for e-scooter riding using an IMU: Threshold-based, supervised, and unsupervised approaches," *Applied Sciences*, vol. 14, no. 22, Art. no. 10443, 2024, doi: 10.3390/app142210443.

[13] P. Rodegast, S. Maier, J. Kneifl, and J. Fehr, "On using machine learning algorithms for motorcycle collision detection," *Discover Applied Sciences*, vol. 6, Art. no. 326, 2024, doi: 10.1007/s42452-024-06014-w.

[14] S. Gelmini, G. Panzani, and S. M. Savaresi, “Analysis and development of an automatic eCall for motorcycles: A one-class cepstrum approach,” in *Proc. IEEE Intelligent Transportation Systems Conf. (ITSC)*, 2019, doi: 10.1109/ITSC.2019.8916907.

[15] A. J. Mahariba, R. A. Uthra, and G. B. Rajan, “An efficient automatic accident detection system using inertial measurement through machine learning techniques for powered two wheelers,” *Expert Systems with Applications*, vol. 192, Art. no. 116389, 2022, doi: 10.1016/j.eswa.2021.116389.

[16] X. Cai, S. Li, X. Liu, and G. Han, “Vision-based fall detection with multi-task hourglass convolutional auto-encoder,” *IEEE Access*, vol. 8, pp. 44493–44502, 2020, doi: 10.1109/ACCESS.2020.2978249.

[17] Y. Zhou, X. Song, Y. Zhang, F. Liu, C. Zhu, and L. Liu, “Feature encoding with autoencoders for weakly supervised anomaly detection,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 6, pp. 2454–2465, 2022, doi: 10.1109/TNNLS.2021.3086137.

[18] X. Yu, J. Wan, G. An, X. Yin, and S. Xiong, “A novel semi-supervised model for pre-impact fall detection with limited fall data,” *Engineering Applications of Artificial Intelligence*, vol. 132, Art. no. 108469, 2024, doi: 10.1016/j.engappai.2024.108469.

[19] S. S. Khan and B. Taati, “Detecting unseen falls from wearable devices using channel-wise ensemble of autoencoders,” *Expert Systems with Applications*, vol. 87, pp. 280–290, 2017, doi: 10.1016/j.eswa.2017.06.011.

[20] D. Selmanaj, M. Corno, and S. M. Savaresi, “Hazard detection for motorcycles via accelerometers: A self-organizing map approach,” *IEEE Transactions on Cybernetics*, vol. 47, no. 11, pp. 3609–3620, 2017, doi: 10.1109/TCYB.2016.2573321.

[21] F. Attal, A. Boubezoul, A. Samé, L. Oukhellou, and S. Espié, “Powered two-wheelers critical events detection and recognition using data-driven approaches,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 12, pp. 4011–4022, 2018, doi: 10.1109/TITS.2018.2797065.

[22] V. Viswanath *et al.*, “Accident detection and reporting system for powered two-wheelers using Internet of Things and edge AI augmented with MCUSUM features,” in *Proc. Emerging Trends in Industry 4.0 (ETIS)*, 2025, doi: 10.1109/ETIS64005.2025.10961751.

[23] S. Shevtekar, C. Maurya, G. Sil, and S. Das, “Proactive fall detection for powered two-wheelers safety interventions: A cross-dataset benchmark on simulation collision data,” SSRN preprint, 2025, doi: 10.2139/ssrn.5626465.

[24] L. Deng, “Development of a crash detection algorithm for motorcycle drivers using machine learning,” M.Sc. thesis, Chalmers University of Technology, Gothenburg, Sweden, 2021.

[25] S. A. Mian, “Development of a smartphone-based crash notification system for motorcycle drivers using machine learning,” M.Sc. thesis, Chalmers University of Technology, Gothenburg, Sweden, 2021.

[26] L. Kubin, T. Bianconcini, D. C. de Andrade, M. Simoncini, L. Taccari, and F. Sambo, “Deep crash detection from vehicular sensor data with multimodal self-supervision,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12480–12489, 2022, doi: 10.1109/TITS.2021.3114816.

[27] M. I. B. Ahmed, H. Alabdulkarem, F. Alomair, D. Aldossary, M. Alahmari, M. Alhumaidan, S. Alrassan, A. Rahman, M. Youldash, and G. Zaman, “A deep-learning approach to driver drowsiness detection,” *Safety*, vol. 9, no. 3, Art. no. 65, 2023, doi: 10.3390/safety9030065.

[28] C. B. S. Maior, M. J. C. Moura, J. M. M. Santana, and I. D. Lins, “Real-time classification for autonomous drowsiness detection using eye aspect ratio,” *Expert Systems with Applications*, vol. 158, Art. no. 113505, 2020, doi: 10.1016/j.eswa.2020.113505.

[29] E. Magán, M. P. Sesmero, J. M. Alonso-Weber, and A. Sanchis, “Driver drowsiness detection by applying deep learning techniques to sequences of images,” *Applied Sciences*, vol. 12, no. 3, Art. no. 1145, 2022, doi: 10.3390/app12031145.

[30] S. R. Dhole, A. Kashyap, A. N. Dangwal, and R. Mohan, “A novel helmet design and implementation for drowsiness and fall detection of workers on-site using EEG and random-forest classifier,” *Procedia Computer Science*, vol. 151, pp. 947–952, 2019, doi: 10.1016/j.procs.2019.04.132.

[31] W. von Rosenberg, T. Chanwimalueang, V. Goverdovsky, D. Looney, D. Sharp, and D. P. Mandic, “Smart helmet: Wearable multichannel ECG and EEG,” *IEEE Journal of Translational Engineering in Health and Medicine*, vol. 4, Art. no. 2700111, 2016, doi: 10.1109/JTEHM.2016.2609927.

[32] R. Ghoddoosian, M. Galib, and V. Athitsos, “A realistic dataset and baseline temporal model for early drowsiness detection,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 178–187, doi: 10.1109/CVPRW.2019.00027.

[33] R. Fusek, “MRL Eye Dataset,” VSB – Technical University of Ostrava, dataset. [Online]. Available: https://mrl.cs.vsb.cz/eyedataset.html (accessed Nov. 2026).

[34] K. I. Wong, Y.-C. Chen, T.-C. Lee, and S.-M. Wang, “Head motion recognition using a smart helmet for motorcycle riders,” in *Proc. Int. Conf. Machine Learning and Cybernetics (ICMLC)*, 2019, pp. 1–7, doi: 10.1109/ICMLC48188.2019.8949319.

[35] JARVISH Inc., “JARVISH X-AR smart helmet,” product information. [Online]. Available: https://jarvish.com (accessed Nov. 2026).

[36] P. Lee *et al.*, “Trends in smart helmets with multimodal sensing for health and safety: Scoping review,” *JMIR mHealth and uHealth*, vol. 10, no. 11, Art. no. e40797, 2022, doi: 10.2196/40797.

[37] M. Kan, L. Zhang, H. Liang, B. Zhang, M. Fang, D. Liu, S. Shan, and X. Chen, “eLabrador: A wearable navigation system for visually impaired individuals,” *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 12228–12244, 2025, doi: 10.1109/TASE.2025.3541055.

[38] MapLibre, “MapLibre GL JS documentation.” [Online]. Available: https://maplibre.org/maplibre-gl-js/docs/ (accessed Nov. 2026).

[39] Project OSRM, “Open Source Routing Machine,” and Komoot, “Photon geocoder.” [Online]. Available: https://project-osrm.org and https://github.com/komoot/photon (accessed Nov. 2026).

[40] Sena Technologies, “Stryker smart motorcycle helmet,” product information. [Online]. Available: https://www.sena.com (accessed Nov. 2026).

[41] Cardo Systems, “PACKTALK EDGE communication system,” product information. [Online]. Available: https://www.cardosystems.com (accessed Nov. 2026).

[42] Sena Technologies, “10R low-profile Bluetooth communication system,” product information. [Online]. Available: https://www.sena.com (accessed Nov. 2026).

[43] Cardo Systems, “Sound by JBL helmet speakers,” product information. [Online]. Available: https://www.cardosystems.com (accessed Nov. 2026).

[44] Juniper Research, “Number of voice assistant devices in use to overtake world population by 2024, reaching 8.4bn, led by smartphones,” press release, Apr. 2020.

[45] National Insurance Crime Bureau, “Lock up your bikes – motorcycle thefts rise for the third consecutive year,” news release, Des Plaines, IL, USA, Aug. 2023.

[46] D. A. Preetham, M. S. Rohit, A. G. Ghontale, and M. J. P. Priyadarsini, “Safety helmet with alcohol detection and theft control for bikers,” in *Proc. Int. Conf. Intelligent Sustainable Systems (ICISS)*, 2017, pp. 668–673, doi: 10.1109/ISS1.2017.8389255.

[47] K. Sudarsan and P. Kumaraguru, “Helmet for road hazard warning with wireless bike authentication and traffic adaptive MP3 playback,” *International Journal of Science and Research (IJSR)*, vol. 3, no. 3, 2014.

[48] K. Kalkotwar, M. Ukey, S. Nanwatkar, S. Gulwade, Y. Padole, and P. Jumle, “Internet of Things (IoT) based smart helmet and intelligent bike system,” in *Proc. 7th Int. Conf. Trends in Electronics and Informatics (ICOEI)*, 2023, pp. 1458–1461, doi: 10.1109/ICOEI56765.2023.10125884.

[49] A. K. Jain, A. Ross, and S. Prabhakar, “An introduction to biometric recognition,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 14, no. 1, pp. 4–20, 2004, doi: 10.1109/TCSVT.2003.818349.

[50] S. Chandran, S. Chandrasekar, and N. E. Elizabeth, “Konnect: An Internet of Things (IoT) based smart helmet for accident detection and notification,” in *Proc. IEEE Annual India Conf. (INDICON)*, 2016, pp. 1–4, doi: 10.1109/INDICON.2016.7839052.

[51] C. Priya, C. Ramya, N. M. Poovitha, P. S. Rithika, and M. Sujithra, “Smart bike helmet with vehicle tracking system using Arduino,” in *Proc. Int. Conf. Edge Computing and Applications (ICECAA)*, 2022, pp. 579–582, doi: 10.1109/ICECAA55415.2022.9936590.

[52] R. Vashisth, S. Gupta, A. Jain, S. Gupta, Sahil, and P. Rana, “Implementation and analysis of smart helmet,” in *Proc. 4th Int. Conf. Signal Processing, Computing and Control (ISPCC)*, 2017, pp. 111–117, doi: 10.1109/ISPCC.2017.8269660.

[53] A. D. T. Alcantara *et al.*, “Internet of Things-based smart helmet with accident identification and logistics monitoring for delivery riders,” *Engineering Proceedings*, vol. 58, no. 1, Art. no. 129, 2023, doi: 10.3390/ecsa-10-16238.

[54] A. K. Pardeshi, H. Pahuja, and B. Singh, “Development of real time helmet based authentication with smart dashboard for two wheelers,” in *Advances in Intelligent Systems and Computing*. Cham, Switzerland: Springer, 2016, pp. 989–1003, doi: 10.1007/978-3-319-47952-1_79.

[55] M. J. Islam, M. N. Pathan, A. Sultana, and A. Rahman, “An IoT-based smart helmet for riding security and emergency notification,” in *Proc. 6th Int. Conf. Electrical Engineering and Information & Communication Technology (ICEEICT)*, 2024, pp. 1211–1216, doi: 10.1109/ICEEICT62016.2024.10534489.

[56] Android Developers, “Bluetooth Low Energy overview.” [Online]. Available: https://developer.android.com/develop/connectivity/bluetooth/ble (accessed Nov. 2026).

[57] Google, “Firebase Authentication and Cloud Firestore documentation.” [Online]. Available: https://firebase.google.com/docs (accessed Nov. 2026).

# Appendix A: Originality Report

Figure A.1 reproduces the originality report generated for the HelmX project proposal and referred to in the plagiarism-free certificate. It reports an overall similarity index of 3%. The report applies to the proposal document; a similarity check of the final report should be attached if required by the department.

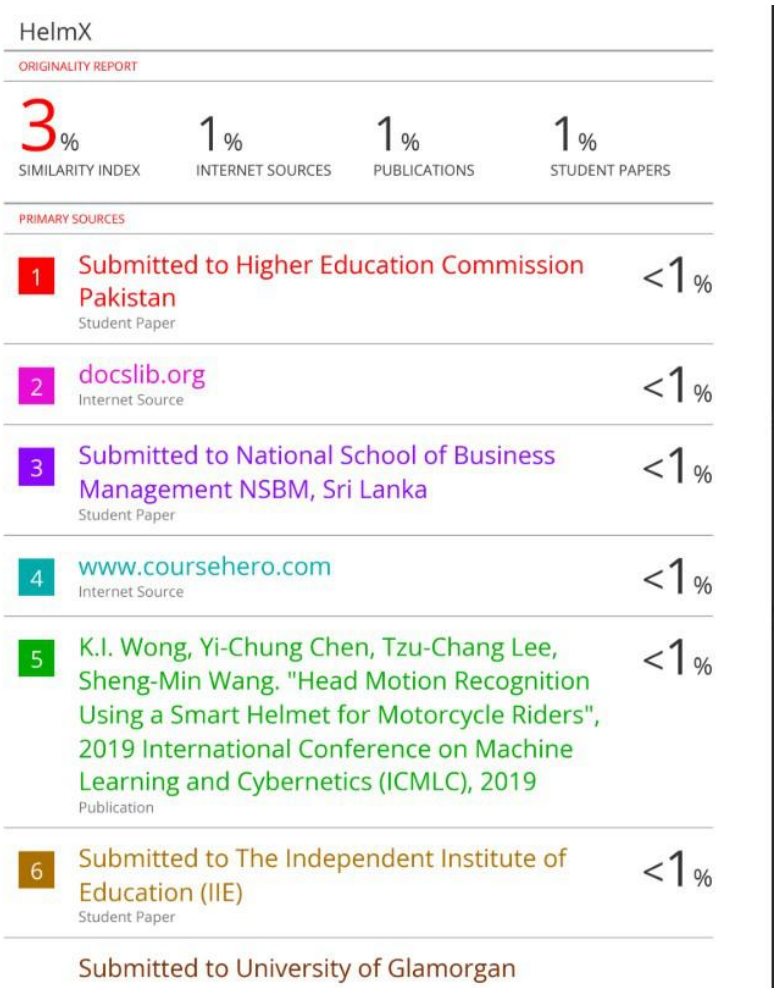



**Figure A.1:** Originality report for the HelmX project proposal (similarity index 3%).

# Appendix B: Repository and Artefact Index

Table B.1 lists the source repositories and archived artefacts on which this report is based. All branches belong to the public repository https://github.com/Usman-Azfar/HelmX-AI-Powered-Smart-Helmet.

**Table B.1:** Source repositories and archived artefacts

| Artefact | Version | Contents |
|---|---|---|
| Crash-detection reproducibility release | v1.1.0, DOI 10.5281/zenodo.23770989 | Frozen hybrid crash model, evaluation scripts, Raspberry Pi evidence and provenance records [4] |
| `crash-detection` branch | commit 63fd5c9 | Crash model, Raspberry Pi runtime (`deploy/crash_monitor.py`) and evaluation suite |
| `drowsiness-detection-model` branch | commit 282c911 | Drowsiness model, training and calibration code, ONNX export, live camera runtime and verification snapshot [5] |
| `android-app` branch | commit bce53f7 (APK v1.0.0) | Kotlin companion application, 41 unit tests and screenshots [6] |
| `web-app` branch | commit 3b4be8e | Next.js storefront, Prisma schema, Vitest tests and screenshots [7] |
| `main` branch | commit 6c14ce1 | Project overview, Raspberry Pi setup and bring-up guides and prototype photographs [8] |